

Problem Set for Strength of Materials

Edited by James Lord and Jacob Grohs

2024-08-27

Table of contents

Welcome to the Problem Set for Strength of Materials	9
How to Use This Resource	9
Copyright Information	9
ACKNOWLEDGMENTS	10
I Problem Submission App	13
II Chapter 1 Problems	14
Problem 1.1 - Equilibrium in 2D	15
Problem 1.2 - Equilibrium in 2D	18
Problem 1.3 - Equilibrium in 2D	21
Problem 1.4 - Internal Reactions	25
Problem 1.5 - Internal Reactions	29
Problem 1.6 - Internal Reactions	33
Problem 1.7 - Equilibrium & Reactions in 3D	37
Problem 1.8 - Equilibrium & Reactions in 3D	41
Problem 1.9	45
III Chapter 2 Problems	50
Problem 2.1 - Average Normal Stress	51
Problem 2.2 - Average Normal Stress	56
Problem 2.3 - Average Normal Stress	61

Problem 2.4 - Average Normal Stress	66
Problem 2.5 - Average Normal Stress	70
Problem 2.6 - Average Normal Stress	74
Problem 2.7 - Average Normal Stress	77
Problem 2.8 - Average Normal Stress	81
Problem 2.9 - Average Normal Stress	85
Problem 2.10 - Average Normal Stress	89
Problem 2.11 -Average Normal Stress	93
Problem 2.12 -Average Normal Stress	96
Problem 2.13 - Average Normal Stress	100
Problem 2.14 - Average Normal Stress	104
Problem 2.15 - Average Normal Stress	109
Problem 2.16 - Average Normal Stress	113
Problem 2.17 - Average Normal Stress	117
Problem 2.21 - Average Shear Stress	121
Problem 2.22 - Average Shear Stress	126
Problem 2.23 - Average Shear Stress	130
Problem 2.24 - Average Shear Stress	134
Problem 2.25 - Average Shear Stress	139
Problem 2.26 - Average Shear Stress	143
Problem 2.27 - Average Shear Stress	147
Problem 2.28 - Average Shear Stress	152
Problem 2.29 - Average Shear Stress	157
Problem 2.30 - Average Shear Stress	161

Problem 2.31 - Average Shear Stress	165
Problem 2.32 - Average Shear Stress	168
Problem 2.33 - Average Shear Stress	173
Problem 2.38 - Bearing Stress	177
Problem 2.39 - Bearing Stress	181
Problem 2.40 - Bearing Stress	186
Problem 2.41 - Bearing Stress	190
Problem 2.42 - Bearing Stress	194
Problem 2.43 - Bearing Stress	198
Problem 2.47 - Stress on an Inclined Plane	202
Problem 2.48 - Stress on an Inclined Plane	205
 IV Chapter 3 Problems	 209
Problem 3.1 - Normal Strain	210
Problem 3.2 - Normal Strain	214
Problem 3.3 - Normal Strain	218
Problem 3.7 - Shear Strain	221
Problem 3.8 - Shear Strain	225
Problem 3.9 - Shear Strain	229
Problem 3.10 - Shear Strain	234
 V Chapter 4 Problems	 238
Problem 4.2 - Stress/Strain Diagram	239
Problem 4.5 - Hooke's Law	243
Problem 4.6 - Hooke's Law	247

Problem 4.7 - Hooke's Law	251
Problem 4.10 - Poisson's Ratio	254
Problem 4.11 - Poisson's Ratio	257
Problem 4.12 - Poisson's Ratio	261
Problem 4.17 - Shear Stress/Strain	264
Problem 4.18 - Thermal Strain	268
Problem 4.19 - Thermal Strain	271
Problem 4.20 - Thermal Strain	274
Problem 4.21 - Thermal Strain	278
Problem 4.23 - Multiaxial Hooke's Law	282
Problem 4.24 - Multiaxial Hooke's Law	287
Problem 4.25 - Multiaxial Hooke's Law	292
Problem 4.26 - Multiaxial Hooke's Law	296
Problem 4.27 - Multiaxial Hooke's Law	300
Problem 4.28 - Multiaxial Hooke's Law	304
Problem 4.29 - Multiaxial Hooke's Law	308
Problem 4.30 - Multiaxial Hooke's Law	312
Problem 4.31 - Multiaxial Hooke's Law	316
Problem 4.35 - Allowable Stress/Safety Factor	321
 VI Chapter 5 Problems	 325
Problem 5.1 - Axial Stress	326
Problem 5.2 - Axial Stress	329
Problem 5.6 - Stress Concentrations	332

Problem 5.7 - Stress Concentrations	335
Problem 5.8 - Stress Concentrations	338
Problem 5.9 - Stress Concentrations	341
Problem 5.10 - Stress Concentrations	344
Problem 5.11 - Axial Deformation	348
Problem 5.12 - Axial Deformation	351
Problem 5.13 - Axial Deformation	355
Problem 5.14 - Axial Deformation	359
Problem 5.15 - Axial Deformation	363
Problem 5.16 - Axial Deformation	367
Problem 5.17 - Axial Deformation	371
Problem 5.18 - Axial Deformation	375
Problem 5.19 - Axial Deformation	378
Problem 5.20 - Axial Deformation	382
Problem 5.27 - Deformation in Systems of Bars	386
Problem 5.28 - Deformation in Systems of Bars	390
Problem 5.33 - Deformation in Systems of Bars	394
Problem 5.34 - Statically Indeterminate Axial Loads	398
Problem 5.35 - Statically Indeterminate Axial Loads	402
Problem 5.36 - Statically Indeterminate Axial Loads	406
Problem 5.37 - Statically Indeterminate Axial Loads	410
Problem 5.38 - Statically Indeterminate Axial Loads	414
Problem 5.39 - Statically Indeterminate Axial Loads	418
Problem 5.40 - Statically Indeterminate Axial Loads	422

Problem 5.41 - Statically Indeterminate Axial Loads	426
Problem 5.42 - Statically Indeterminate Axial Loads	430
Problem 5.43 - Statically Indeterminate Axial Loads	434
Problem 5.44 - Statically Indeterminate Axial Loads	439
Problem 5.45 - Statically Indeterminate Axial Loads	443
Problem 5.50 - Thermal Deformation	447
Problem 5.51 - Thermal Deformation	450
Problem 5.52 - Thermal Deformation	454
Problem 5.53 - Thermal Deformation	458
Problem 5.54 - Thermal Deformation	462
Problem 5.55 - Thermal Deformation	466
Problem 5.56 - Thermal Deformation	471
Problem 5.57 - Thermal Deformation	475
Problem 5.58 - Thermal Deformation	478
Problem 5.59 - Thermal Deformation	481
Problem 5.60 - Thermal Deformation	485
Problem 5.61 - Thermal Deformation	488
Problem 5.62 - Thermal Deformation	493
Problem 5.63 - Thermal Deformation	497
Problem 5.64 - Thermal Deformation	500
Problem 5.65 - Thermal Deformation	504
Problem 5.66 - Thermal Deformation	507
Problem 5.67 - Thermal Deformation	511
Problem 5.68 - Thermal Deformation	515

Problem 5.69 - Thermal Deformation	519
Problem 5.70 - Thermal Deformation	523
Problem 5.71 - Thermal Deformation	527
Problem 5.72 - Thermal Deformation	531
Problem 5.73 - Thermal Deformation	536
Problem 5.74 - Thermal Deformation	541

Welcome to the Problem Set for Strength of Materials

How to Use This Resource

Note: This version of the problem set is a review copy for in-class piloting. This is not the final, published version of the problem set.

The navigation bar of this site will allow you to browse to the specific problems you want to view and solve. The Problem Submission App enables you to create your own unique problem used to solve homework problems. Once your problem is generated, you will be able to submit an answer to the problem and receive instantaneous right/wrong feedback before attempting another solution. At the end of your work, you can download a file log of your submissions to share with your instructor. The problem generation process is based on the ID number you use and so if you attempt the problem from a different computer or at a different time, the variables generated for you will be stable across these runs. However, the log downloader is specific to your current session and so attempting the problem across devices does not save attempt across devices

Additionally, there are versions of the problems with fixed variables that do not use the Problem Submission App. These are available to you also in the sidebar should you want a more traditional textbook listing of problems.

If you have any questions or issues while using this system, you are encouraged to contact your instructor and they will help work through the issues with the developers.

Copyright Information

Note: This version of the problem set is being slightly modified by Greg Teichert so that the correct answer is not given to the student and the problem interface is simplified, removing the ID input (a fixed value is used) and option to download, and addition to corrections to problems.

©2024. James Lord and Jacob Grohs, eds., Problem Set for Strength of Materials is licensed under a Creative Commons Attribution NonCommercial ShareAlike 4.0 International License, except where otherwise noted.

You are free to copy, share, adapt, remix, transform, and build on the material for any purpose as long as you follow the terms of the license: <https://creativecommons.org/licenses/by-nc-sa/4.0>.

You must:

- **Attribute**—You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the [same license](#) as the original.

You may not:

- **NonCommercial** — You may not use the material for [commercial purposes](#).
- **Additional restrictions**—You may not add any legal terms or technological measures that legally restrict others from doing anything the license permits.

ACKNOWLEDGMENTS

The team gratefully acknowledges the generosity of Kurt Gramoll. Many of the problems were adapted from Kurt Gramoll's collection in the [Apple Store](#) and [Google Play](#) and released with permission under a CC BY NC SA 4.0 license.

Problem Set Editor and Content Coordinator

James K. Lord, Associate Collegiate Professor, Virginia Tech

Authors of Problems

Kurt Gramoll, Emeritus Professor, University of Oklahoma

James K. Lord, Mechanical Engineering, Virginia Tech

Chris Galitz, Mechanical Engineering, Virginia Tech

Frances Davis, Department of Engineering, Brightpoint Community College

Sneha Davison. Mechanical Engineering, Virginia Tech

Richard Kent, Department of Mechanical & Aerospace Engineering, University of Virginia

Nina Lord, Department of Engineering, North Virginia Community College

Amy Richardson, Dept of Engineering, North Virginia Community College / Engineering Education, Virginia Tech

Joao Soares, Department of Mechanical and Nuclear Engineering, Virginia Commonwealth University

Sevak Tahmasian, Mechanical Engineering, Virginia Tech

Figure Design of Problems

Kindred Grey, Graphic Design and Production Specialist, University Libraries, Virginia Tech

Software Editor

Jacob Grohs, Associate Professor, Engineering Education, Virginia Tech

Review, Problem Redevelopment, and Programming

James K. Lord, Associate Collegiate Professor, Mechanical Engineering, Virginia Tech

Olivia Ryan, Doctoral Student, Engineering Education, Virginia Tech

Benjamin E. Chaback, Doctoral Student, Engineering Education, Virginia Tech

Layout and Design

Kindred Grey, Graphic Design and Production Specialist, University Libraries, Virginia Tech

Project Coordination and Management

Anita Walz, Associate Professor, University Libraries, Virginia Tech

FUNDING

This project was made possible in part through funding from an Open Course Grant from VIVA (The Virtual Library of Virginia) and the Open Education Initiative of the University Libraries at Virginia Tech.

Suggested citation: James Lord and Jacob Grohs, eds. (2024). Problem Set for Strength of Materials. Blacksburg: Virginia Tech Publishing. <https://engineeringmechanicsoer.github.io/StrengthOfMaterialsExercises>. Licensed with CC BY NC-SA 4.0 <https://creativecommons.org/licenses/by-nc-sa/4.0>.

Publisher: This work is published by the Department of Mechanical Engineering and Department of Engineering Education at Virginia Tech in association with the Open Education Initiative and Virginia Tech Publishing.

Virginia Tech Department of Mechanical Engineering

445 Goodwin Hall, 635 Prices Fork Road

Blacksburg,, VA 24061 USA

<https://me.vt.edu>

Department of Engineering Education at Virginia Tech

345 Goodwin Hall, 635 Prices Fork Rd,
Blacksburg, VA 24061 USA

<https://enge.vt.edu>

Open Education Initiative

University Libraries at Virginia Tech
560 Drillfield Drive Blacksburg, VA 24061 USA

openeducation@vt.edu

Virginia Tech Publishing

University Libraries at Virginia Tech
560 Drillfield Drive Blacksburg, VA 24061 USA

<https://publishing.vt.edu> | publishing@vt.edu

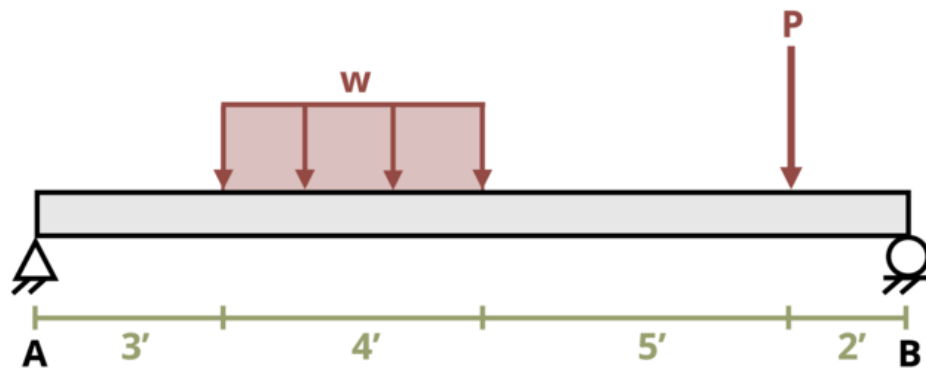
Part I

Problem Submission App

Part II

Chapter 1 Problems

Problem 1.1 - Equilibrium in 2D



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "641"
P = reactive.Value("__")
w = reactive.Value("__")
```

```

attempts = ["Timestamp,Attempt,Answer1,Answer2,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of kip", placeholder="Please enter your
    ui.input_text("answer2", "Your Answer 2 in units of kip", placeholder="Please enter your
    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"The simply-supported beam is subjected to the loading shown. Determine the
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        P.set(random.randrange(5,50,1))
        w.set(random.randrange(20,100,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        ) # Increment the attempt counter on each submission.

```



```

# Calculate the instructor's answer and determine if the user's answer is correct.
instr1 = 4*w()+P()-(20*w()+12*P())/14
instr2 = (20*w()+12*P())/14

correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)

if correct1 and correct2:
    check = f"{'both correct.'}"
elif correct1:
    check = f"{'correct for answer 1 and incorrect for answer 2.'}"
elif correct2:
    check = f"{'incorrect for answer 1 and correct for answer 2.'}"
else:
    check = f"{'both incorrect.'}"

correct_indicator = "JL" if correct1 and correct2 else "JG"

random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

#session.encoded_attempt = reactive.Value(encoded_attempt)
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

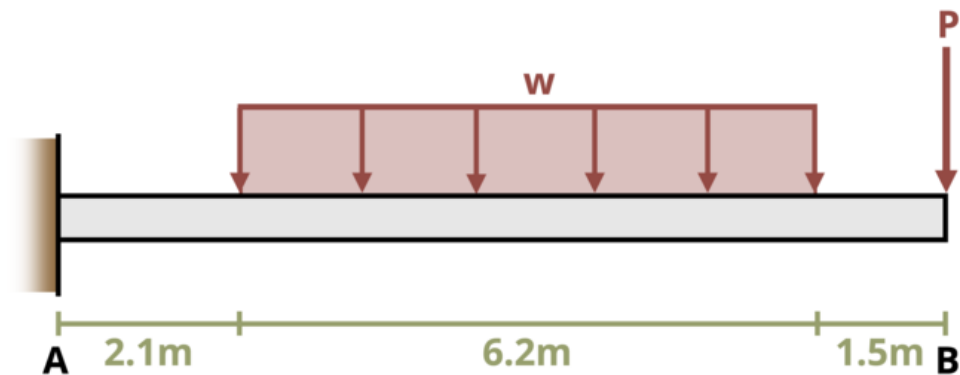
feedback = ui.markdown(f"Your answers of {input.answer1()} and {input.answer2()} are
m = ui.modal(feedback, title="Feedback", easy_close=True)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"
    yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
    for attempt in attempts[1:]:
        await asyncio.sleep(0.25)
        yield attempt

app = App(app_ui, server)

```

Problem 1.2 - Equilibrium in 2D



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "642"
P = reactive.Value("__")
w = reactive.Value("__")
```

```

attempts = ["Timestamp,Attempt,Answer1,Answer2,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of kN", placeholder="Please enter your answer"),
    ui.input_text("answer2", "Your Answer 2 in units of kN-m", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"The cantilever beam is subjected to the loading shown. Determine the reaction at support A."
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        P.set(random.randrange(5,30,1))
        w.set(random.randrange(20,50,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        ) # Increment the attempt counter on each submission.

```

```

# Calculate the instructor's answer and determine if the user's answer is correct.
instr1 = P()+w()*6.2
instr2 = P()*(2.1+6.2+1.5)+w()*6.2*5.2

correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)

if correct1 and correct2:
    check = f"{'both correct.'}"
elif correct1:
    check = f"{'correct for answer 1 and incorrect for answer 2.'}"
elif correct2:
    check = f"{'incorrect for answer 1 and correct for answer 2.'}"
else:
    check = f"{'both incorrect.'}"

correct_indicator = "JL" if correct1 and correct2 else "JG"

random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

#session.encoded_attempt = reactive.Value(encoded_attempt)
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

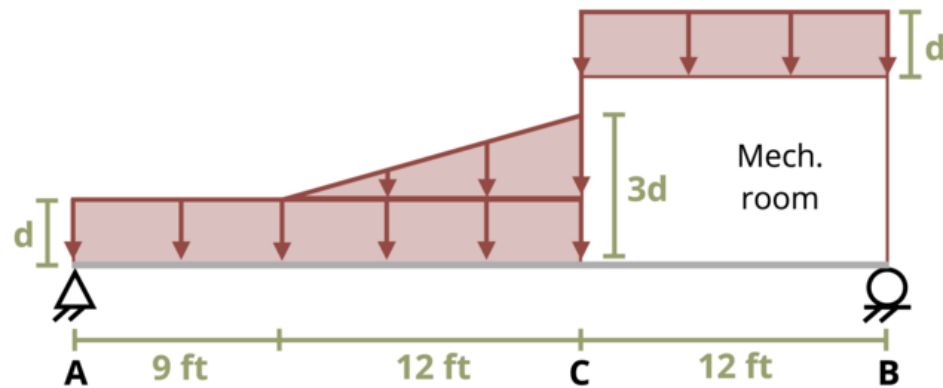
feedback = ui.markdown(f"Your answers of {input.answer1()} and {input.answer2()} are
m = ui.modal(feedback, title="Feedback", easy_close=True)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"
    yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
    for attempt in attempts[1:]:
        await asyncio.sleep(0.25)
        yield attempt

app = App(app_ui, server)

```

Problem 1.3 - Equilibrium in 2D



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "643"
d = reactive.Value("__")
w = reactive.Value("__")
```

```

attempts = ["Timestamp,Attempt,Answer1,Answer2,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of kip", placeholder="Please enter your answer 1"),
    ui.input_text("answer2", "Your Answer 2 in units of kip", placeholder="Please enter your answer 2"),
    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"The simply-supported roof beam AB is subjected to snow loads as a uniform load of 10 k/ft over the entire span."
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        d.set(random.randrange(10,30,1))
        w.set(random.randrange(40,80,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        ) # Increment the attempt counter on each submission.

```

```

# Calculate the instructor's answer and determine if the user's answer is correct.
w1 = w()*12*d()
w2 = w()*21*d()
w3 = 0.5*w()*2*d()*12
instr1 = (w1+w2+w3-(10.5*w2+17*w3+21*w1/2+33*w1/2)/33)/1000
instr2 = ((10.5*w2+17*w3+21*w1/2+33*w1/2)/33)/1000

correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)

if correct1 and correct2:
    check = f"{'both correct.'}"
elif correct1:
    check = f"{'correct for answer 1 and incorrect for answer 2.'}"
elif correct2:
    check = f"{'incorrect for answer 1 and correct for answer 2.'}"
else:
    check = f"{'both incorrect.'}"

correct_indicator = "JL" if correct1 and correct2 else "JG"

random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

#session.encoded_attempt = reactive.Value(encoded_attempt)
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

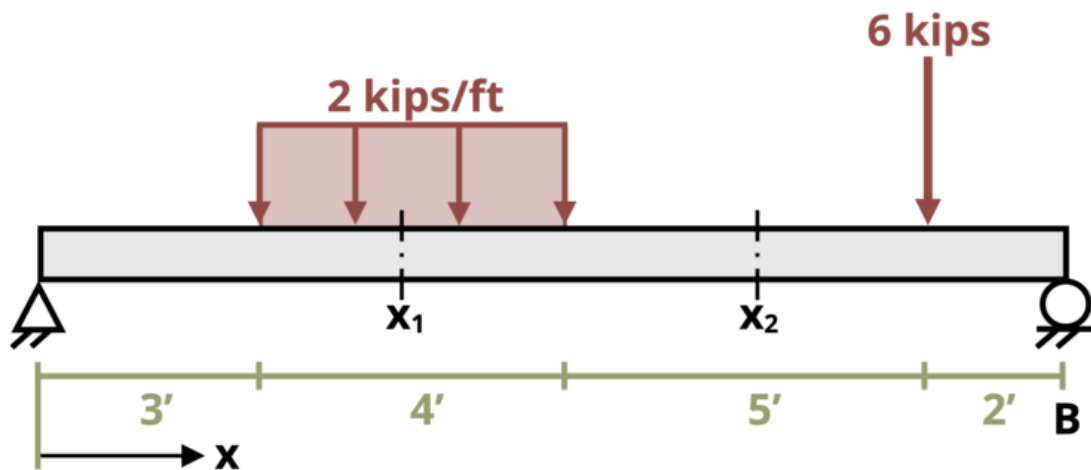
feedback = ui.markdown(f"Your answers of {input.answer1()} and {input.answer2()} are
m = ui.modal(feedback, title="Feedback", easy_close=True)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"
    yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
    for attempt in attempts[1:]:
        await asyncio.sleep(0.25)
        yield attempt

```

```
app = App(app_ui, server)
```


Problem 1.4 - Internal Reactions



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "644"
```

```

x1 = reactive.Value("__")
x2 = reactive.Value("__")
P = reactive.Value("__")
w = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer1,Answer2,Answer3,Answer4,Answer5,Answer6,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of kip", placeholder="Please enter your
    ui.input_text("answer2", "Your Answer 2 in units of kip", placeholder="Please enter your
    ui.input_text("answer3", "Your Answer 3 in units of kip-ft", placeholder="Please enter y
    ui.input_text("answer4", "Your Answer 4 in units of kip", placeholder="Please enter your
    ui.input_text("answer5", "Your Answer 5 in units of kip", placeholder="Please enter your
    ui.input_text("answer6", "Your Answer 6 in units of kip-ft", placeholder="Please enter y

    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"The simply-supported beam is subjected to the loading shown. Determine the
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        x1.set(random.randrange(41,69,1)/10)

```

```

x2.set(random.randrange(75,115,1)/10)
P.set(6) #random.randrange(5,25,1))
w.set(2) #random.randrange(20,50,1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

    # Calculate the instructor's answer and determine if the user's answer is correct.
    instr1 = 0
    Ay = (P()*2+w()*4*9)/14
    instr2 = Ay-w()*(x1()-3)
    instr3 = w()*(x1()-3)*(3+(x1()-3)/2)+instr2*x1()
    instr4=0
    instr5= Ay-w()*4
    instr6= w()*4*5+instr5*x2()

    correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
    correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)
    correct3 = math.isclose(float(input.answer3()), instr3, rel_tol=0.01)
    correct4 = math.isclose(float(input.answer4()), instr4, rel_tol=0.01)
    correct5 = math.isclose(float(input.answer5()), instr5, rel_tol=0.01)
    correct6 = math.isclose(float(input.answer6()), instr6, rel_tol=0.01)

    if correct1 and correct2 and correct3 and correct4 and correct5 and correct6:
        check = "All answers are correct."
    else:
        conditions = []
        for i, correct in enumerate([correct1, correct2, correct3, correct4, correct5, c
            if correct:
                conditions.append(f"correct for answer {i}")
            else:
                conditions.append(f"incorrect for answer {i}")
        check = "; ".join(conditions) + "."

    correct_indicator = "JL" if correct1 and correct2 and correct3 and correct4 and correct5 and correct6

    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)

```

```

random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

#session.encoded_attempt = reactive.Value(encoded_attempt)
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

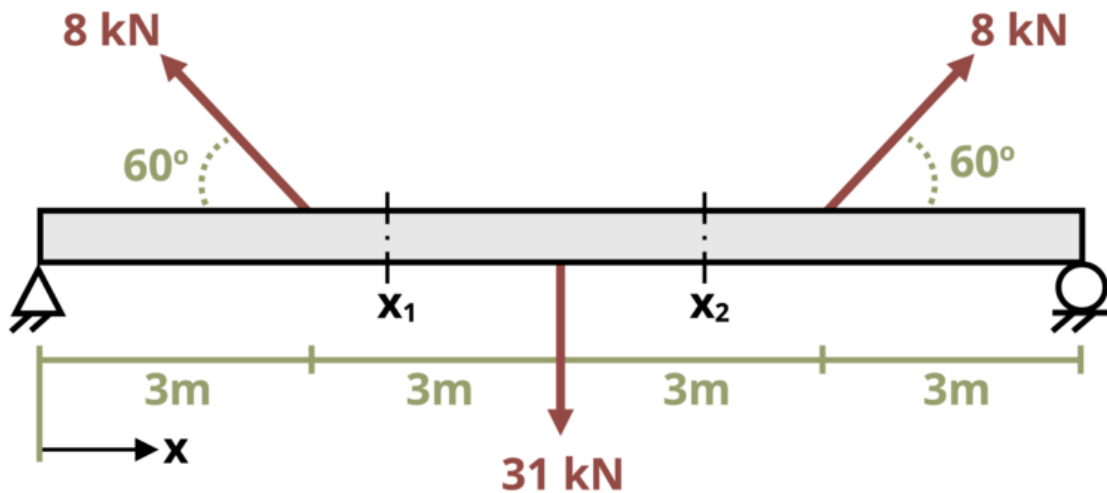
feedback = ui.markdown(f"Your answers of {input.answer1()}, {input.answer2()}, {input
m = ui.modal(feedback, title="Feedback", easy_close=True)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"
    yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
    for attempt in attempts[1:]:
        await asyncio.sleep(0.25)
        yield attempt

app = App(app_ui, server)

```

Problem 1.5 - Internal Reactions



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```

problem_ID = "645"
x = reactive.Value("__")
F1 = reactive.Value("__")
F2 = reactive.Value("__")
F3 = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer1,Answer2,Answer3,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of kN", placeholder="Please enter your answer"),
    ui.input_text("answer2", "Your Answer 2 in units of kN", placeholder="Please enter your answer"),
    ui.input_text("answer3", "Your Answer 3 in units of kN-m", placeholder="Please enter your answer"),

    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"The simply-supported beam is subjected to the loading shown. Determine the reactions."
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        x.set(6.8) #random.randrange(61,89,1)/10)
        F1.set(8) #random.randrange(5,15,1))
        F2.set(31) #random.randrange(20,40,1))

```

```

F3.set(8) #random.randrange(5,15,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

    # Calculate the instructor's answer and determine if the user's answer is correct.
    Ax = F1()*math.cos(math.pi/3)-F3()*math.cos(math.pi/3)
    Ay = (-F1()*math.sin(math.pi/3)*9-F3()*math.sin(math.pi/3)*3+F2()*6)/12
    instr1 = F1()*math.cos(math.pi/3)
    instr2 = Ay+F1()*math.sin(math.pi/3)-31
    instr3 = -F1()*math.sin(math.pi/3)*3+instr2*x()+31*6

    correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
    correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)
    correct3 = math.isclose(float(input.answer3()), instr3, rel_tol=0.01)

    if correct1 and correct2 and correct3:
        check = "All answers are correct."
    else:
        conditions = []
        for i, correct in enumerate([correct1, correct2, correct3], start=1):
            if correct:
                conditions.append(f"correct for answer {i}")
            else:
                conditions.append(f"incorrect for answer {i}")
        check = "; ".join(conditions) + "."

    correct_indicator = "JL" if correct1 and correct2 and correct3 else "JG"

    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    #session.encoded_attempt = reactive.Value(encoded_attempt)
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

    feedback = ui.markdown(f"Your answers of {input.answer1()}, {input.answer2()}, and {

```

```

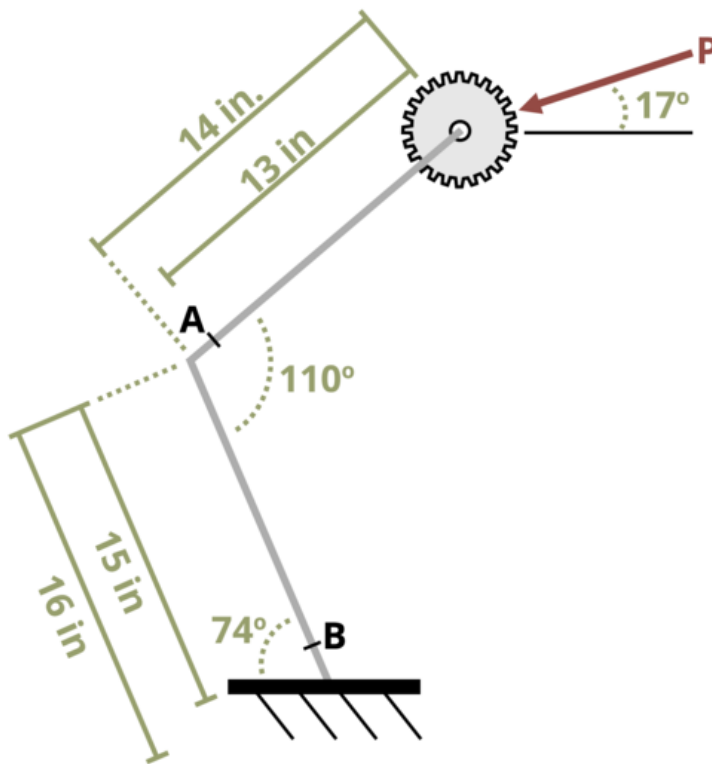
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"
    yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
    for attempt in attempts[1:]:
        await asyncio.sleep(0.25)
        yield attempt

app = App(app_ui, server)

```


Problem 1.6 - Internal Reactions



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
```

```

import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "646"
P = reactive.Value("__")
theta = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer1,Answer2,Answer3,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of lb", placeholder="Please enter your answer"),
    ui.input_text("answer2", "Your Answer 2 in units of lb", placeholder="Please enter your answer"),
    ui.input_text("answer3", "Your Answer 3 in units of lb-in", placeholder="Please enter your answer"),

    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"A gear shift, once engaged, can be modeled as the bent cantilever beam shown below."
            )
        ]

    #@reactive.Effect

```

```

#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    P.set(random.randrange(20,75,1))
    theta.set(random.randrange(10,25,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

    # Calculate the instructor's answer and determine if the user's answer is correct.
    instr1 = abs(P()*math.cos((70+theta())*math.pi/180))
    instr2 = abs(P()*math.sin((70+theta())*math.pi/180))
    instr3 = abs(P()*math.sin((70+theta())*math.pi/180)*(15+14*math.cos(70*math.pi/180))

    correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
    correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)
    correct3 = math.isclose(float(input.answer3()), instr3, rel_tol=0.01)

    if correct1 and correct2 and correct3:
        check = "All answers are correct."
    else:
        conditions = []
        for i, correct in enumerate([correct1, correct2, correct3], start=1):
            if correct:
                conditions.append(f"correct for answer {i}")
            else:
                conditions.append(f"incorrect for answer {i}")
        check = "; ".join(conditions) + "."

    correct_indicator = "JL" if correct1 and correct2 and correct3 else "JG"

    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    #session.encoded_attempt = reactive.Value(encoded_attempt)
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

```

```

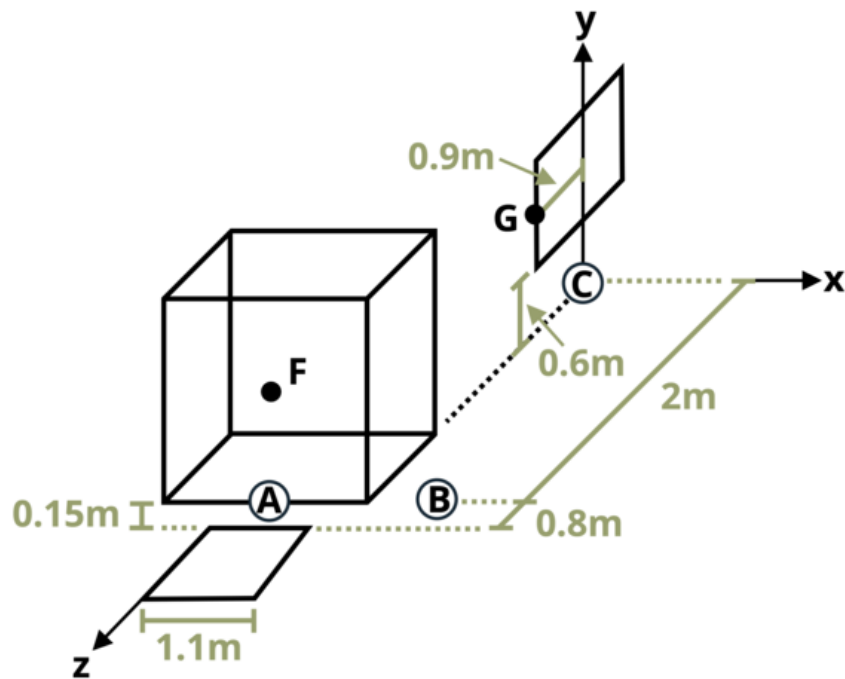
        feedback = ui.markdown(f"Your answers of {input.answer1()}, {input.answer2()}, and {input.answer3()}.")
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"
    yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
    for attempt in attempts[1:]:
        await asyncio.sleep(0.25)
        yield attempt

app = App(app_ui, server)

```

Problem 1.7 - Equilibrium & Reactions in 3D



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```

```

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "647"
W = reactive.Value("__")
P = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer1,Answer2,Answer3,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of kN", placeholder="Please enter your a
    ui.input_text("answer2", "Your Answer 2 in units of kN", placeholder="Please enter your a
    ui.input_text("answer3", "Your Answer 3 in units of kN", placeholder="Please enter your a

    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"A 3-wheeled forklift weighing  $W = \{W()\}$  kN centered at G is carrying an im
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        P.set(random.randrange(100,200,1)/10)

```

```

W.set(random.randrange(10,30,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

    # Calculate the instructor's answer and determine if the user's answer is correct.
    instr1 = ((0.6*W()+2.8*P())/2+(0.15*P())/0.55)/2
    instr2 = (0.6*W()+2.8*P())/2-instr1
    instr3 = P()+W()-instr1-instr2

    correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
    correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)
    correct3 = math.isclose(float(input.answer3()), instr3, rel_tol=0.01)

    if correct1 and correct2 and correct3:
        check = "All answers are correct."
    else:
        conditions = []
        for i, correct in enumerate([correct1, correct2, correct3], start=1):
            if correct:
                conditions.append(f"correct for answer {i}")
            else:
                conditions.append(f"incorrect for answer {i}")
        check = "; ".join(conditions) + "."

    correct_indicator = "JL" if correct1 and correct2 and correct3 else "JG"

    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    #session.encoded_attempt = reactive.Value(encoded_attempt)
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

    feedback = ui.markdown(f"Your answers of {input.answer1()}, {input.answer2()}, and {
    m = ui.modal(feedback, title="Feedback", easy_close=True)
    ui.modal_show(m)

```

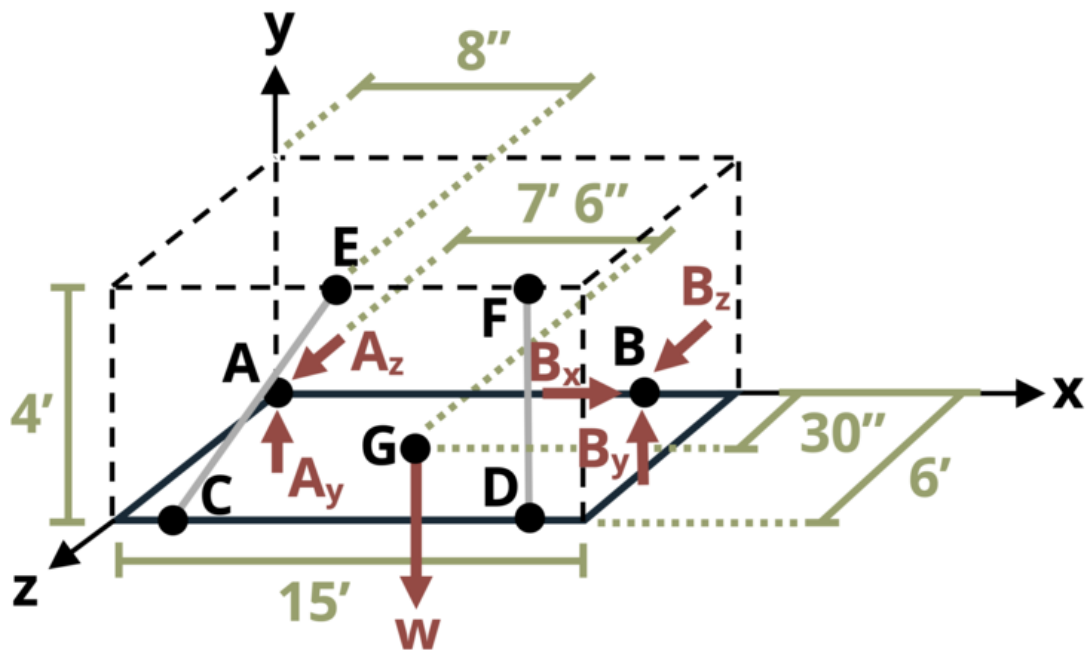
```

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"
    yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
    for attempt in attempts[1:]:
        await asyncio.sleep(0.25)
        yield attempt

app = App(app_ui, server)

```


Problem 1.8 - Equilibrium & Reactions in 3D



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```



```

random.seed(101)
w.set(random.randrange(3,10,1))
L.set(w()*random.randrange(20,30,1)/10)
weight.set(random.randrange(50,100,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

    # Calculate the instructor's answer and determine if the user's answer is correct.
    instr1 = 2.5*weight()/(1.106*w())
    instr2 = -0.3869*instr1
    Ay = (L()*0.552*instr1-7.5*weight())/(-L())
    instr3 = weight()-Ay-instr1*(0.552)-instr1*(0.554)
    Az = (L()*w()/7.238*instr1-0.0921*instr1*w()+w()*0.0461*instr1)/L()
    instr4 = -Az+instr1*w()/7.238+instr1*w()/7.22

    correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
    correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)
    correct3 = math.isclose(float(input.answer3()), instr3, rel_tol=0.01)
    correct4 = math.isclose(float(input.answer4()), instr4, rel_tol=0.01)

    if correct1 and correct2 and correct3 and correct4:
        check = "All answers are correct."
    else:
        conditions = []
        for i, correct in enumerate([correct1, correct2, correct3, correct4], start=1):
            if correct:
                conditions.append(f"correct for answer {i}")
            else:
                conditions.append(f"incorrect for answer {i}")
        check = "; ".join(conditions) + "."

    correct_indicator = "JL" if correct1 and correct2 and correct3 and correct4 else "JG"

    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)

```

```

        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        #session.encoded_attempt = reactive.Value(encoded_attempt)
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

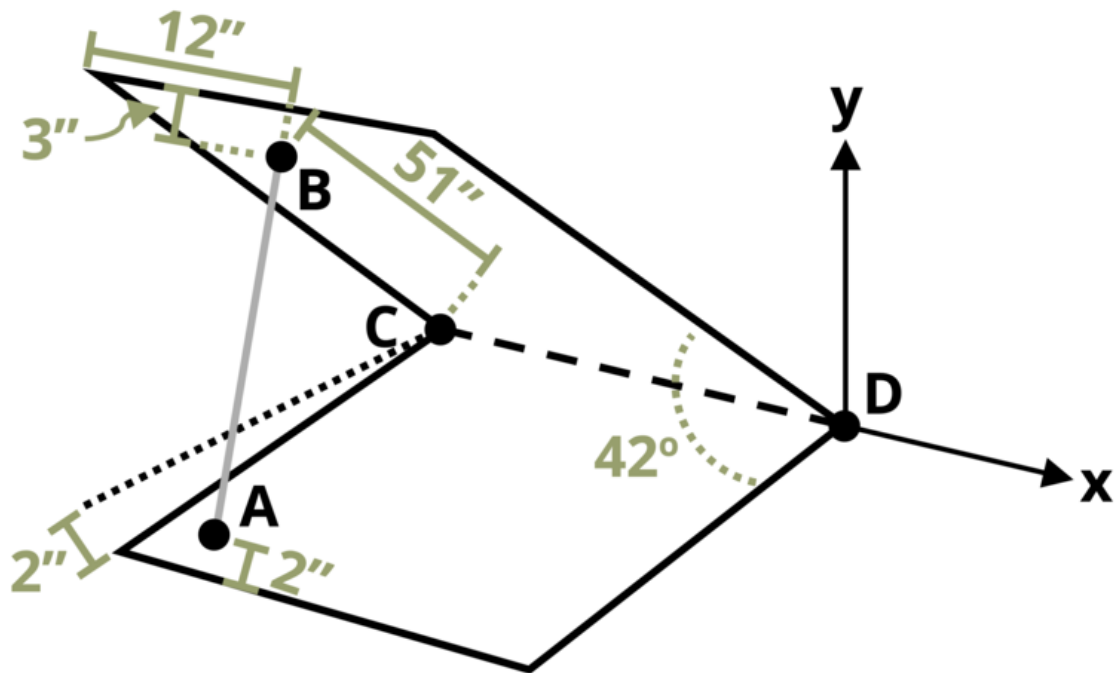
        feedback = ui.markdown(f"Your answers of {input.answer1()}, {input.answer2()}, {input
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"
    yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
    for attempt in attempts[1:]:
        await asyncio.sleep(0.25)
        yield attempt

app = App(app_ui, server)

```

Problem 1.9



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```

```

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "649"
w = reactive.Value("__")
L = reactive.Value("__")
weight = reactive.Value("__")
attempts = ["Timestamp", Attempt, Answer1, Answer2, Answer3, Answer4, Answer5, Answer6, Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of lb", placeholder="Please enter your answer"),
    ui.input_text("answer2", "Your Answer 2 in units of lb", placeholder="Please enter your answer"),
    ui.input_text("answer3", "Your Answer 3 in units of lb", placeholder="Please enter your answer"),
    ui.input_text("answer4", "Your Answer 4 in units of lb", placeholder="Please enter your answer"),
    ui.input_text("answer5", "Your Answer 5 in units of lb", placeholder="Please enter your answer"),
    ui.input_text("answer6", "Your Answer 6 in units of lb", placeholder="Please enter your answer"),

    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"During engine work, a car's hood of length  $L = \{L()\}$  in. and width  $w = \{w()\}$ 
            )
        ]

```

```

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    L.set(random.randrange(50,70,1))
    w.set(L()+random.randrange(10,20,1))
    weight.set(random.randrange(30,80,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

    # Calculate the instructor's answer and determine if the user's answer is correct.
    RDB = [-w()+12, 34.13, 37.9]
    RDA = [-w()+6, 0, L()-2]
    RDC = [-w(), 0, 0]
    RDG = [-w()/2, L()/2*0.6691, L()/2*0.7431]
    def vector_subtraction(RDB, RDA):
        return [x - y for x, y in zip(RDB, RDA)]
    RAB = vector_subtraction(RDB,RDA)
    def vector_magnitude(RAB):
        return math.sqrt(sum(x ** 2 for x in RAB))
    RABmag = vector_magnitude(RAB)
    FbarAB = [x/RABmag for x in RAB]
    comp1 = RDB[1]*FbarAB[2]-RDB[2]*FbarAB[1]
    comp2 = RDB[2]*FbarAB[0]-RDB[0]*FbarAB[2]
    comp3 = RDB[0]*FbarAB[1]-RDB[1]*FbarAB[0]
    FABCross = [comp1, comp2, comp3]
    Weightj = [0, -weight(), 0]
    comp4 = RDG[1]*Weightj[2]-RDG[2]*Weightj[1]
    comp5 = RDG[2]*Weightj[0]-RDG[0]*Weightj[2]
    comp6 = RDG[0]*Weightj[1]-RDG[1]*Weightj[0]
    RDGCross = [comp4, comp5, comp6]
    instr1 = comp4/comp1
    Cract = [0,1,1]
    comp7 = RDC[1]*Cract[2]-RDC[2]*Cract[1]
    comp8 = RDC[2]*Cract[0]-RDC[0]*Cract[2]
    comp9 = RDC[0]*Cract[1]-RDC[1]*Cract[0]
    RDCCross = [comp7, comp8, comp9]

```

```

instr2 = comp2*instr1/comp8
instr3 = (comp3*instr1-comp6)/comp9
instr4 = -FbarAB[0]*instr1
instr5 = -FbarAB[1]*instr1+weight()-instr3
instr6 = -FbarAB[2]-instr2

correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)
correct3 = math.isclose(float(input.answer3()), instr3, rel_tol=0.01)
correct4 = math.isclose(float(input.answer4()), instr4, rel_tol=0.01)
correct5 = math.isclose(float(input.answer5()), instr5, rel_tol=0.01)
correct6 = math.isclose(float(input.answer6()), instr6, rel_tol=0.01)

if correct1 and correct2 and correct3 and correct4 and correct5 and correct6:
    check = "All answers are correct."
else:
    conditions = []
    for i, correct in enumerate([correct1, correct2, correct3, correct4, correct5, correct6]):
        if correct:
            conditions.append(f"correct for answer {i}")
        else:
            conditions.append(f"incorrect for answer {i}")
    check = "; ".join(conditions) + "."

correct_indicator = "JL" if correct1 and correct2 and correct3 and correct4 and correct5 and correct6 else "N"

random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{random_end}"

#session.encoded_attempt = reactive.Value(encoded_attempt)
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.answer2()}, {input.answer3()}, {input.answer4()}, {input.answer5()}, {input.answer6()}")

feedback = ui.markdown(f"Your answers of {input.answer1()}, {input.answer2()}, {input.answer3()}, {input.answer4()}, {input.answer5()}, {input.answer6()}")
m = ui.modal(feedback, title="Feedback", easy_close=True)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

```



```
        yield "Timestamp,Attempt,Answer1,Answer2,Answer3,Answer4,Answer5,Answer6,Feedback\n"  
        for attempt in attempts[1:]:  
            await asyncio.sleep(0.25)  
            yield attempt  
  
app = App(app_ui, server)
```

Part III

Chapter 2 Problems

Problem 2.1 - Average Normal Stress

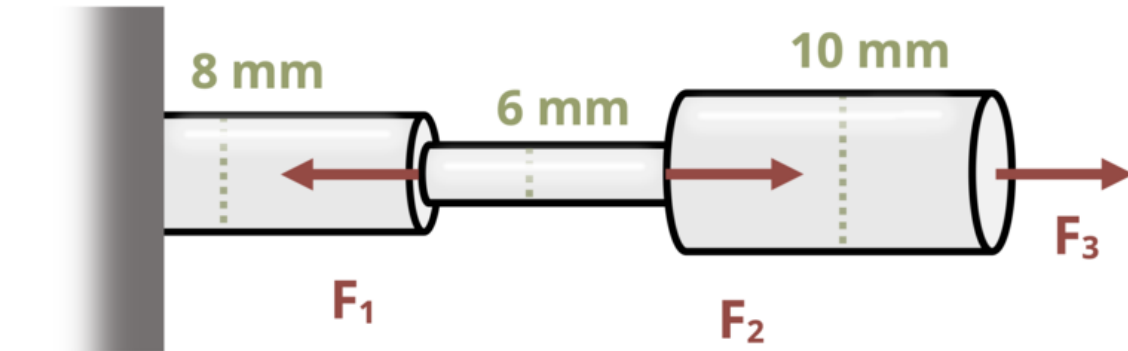


Figure 1: Figure 1: A series of solid circular bars are loaded with three loads

[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
```

```

return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="138"
F1=reactive.Value("__")
F2=reactive.Value("__")
F3=reactive.Value("__")
d1=8
d2=6
d3=10
attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("***Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("***Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A series of solid circular bars are loaded with three loads as s

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(50, 70, 1))
        F2.set(random.randrange(10, 30, 1))
        F3.set(F1()-F2())
```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each

    # Calculate the instructor's answer and determine if the user's answer is correct.
    instr= (F1()/(math.pi*(d2/(1000*2))**2))/10**6

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

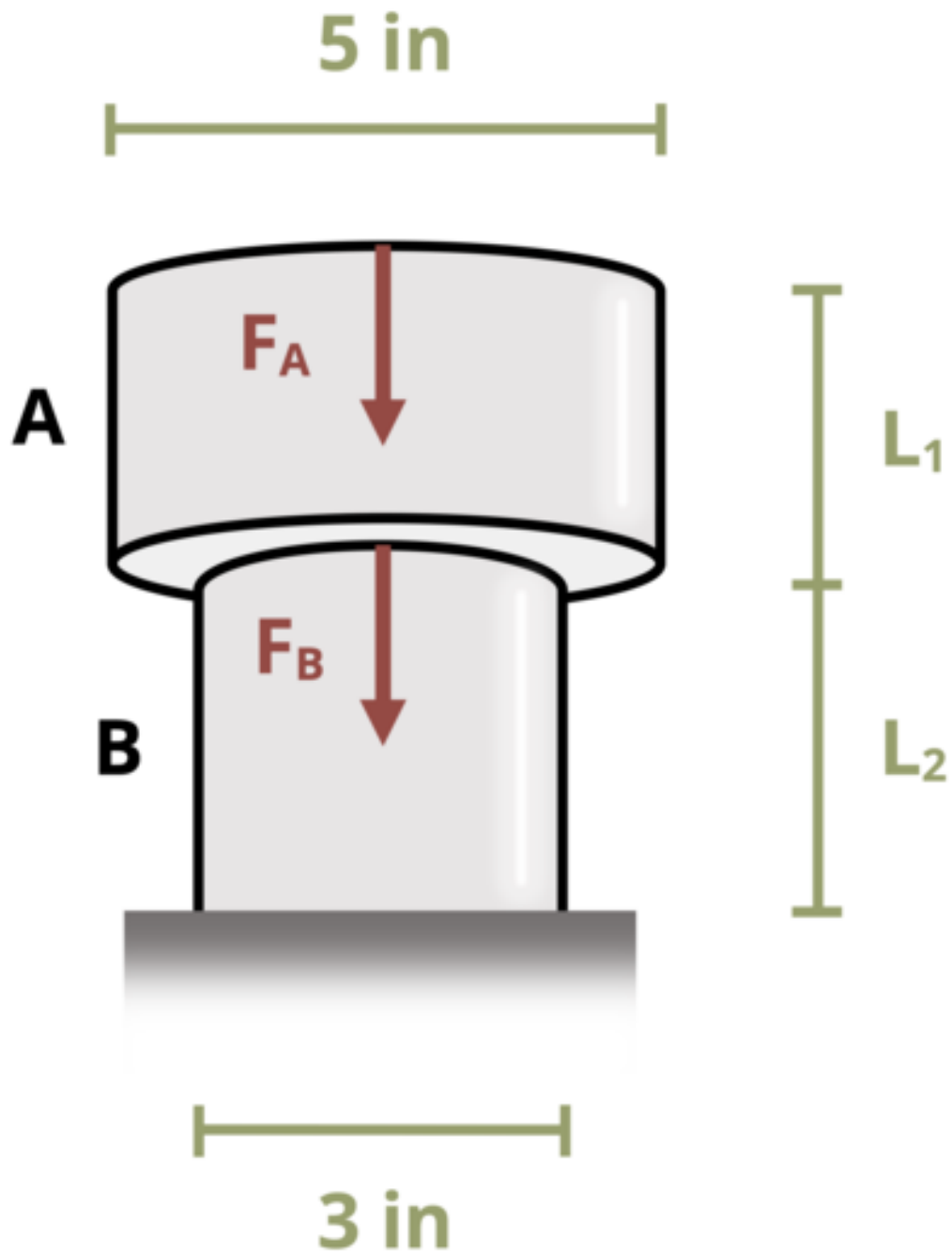
```

```
# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)
```


Problem 2.2 - Average Normal Stress




```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="139"
L1=reactive.Value("__")
L2=reactive.Value("__")
FA=reactive.Value("__")
FB=reactive.Value("__")
E = 30*10**6

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output

```

```

@render.ui
def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"Two cylinders are stacked on top of one another and two forces a

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    FA.set(random.randrange(300, 700, 10))
    FB.set(random.randrange(100, 300, 10))
    L1.set(random.randrange(2, 7, 1))
    L2.set(round(L1() * 1.3,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

    instr = -(FA()+FB()/(math.pi*1.5**2))
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(

```

```

        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

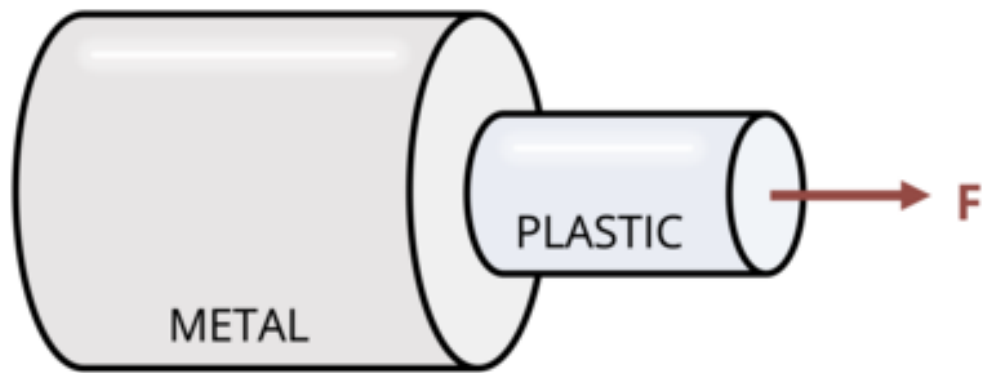
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

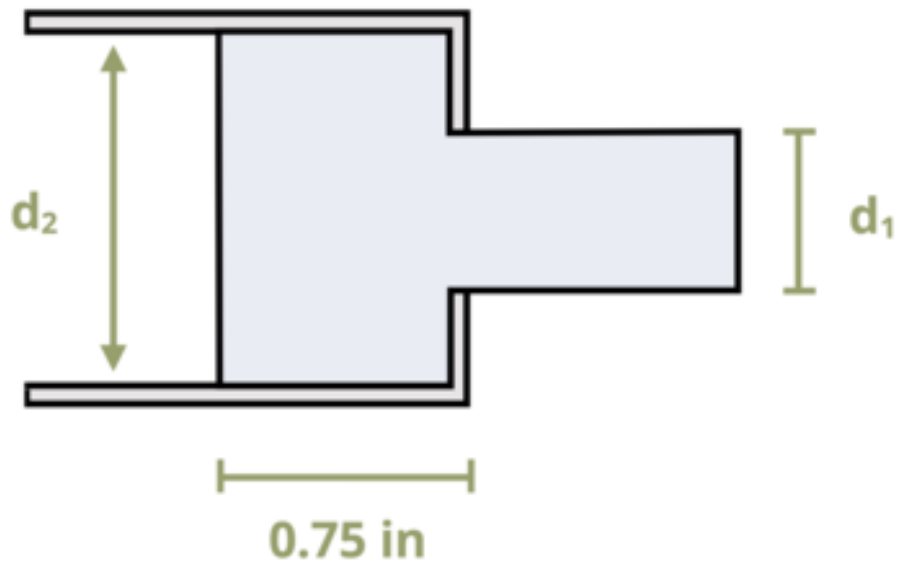
# App installation
app = App(app_ui, server)

```


Problem 2.3 - Average Normal Stress



CROSS SECTION VIEW



```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="144"
F=reactive.Value("__")
d1=reactive.Value("__")
d2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui

```



```

        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

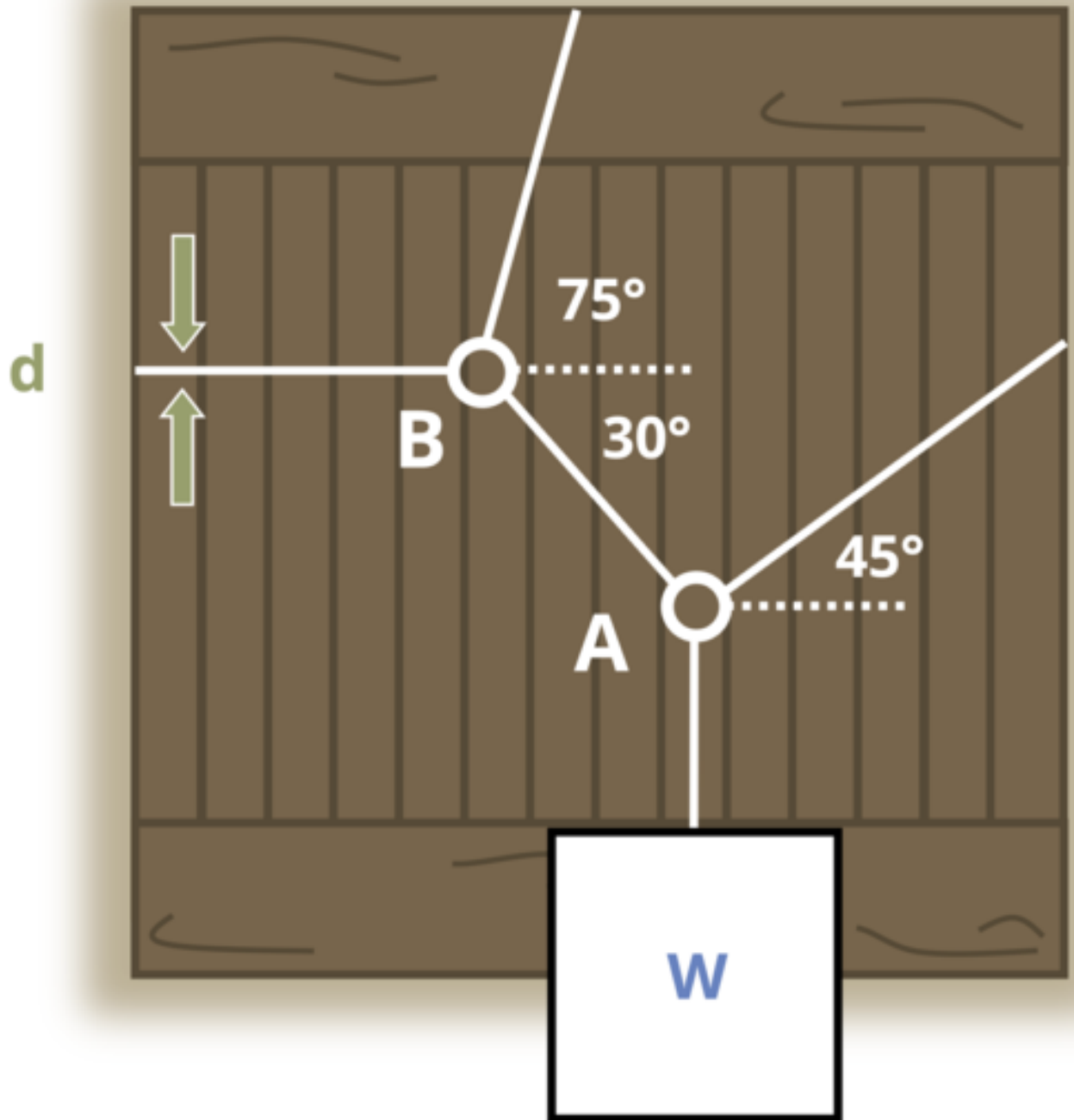
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 2.4 - Average Normal Stress



```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="146"
W=reactive.Value("__")
d=reactive.Value("__")
angle1=math.radians(45)
angle2=math.radians(30)
angle3=math.radians(75)

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

```

```

@output
@render.ui
def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A crate weighing {W()} kN is suspended by a set of cables. The c

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    W.set(random.randrange(30, 90, 1))
    d.set(random.randrange(20, 60, 1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

    R1 = W()/(((math.cos(angle1)/math.cos(angle2))*math.sin(angle2))+math.sin(angle1))
    instr = (R1*10**3/(math.pi*((d()/(1000*2))**2)))/10**6
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")

```

```

        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

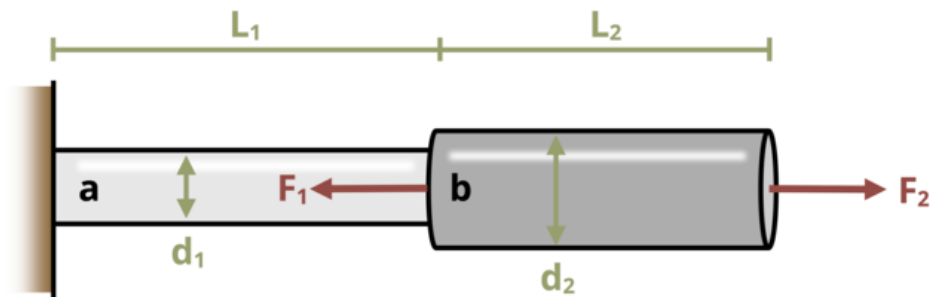
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 2.5 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="141"
F1=reactive.Value("__")
F2=reactive.Value("__")
d1=reactive.Value("__")
d2=reactive.Value("__")
```



```

if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

# Store the most recent encoded attempt in a reactive value so it persists across sessions
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

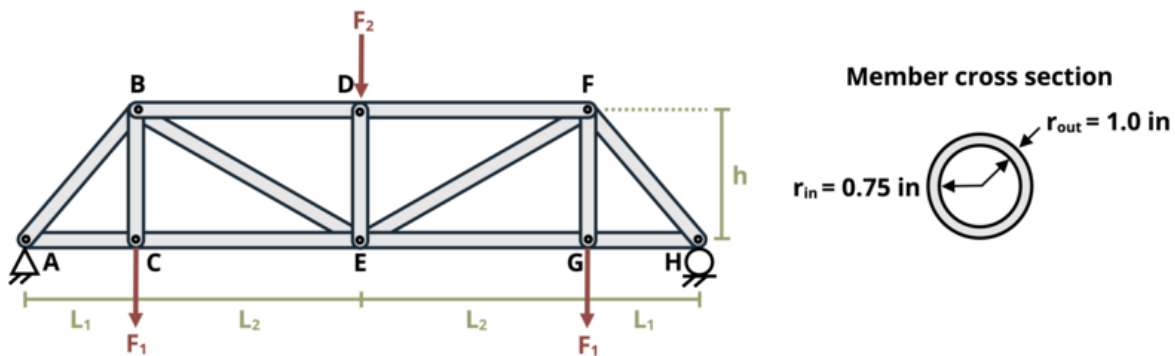
    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

```



```
# App installation  
app = App(app_ui, server)
```

Problem 2.6 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="142"
F1=reactive.Value("__")
F2=reactive.Value("__")
F3=reactive.Value("__")
```



```

else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

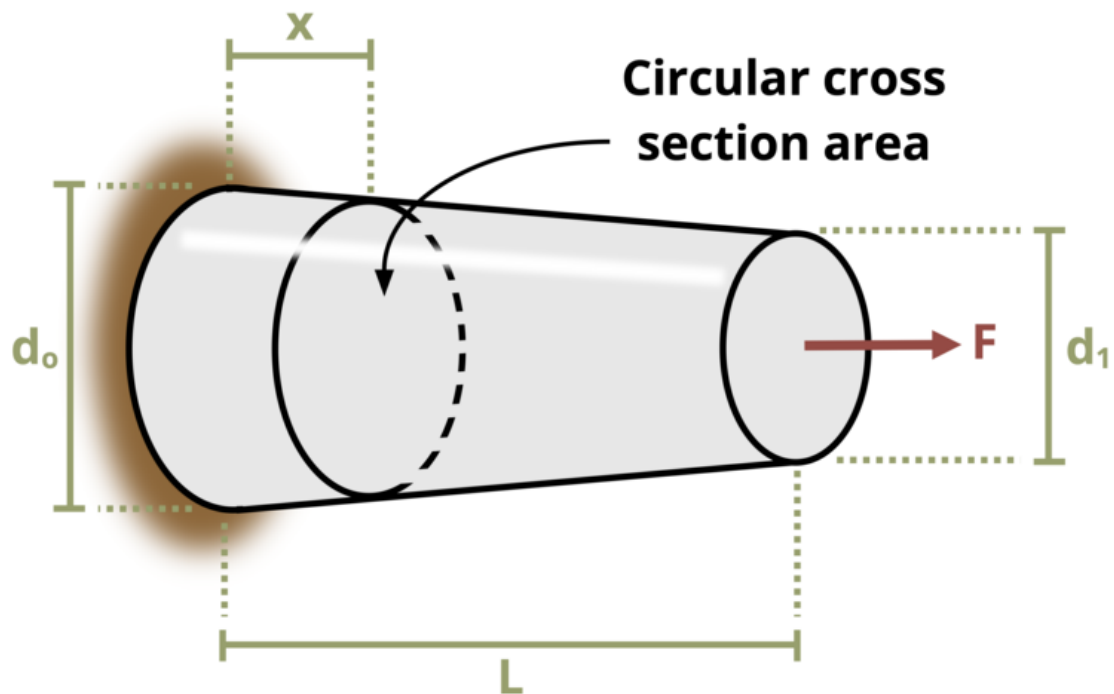
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 2.7 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
```

```

from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="143"
F=reactive.Value("__")
L=reactive.Value("__")
d0=reactive.Value("__")
d1=reactive.Value("__")
x=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A tapered cylindrical rod is pulled by a force  $F = \{F()\}$  N. If I

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(200, 1000, 10))
        L.set(random.randrange(10, 100, 1)/10)

```

```

d0.set(random.randrange(300, 700, 10))
d1.set(d0()/2)
x.set(L()*random.randrange(2, 4, 1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each
    d = d0()/10-(x()/L()*(d0()/10-d1()/10))
    instr = F()/((d/2)**2*math.pi)*10
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else

```

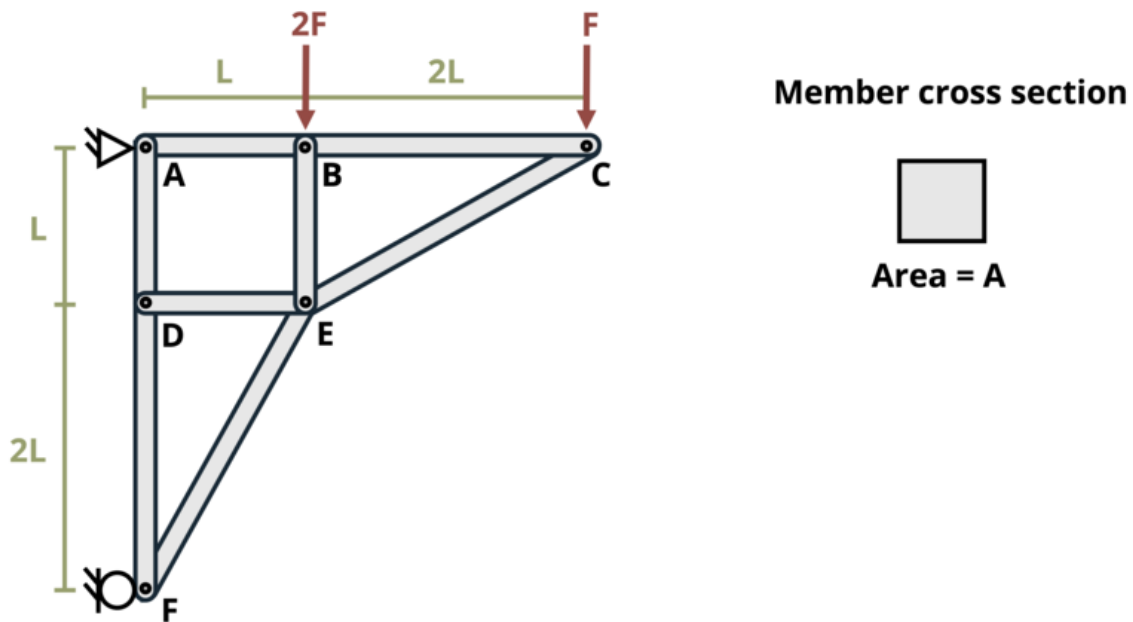
```
yield f"{final_encoded}\n\n"

# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)
```


Problem 2.8 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
```

```

    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="149"
A=reactive.Value("__")
F=reactive.Value("__")
L=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A truss is constructed using solid square members of area A = {

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        A.set(random.randrange(500, 1500, 10))
        F.set(random.randrange(20, 75, 1))
        L.set(random.randrange(5, 10, 1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():

```

```

    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    Fx = (2*F()*L()+F()*3*L())/(3*L())
    FEF = -Fx*2.236
    instr = FEF/A()*10**3
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

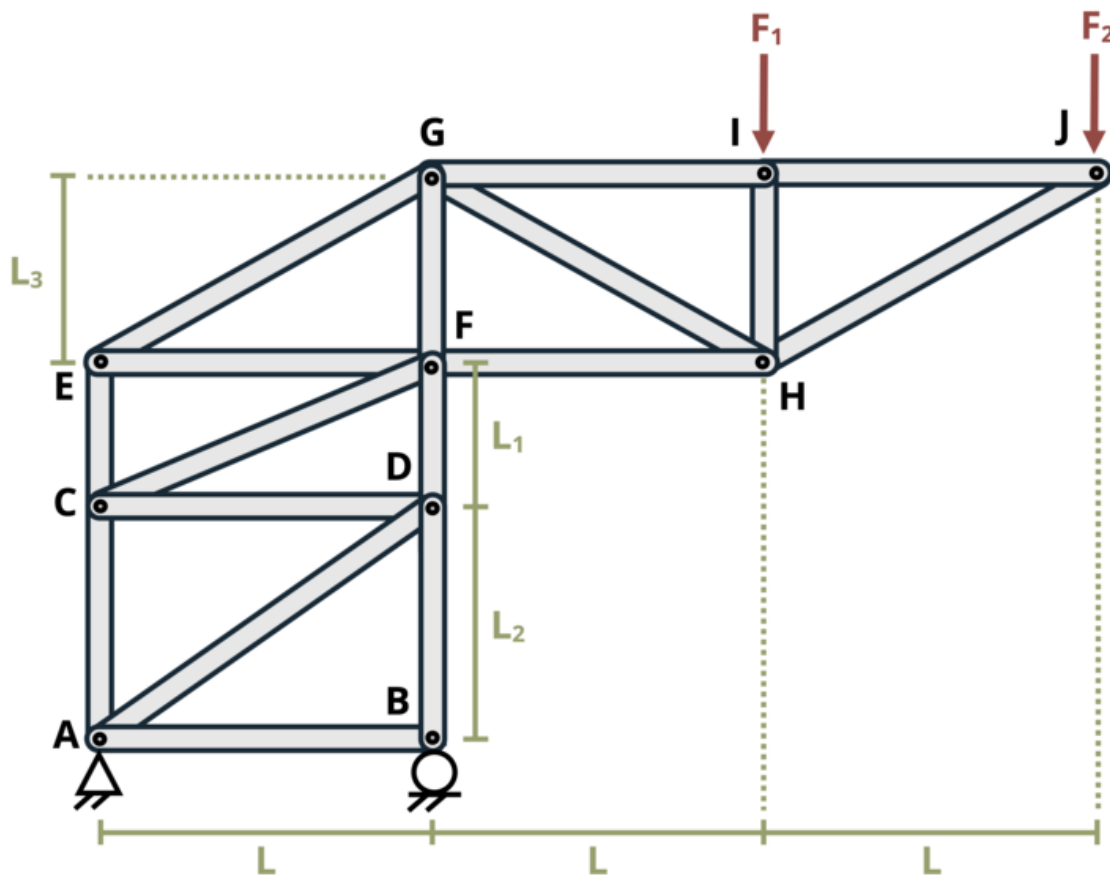
    # Write the attempts data, ensure that the header from the attempts list is not writ

```

```
        for attempt in attempts[1:]: # Skip the first element which is the header
            await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.9 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
```

```

import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="150"
A=reactive.Value("__")
F1=reactive.Value("__")
F2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A truss is constructed using members of area  $A = \{A()\}$  in<sup>2

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():

```

```

    random.seed(101)
    A.set(random.randrange(20, 100, 1)/10)
    F1.set(random.randrange(300, 1000, 10))
    F2.set(random.randrange(300, 1000, 10))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    FEG = (F1()*12+F2()*24)/4.615
    instr = FEG/A()
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)

```

```

    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

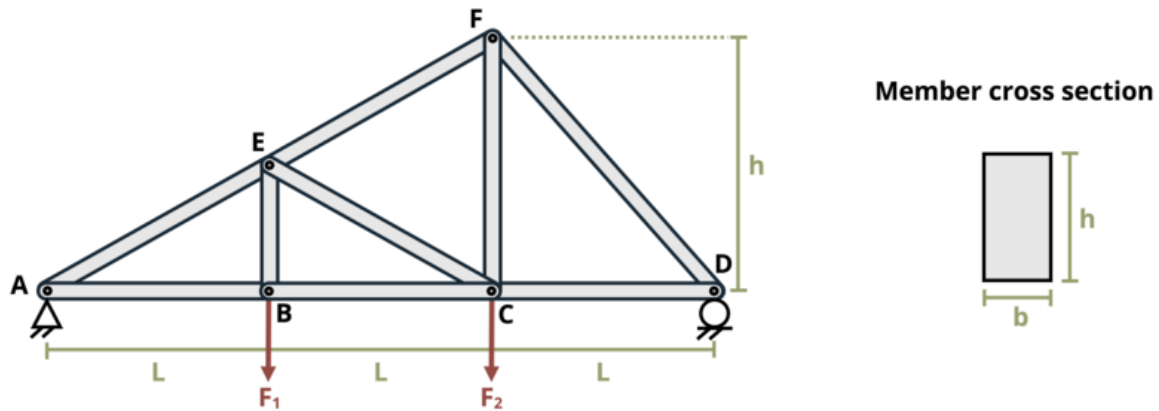
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 2.10 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="152"
b=reactive.Value("__")
h=reactive.Value("__")
```

```
F1=reactive.Value("__")
F2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A truss is constructed of members with rectangular cross-section

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    b.set(random.randrange(10, 50, 1)/10)
    h.set(b()*2)
    F1.set(random.randrange(10, 100, 1)/10)
    F2.set(random.randrange(10, 100, 1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    FD = (6*F2()+3*F1())/9
    FEF = -FD*3/(3*math.sin(63.43*math.pi/180))
    instr = FEF/(b()*h())*1000
```

```

if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

# Store the most recent encoded attempt in a reactive value so it persists across sessions
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

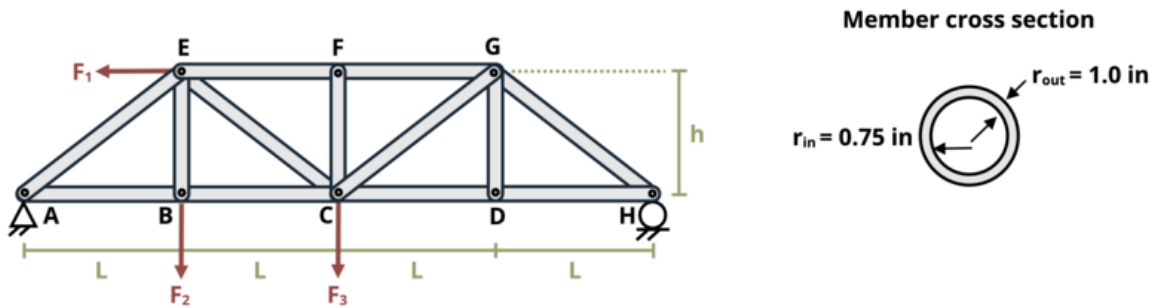
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

```

```
# App installation  
app = App(app_ui, server)
```

Problem 2.11 -Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="154"
F1=reactive.Value("__")
F2=reactive.Value("__")
F3=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]
    
```

```

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A truss is constructed of members with the circular cross-section

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(10, 100, 1)/10)
        F2.set(random.randrange(10, 100, 1)/10)
        F3.set(random.randrange(10, 100, 1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        Hy = (F3()*8+F2()*4-F1()*4)/16
        FEC = (F3()-Hy)/math.cos(45*math.pi/180)
        instr = FEC/1.374
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

```

```

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sub
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

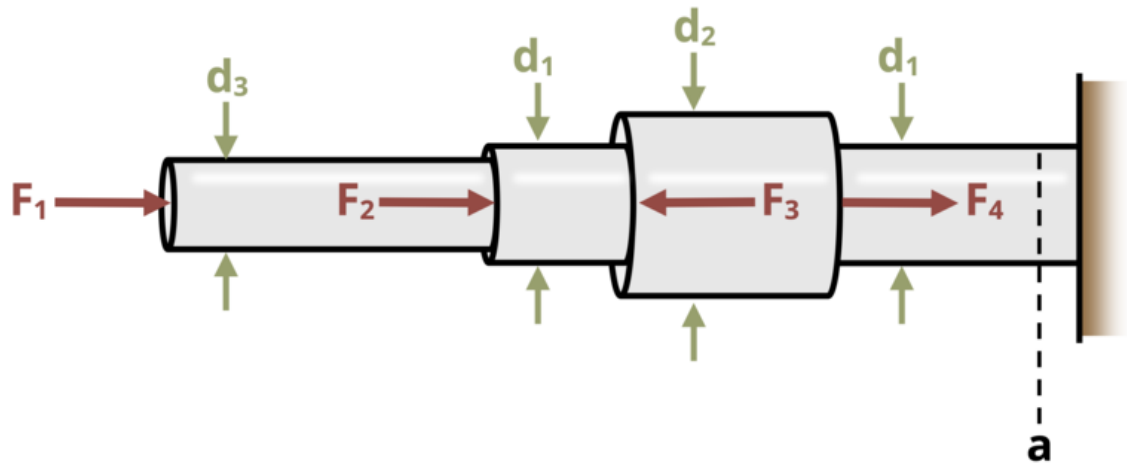
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 2.12 -Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="155"
```



```

F1=reactive.Value("__")
F2=reactive.Value("__")
F3=reactive.Value("__")
F4=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A series of solid circular bars are connected together as shown

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(50, 200, 1)/10)
        F2.set(random.randrange(50, 200, 1)/10)
        F3.set(random.randrange(50, 140, 1)/10)
        F4.set(random.randrange(50, 200, 1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        F = F1()+F2()-F3()+F4()

```

```

instr = F/227*10**4
if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across su
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

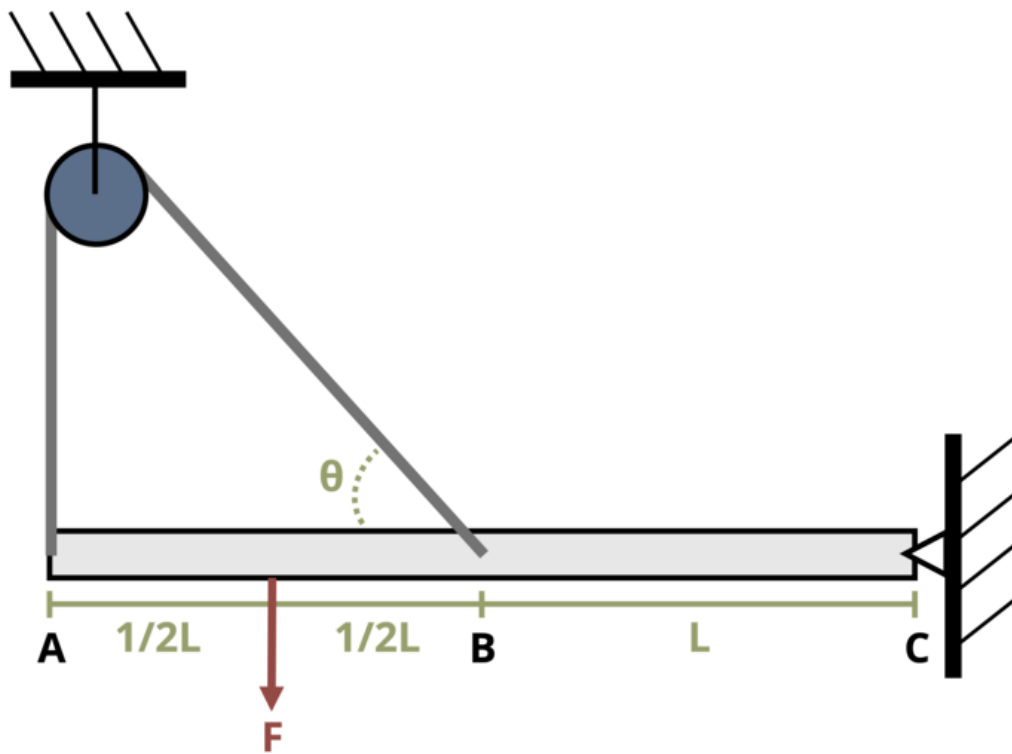
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

```

```
# App installation
app = App(app_ui, server)
```

Problem 2.13 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
```

```
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="158"
F=reactive.Value("__")
theta=reactive.Value("__")
d=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A beam is held in place by a rope that loops over a pulley as sl

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(10, 100, 1))
        theta.set(random.randrange(33, 55, 1))
        d.set(random.randrange(15, 30, 1))
```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each
    Fr = F()*1.5/(2+math.sin(theta()*math.pi/180))*1000
    r = d()/2000
    A = math.pi*r**2
    instr = Fr/A/10**6
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across ses
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

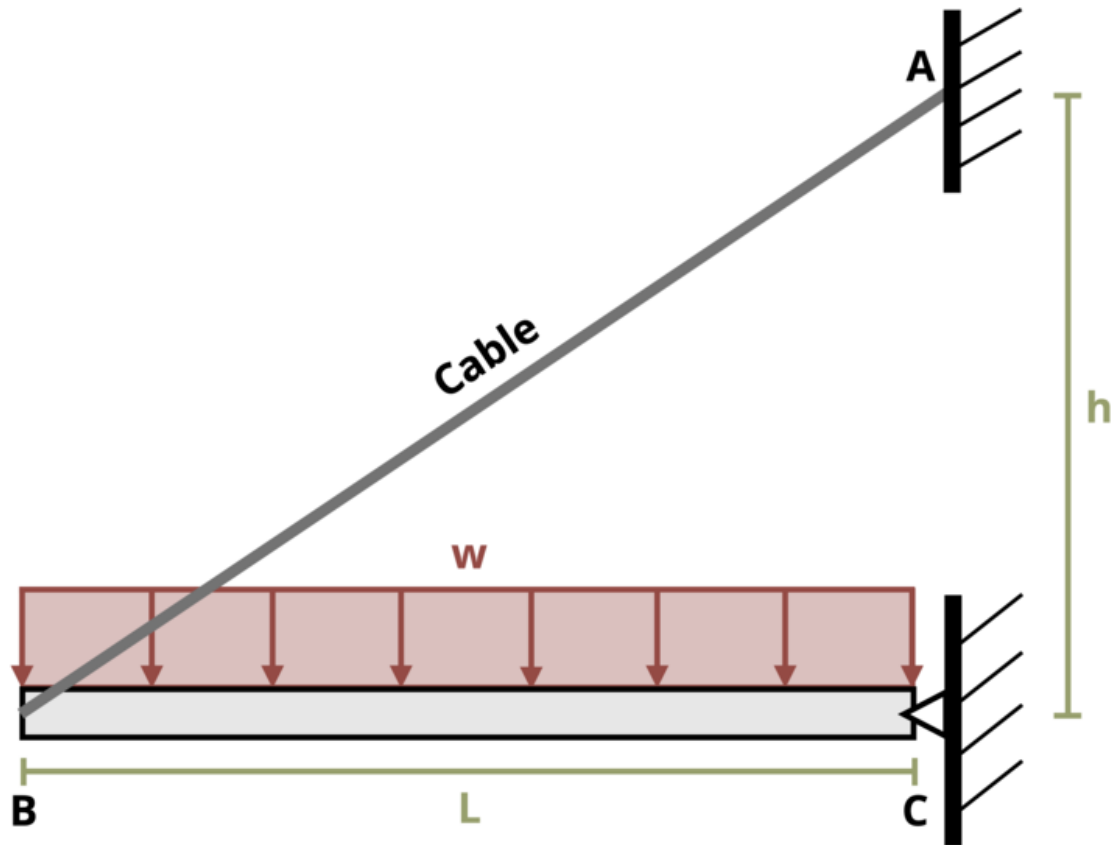
```

```
# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.14 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
```



```

import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="159"
w=reactive.Value("__")
d=reactive.Value("__")
L=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A cable supports a beam with a uniform distributed load  $w = \{w($ 

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():

```

```

    random.seed(101)
    w.set(random.randrange(10,50,1))
    d.set(random.randrange(15,30,1))
    L.set(random.randrange(10,30,1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    theta = math.atan(1/L())
    Fc = (w()*L()*L()/2)/(L()*math.sin(theta))
    instr = Fc/(math.pi*(d()/2000)**2)/10**3

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")

```

```

async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

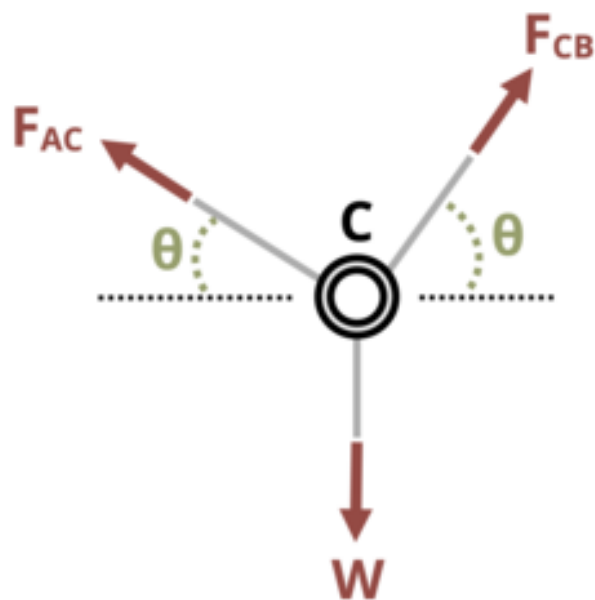
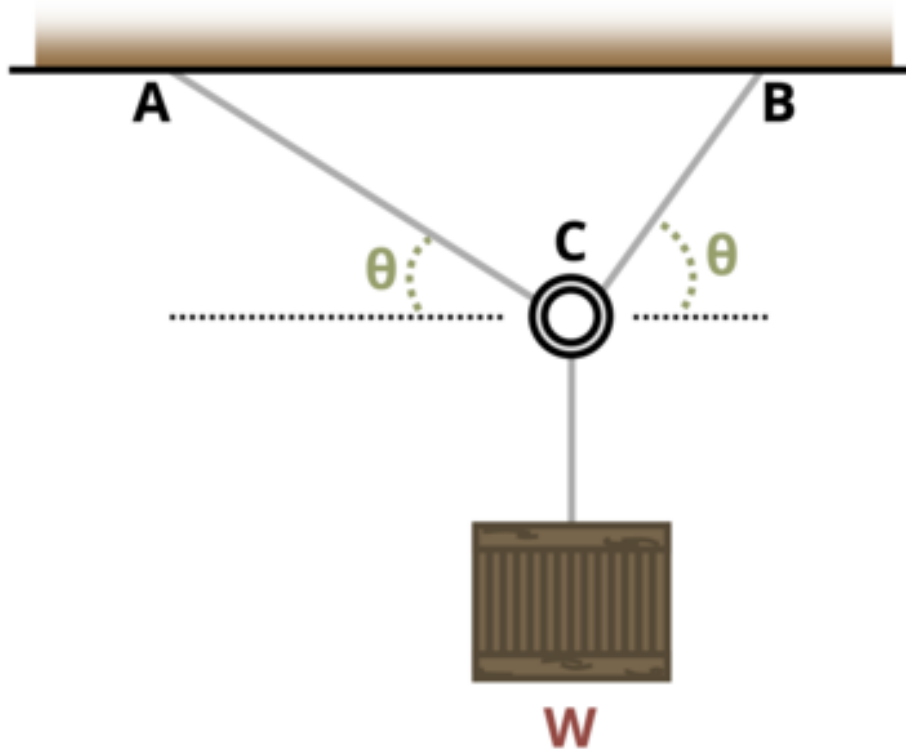
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 2.15 - Average Normal Stress



```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="160"
W=reactive.Value("__")
dAC=reactive.Value("__")
dBC=reactive.Value("__")
alpha=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of degrees", placeholder="Please enter your
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui

```

```

def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A box weighing  $W = \{W()\}$  lb is suspended by two cables as shown

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    W.set(random.randrange(300,1500,10))
    dBC.set(random.randrange(10,50,1)/10)
    dAC.set(dBC()+random.randrange(10,20,1)/10)
    alpha.set(random.randrange(50,60,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    AAC = (dAC()/2)**2*math.pi
    ACB = (dBC()/2)**2*math.pi
    ratio = ACB/AAC
    instr = math.acos(ratio*math.cos(alpha()*math.pi/180))*180/math.pi

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

    # Show feedback to the user.

```

```

        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

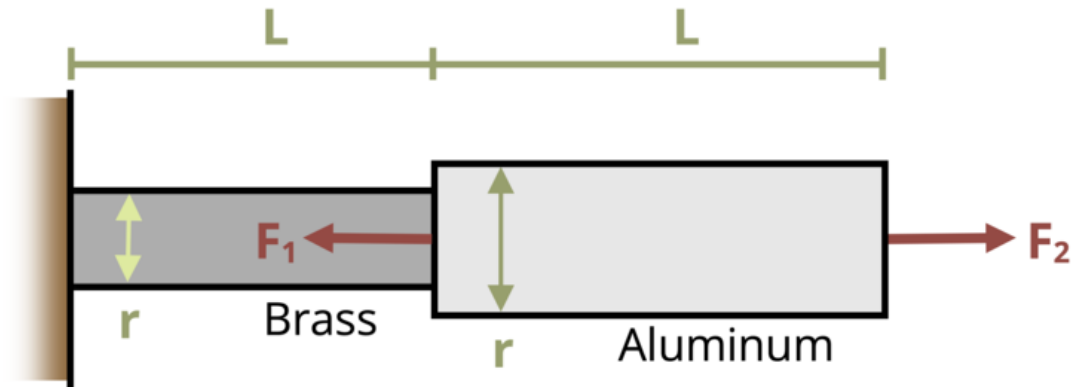
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 2.16 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "162"
F1 = reactive.Value("__")
```

```

F2 = reactive.Value("__")
d1 = reactive.Value("__")
d2 = reactive.Value("__")

attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(

    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),

    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of MPa", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"Two forces,  $F_1$  = {F1()} kN and  $F_2$  = {F2()} kN are ap
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(10, 50, 1))
        F2.set(F1() + random.randrange(10, 50, 1))
        d1.set(random.randrange(15, 30, 1))
        d2.set(d1() * 1.5)

```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.
    FB = F2() - F1()
    sigmaB = FB / ((d1() / 2) ** 2 * math.pi) * 10**3
    FA = F2()
    sigmaA = FA / ((d2() / 2) ** 2 * math.pi) * 10**3
    if sigmaB >= sigmaA:
        instr = sigmaB
    else:
        instr = sigmaA

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(
        f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"
    )

    # Show feedback to the user.
    feedback = ui.markdown(
        f"Your answer of {input.answer()} is {check}."
    )
    m = ui.modal(feedback, title="Feedback", easy_close=True)
    ui.modal_show(m)

```

```

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = (
        session.encoded_attempt()
        if session.encoded_attempt is not None
        else "No attempts"
    )
    yield f"{final_encoded}\n\n"

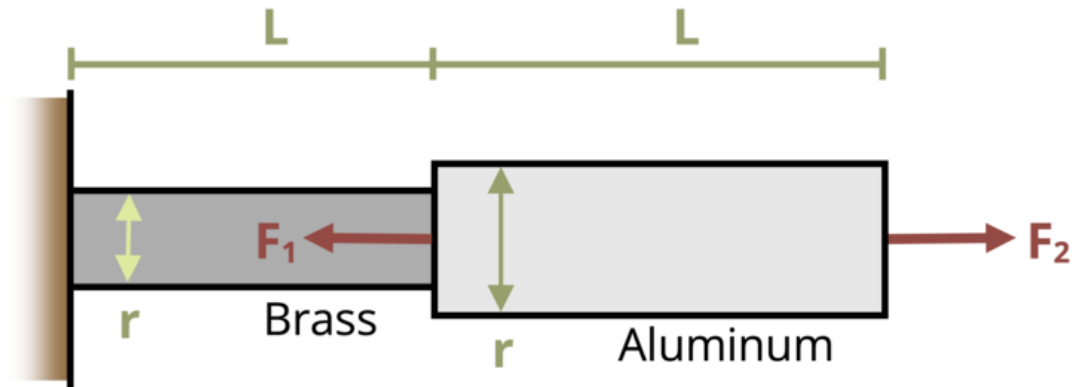
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(
            0.25
        ) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 2.17 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "163"
w0 = reactive.Value("__")
```

```

L = reactive.Value("__")
AA = reactive.Value("__")
AB = reactive.Value("__")

attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(

    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),

    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of ksi", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"A beam structure supported by a tube at each end is loaded with a distribut
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        w0.set(random.randrange(20,100,1)/10)
        L.set(random.randrange(6,15,3))
        AA.set(random.randrange(20,40,1)/10)
        AB.set(random.randrange(50,70,1)/10)

```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.
    FB = w0()*L()*0.5*2/3
    FA = w0()*L()*0.5-FB
    sigmaA = FA/AA()
    sigmaB = FB/AB()
    if sigmaA >= sigmaB:
        instr = sigmaA
    else:
        instr = sigmaB

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(
        f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"
    )

    # Show feedback to the user.
    feedback = ui.markdown(
        f"Your answer of {input.answer()} is {check}."
    )
    m = ui.modal(feedback, title="Feedback", easy_close=True)
    ui.modal_show(m)

```

```

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = (
        session.encoded_attempt()
        if session.encoded_attempt is not None
        else "No attempts"
    )
    yield f"{final_encoded}\n\n"

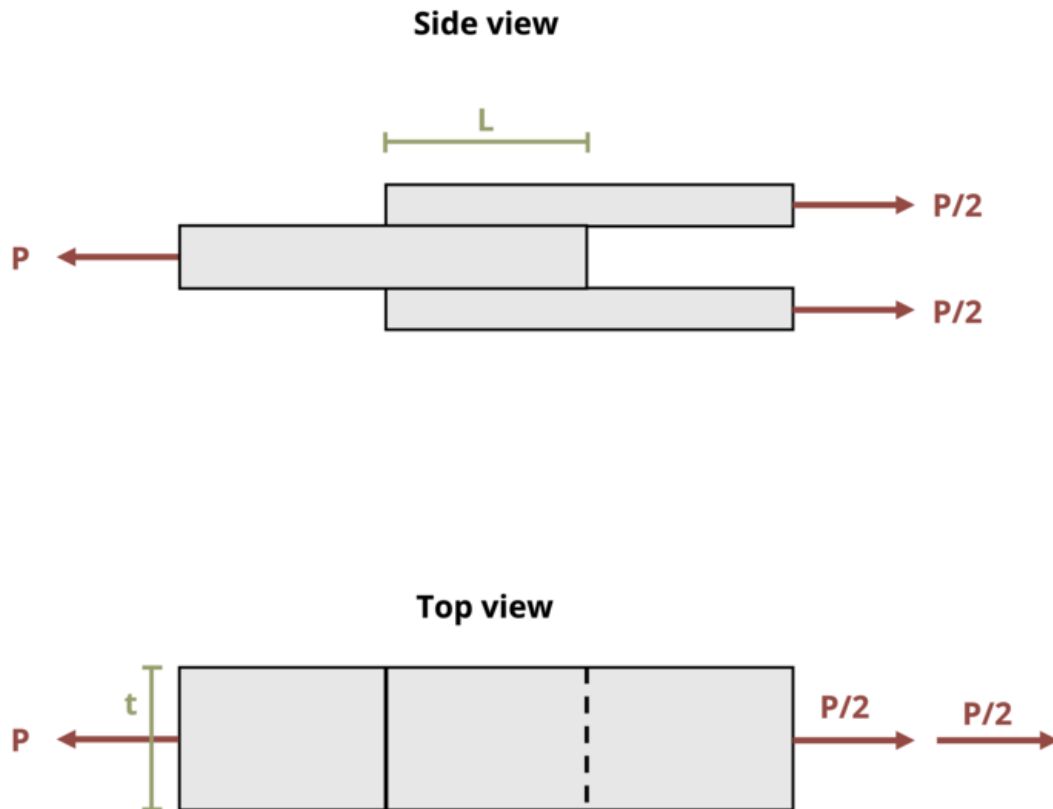
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(
            0.25
        ) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 2.21 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
```

```

import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="164"
fail=reactive.Value("__")
L=reactive.Value("__")
t=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kips", placeholder="Please enter your an
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A double lap joint is glued together using glue with a shear st

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():

```

```

random.seed(101)
fail.set(random.randrange(7000, 9000, 100))
L.set(random.randrange(40, 100, 1)/10)
t.set(random.randrange(40, 100, 1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

    A = L()*t()*2
    instr= (fail()*A)/1000
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")

```

```

async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

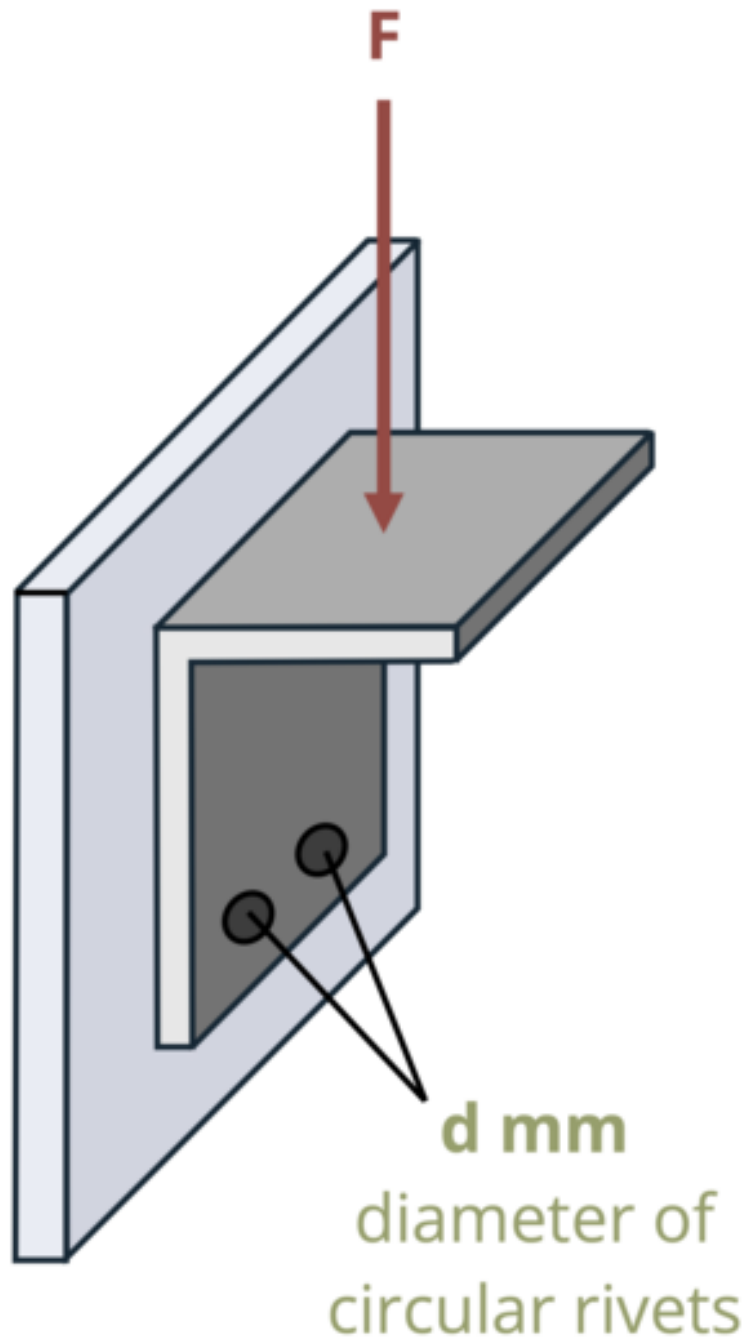
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 2.22 - Average Shear Stress



```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="165"
F=reactive.Value("__")
d=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui

```

```

def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A bracket is attached to a wall with two circular rivets of diameter {d} inches. A force of {F} pounds is applied to the end of the bracket. What is the area of the circular rivet?")]

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    F.set(random.randrange(30, 100, 1))
    d.set(random.randrange(10, 40, 1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each submission

    A = math.pi*(d()/(1000*2))**2
    instr= ((F()/2)/A)/1000
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{random_end}"

    # Store the most recent encoded attempt in a reactive value so it persists across submissions
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,

```



```

        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 2.23 - Average Shear Stress

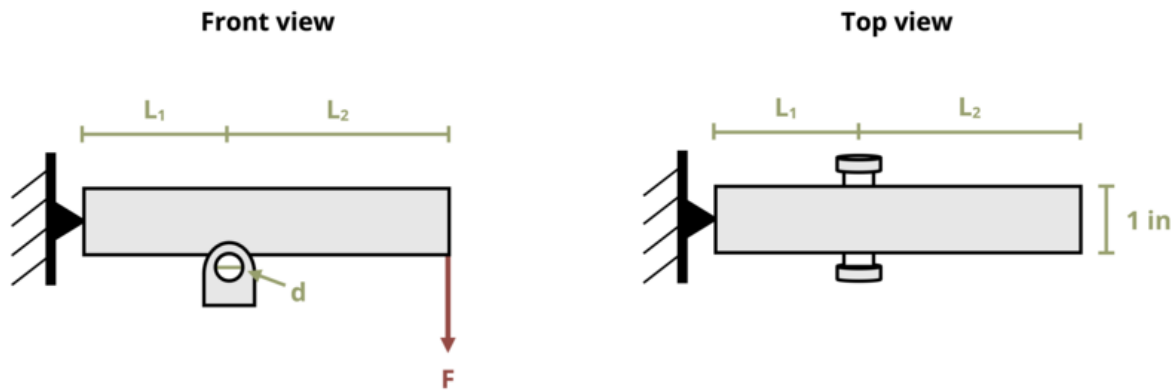


Figure 2: Figure 1: A square bar of length is pinned at one end and rests on a circular rod.

[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```

problem_ID="168"
L1=reactive.Value("__")
L2=reactive.Value("__")
d=reactive.Value("__")
F=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A square bar of length  $L_1$  in. and  $L_2$ 

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L1.set(random.randrange(50, 150, 1)/10)
        L2.set(round(L1() * 1.4, 1))
        d.set(random.randrange(4, 9, 1)/10)
        F.set(random.randrange(30, 100, 1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

```

```

M = F()*(L1()+L2())
R = M/L1()
A = math.pi*(d()/2)**2
instr= R/(2*A)
if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

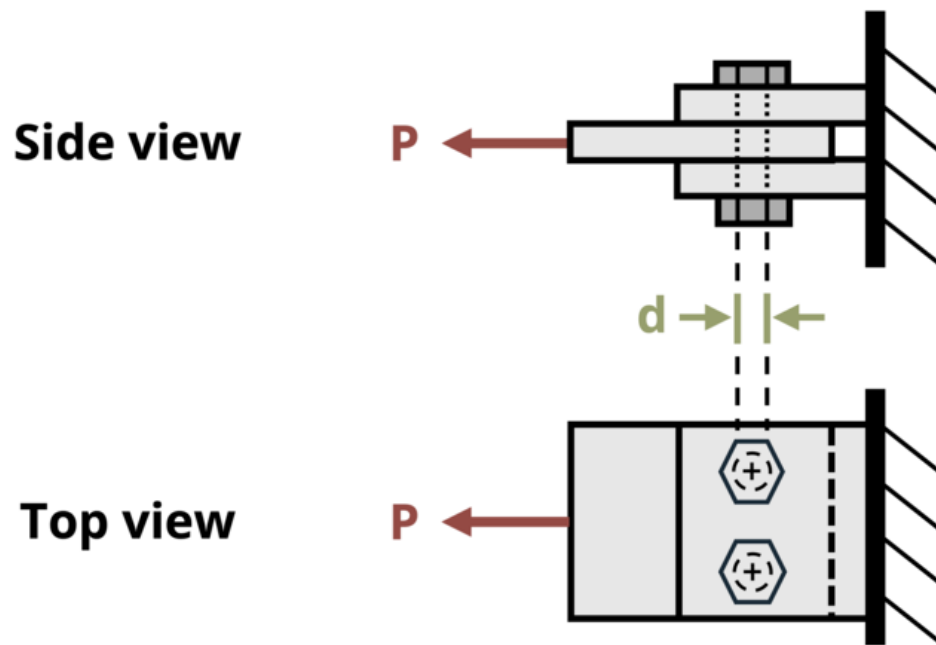
    # Write the attempts data, ensure that the header from the attempts list is not writ

```

```
        for attempt in attempts[1:]: # Skip the first element which is the header
            await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.24 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```

```

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "170"
d = reactive.Value("__")
tau_fail = reactive.Value("__")

attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(

    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),

    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of kN", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"A double lap joint is held together with two steel bolts. Each bolt has a c
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        d.set(random.randrange(10,30,1))

```

```

tau_fail.set(random.randrange(200,300,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.
    instr = tau_fail()*2*2*((d()/2)**2*math.pi)/10**3

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(
        f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"
    )

    # Show feedback to the user.
    feedback = ui.markdown(
        f"Your answer of {input.answer()} is {check}."
    )
    m = ui.modal(feedback, title="Feedback", easy_close=True)
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = (

```



```

        session.encoded_attempt()
        if session.encoded_attempt is not None
        else "No attempts"
    )
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

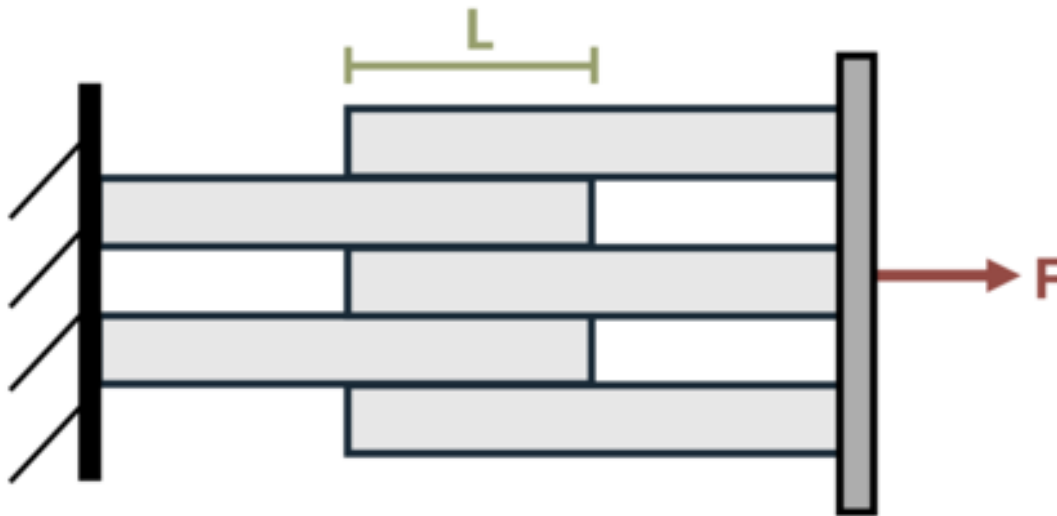
    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(
            0.25
        ) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

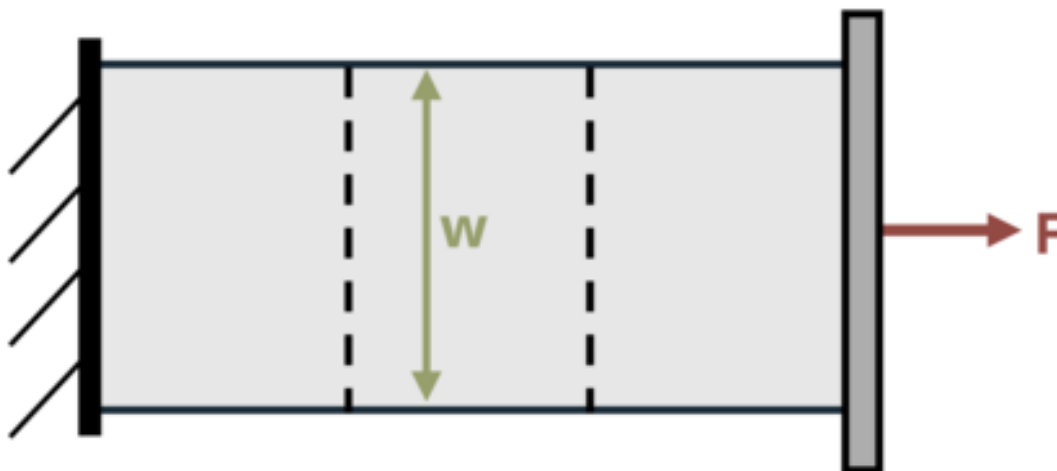
```


Problem 2.25 - Average Shear Stress

Side view



Top view



```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "173"
L = reactive.Value("__")
w = reactive.Value("__")
F = reactive.Value("__")
attempts = ["Timestamp", Attempt, Answer, Feedback\n"]

app_ui = ui.page_fluid(

    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),

    ui.markdown("***Problem Statement***"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of MPa", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output

```

```

@render.ui
def ui_problem_statement():
    randomize_vars()
    return [
        ui.markdown(
            f"A lap joint is constructed with layers as shown. If dimensions L = {L()} mm
        )
    ]

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    L.set(random.randrange(10, 50, 1))
    w.set(round(L() * 1.6, 1))
    F.set(random.randrange(5, 75, 1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

    # Calculate the instructor's answer and determine if the user's answer is correct.
    A = w()/1000*L()/1000*4
    instr = F()*1000/A/10**6

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

    # Store the most recent encoded attempt in a reactive value so it persists across submissions

```

```

#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

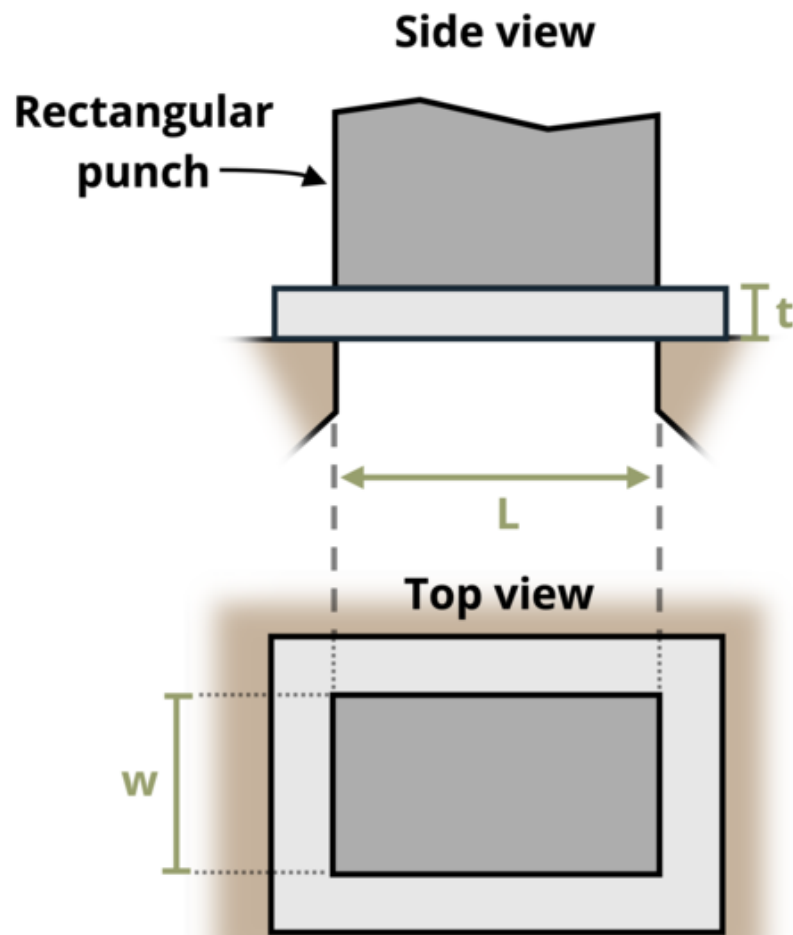
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 2.26 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]
```

```

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "174"
w = reactive.Value("__")
L = reactive.Value("__")
t = reactive.Value("__")
tau_fail = reactive.Value("__")
attempts = ["Timestamp", Attempt, Answer, Feedback\n"]

app_ui = ui.page_fluid(

    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),

    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of kip", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()

```



```

    return [
        ui.markdown(
            f"A rectangular hole punch is configured to punch a hole of  $L = \{L()\}$  in. by
        )
    ]

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    w.set(random.randrange(20,60,1)/10)
    L.set(round(w()*2,1))
    t.set(random.randrange(10,75,1)/100)
    tau_fail.set(random.randrange(5,40,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

    # Calculate the instructor's answer and determine if the user's answer is correct.
    A = (2*L()+2*w())*t()
    instr = tau_fail()*A

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

```

```

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

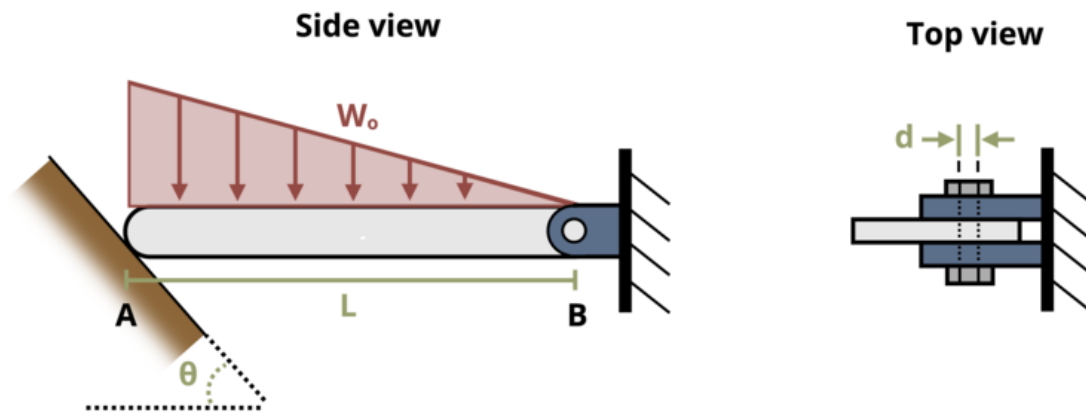
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 2.27 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "175"
theta = reactive.Value("__")
d = reactive.Value("__")
```

```

w0 = reactive.Value("__")
L = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(

    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),

    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of MPa", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"One side of a beam rests on a sloping wall at angle  $\theta = \{theta()\}^\circ$ . The ot
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        theta.set(random.randrange(55,65,1))
        d.set(random.randrange(10,30,1))
        w0.set(random.randrange(5,20,1))
        L.set(random.randrange(10,50,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)

```

```

def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

    # Calculate the instructor's answer and determine if the user's answer is correct.
    By = w0()*L()/2/3
    Ay = w0()*L()/2-By
    Ax = Ay*math.tan(theta()*math.pi/180)
    Bx = Ax
    Babs = (By**2+Bx**2)**0.5
    P = Babs/2
    instr = P/((d()/2)**2*math.pi)*10**3

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

```

```

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

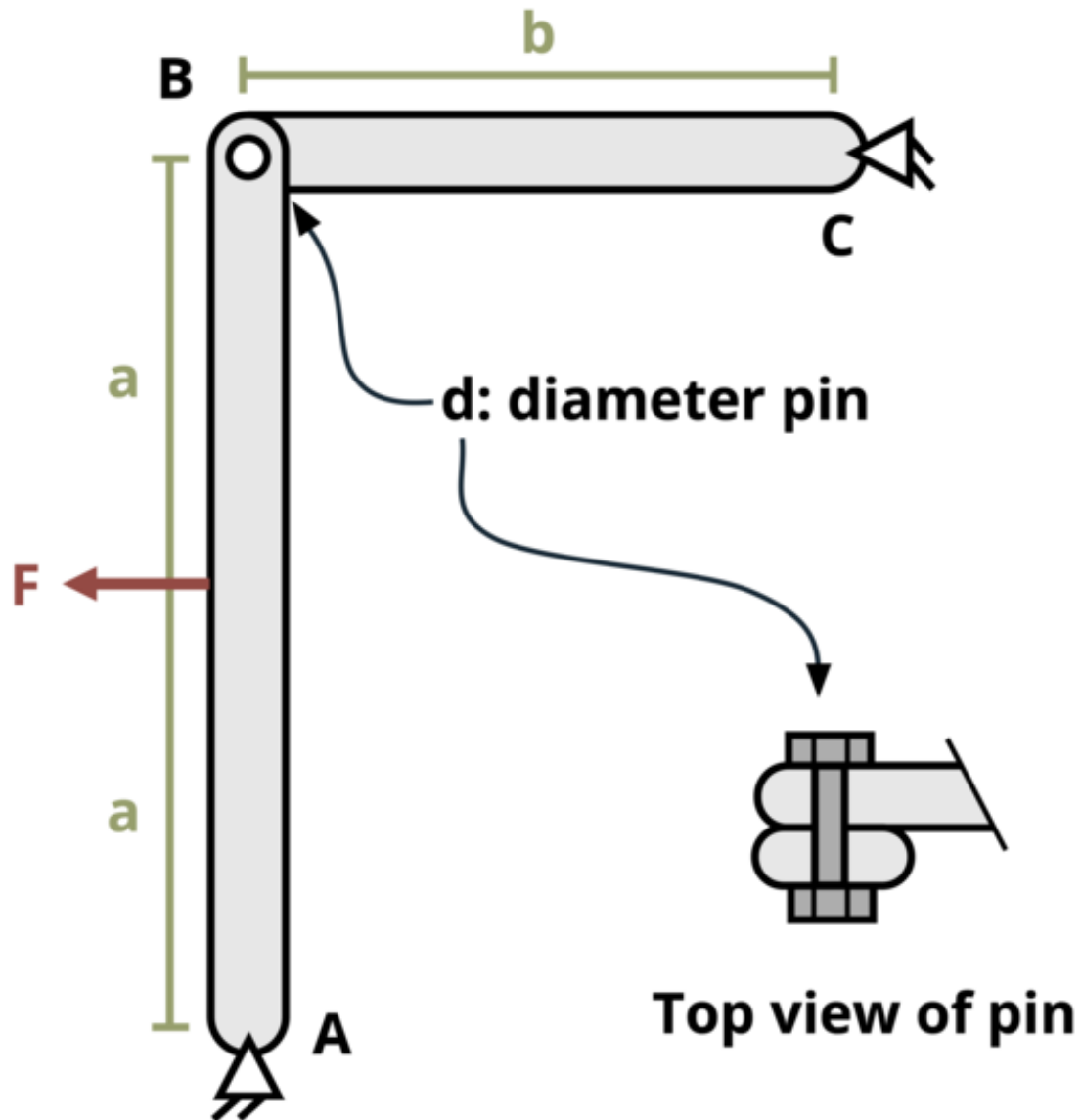
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written again
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 2.28 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
```



```

#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
problem_ID="177"
d=reactive.Value("__")
a=reactive.Value("__")
b=reactive.Value("__")
F=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two structural members are connected with a pin of diameter d =

```

```

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    d.set(random.randrange(20,100,5)/100)
    a.set(random.randrange(10,50,1)/10)
    b.set(round(a(),1)*round(random.randrange(12,17,1)/10))
    F.set(random.randrange(10,100,1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    FB = F()/2
    instr = FB/((d()/2)**2*math.pi)

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )

```

```

        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

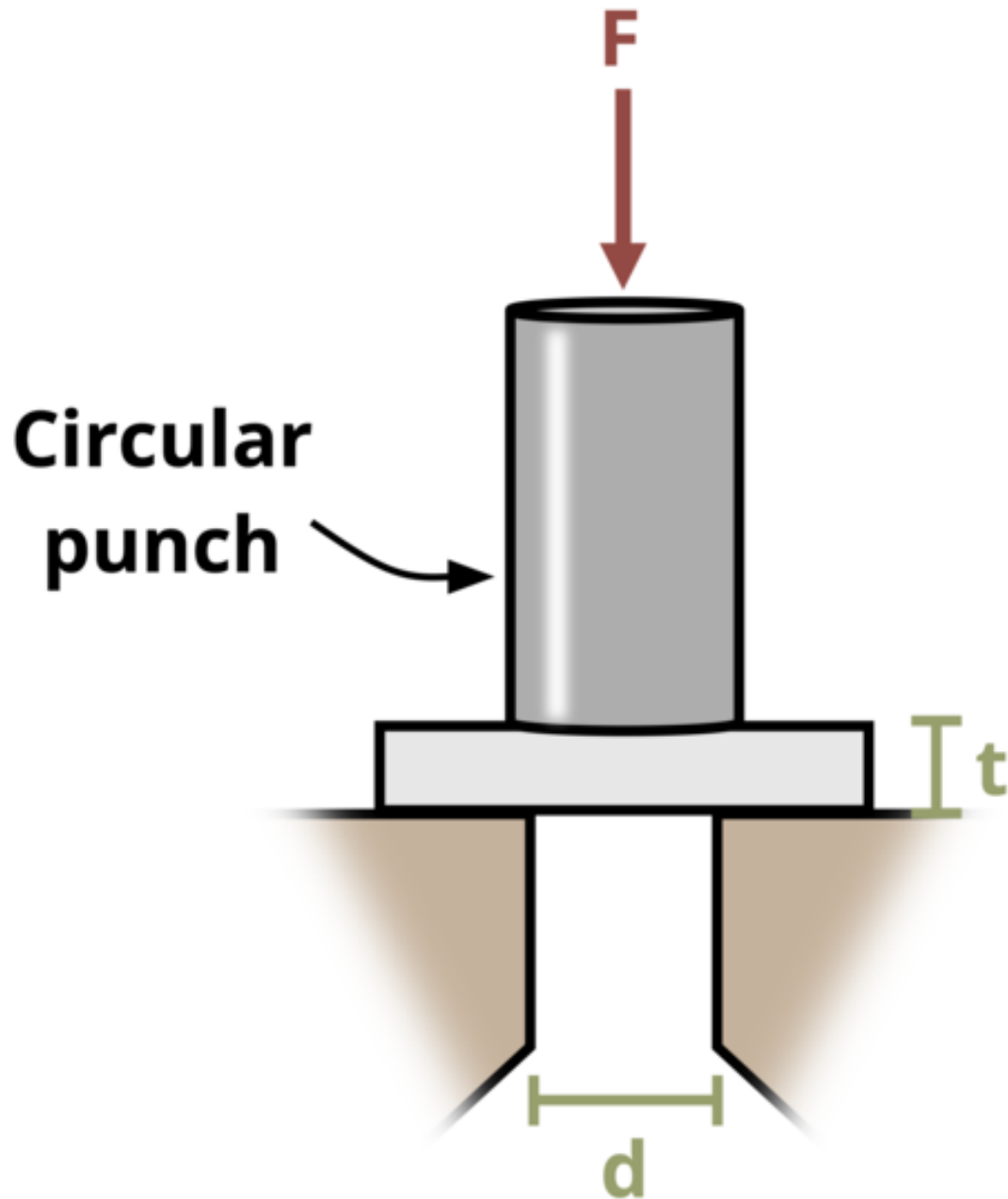
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 2.29 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
problem_ID="178"
d=reactive.Value("__")
t=reactive.Value("__")
tau_fail=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kips", placeholder="Please enter your an
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()

```

```

        return[ui.markdown(f"A circular hole punch is configured to punch a hold of diameter

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    d.set(random.randrange(25,95,5)/100)
    t.set(random.randrange(3,15,1)/10)
    tau_fail.set(random.randrange(20,40,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    A = t()*d()*math.pi
    instr = tau_fail()*A

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True

```

```

    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

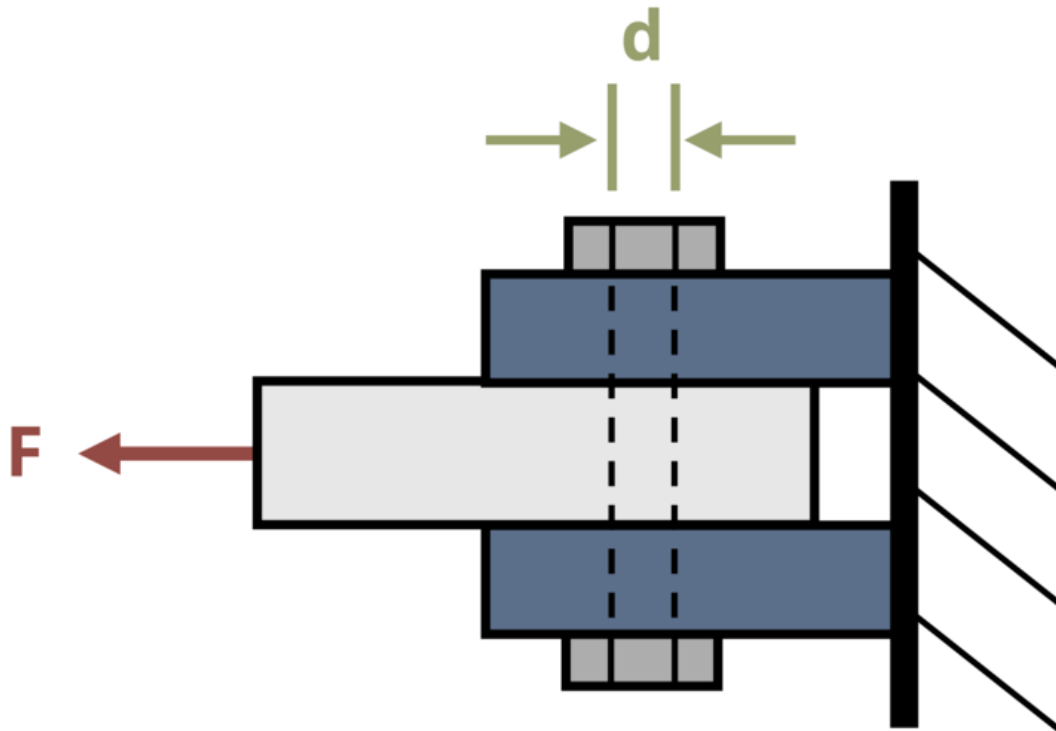
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 2.30 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
```



```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each
    V = F()/2
    A = math.pi*(d()/2)**2
    instr = V/(A)*10**3

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

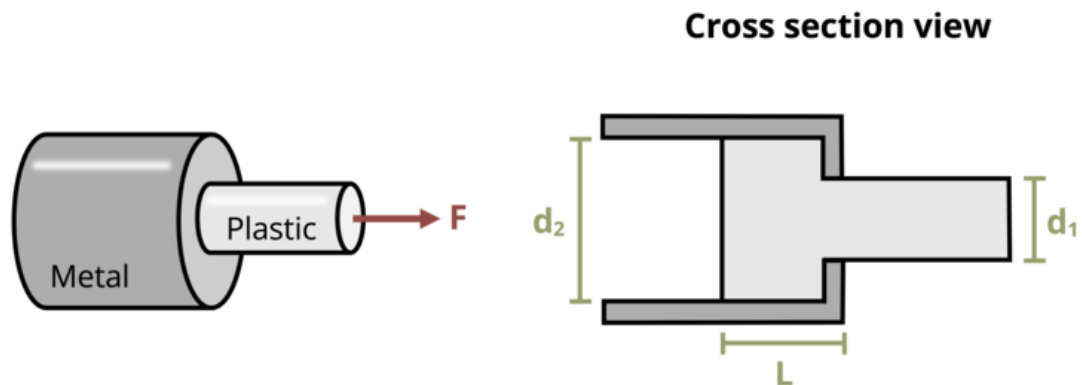
```

```
# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.31 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="181"
d1=reactive.Value("__")
d2=reactive.Value("__")
L=reactive.Value("__")
F=reactive.Value("__")
```



```

else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

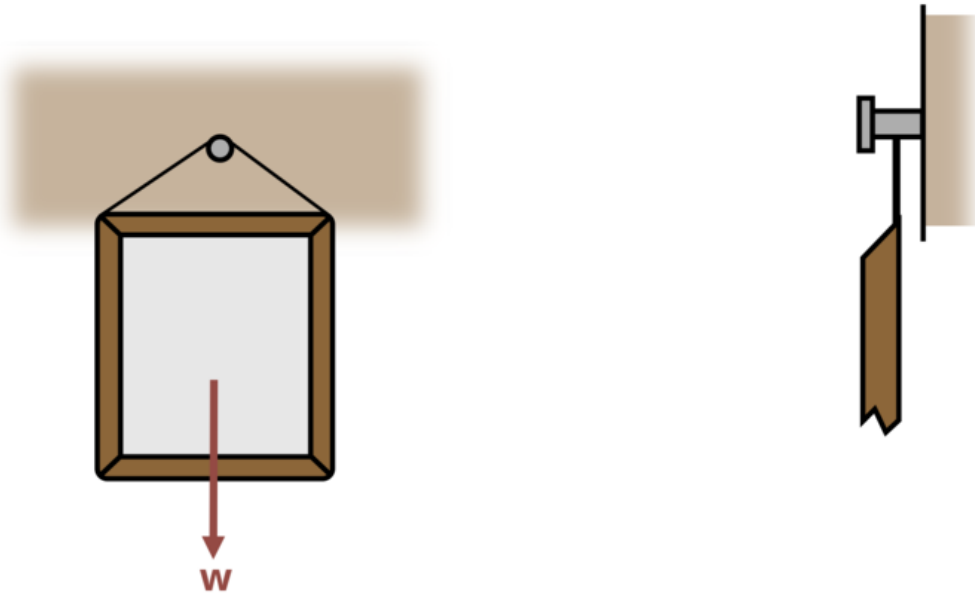
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 2.32 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
```



```

    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="182"
W=reactive.Value("__")
tau_fail=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of in", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A plaque weighing W = {W()} lb is hung from a rafter by a single

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        W.set(random.randrange(50,150,1))
        tau_fail.set(random.randrange(20,40,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        instr = (4*W()/(math.pi*tau_fail()*1000))*0.5

```

```

if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

# Store the most recent encoded attempt in a reactive value so it persists across sessions
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

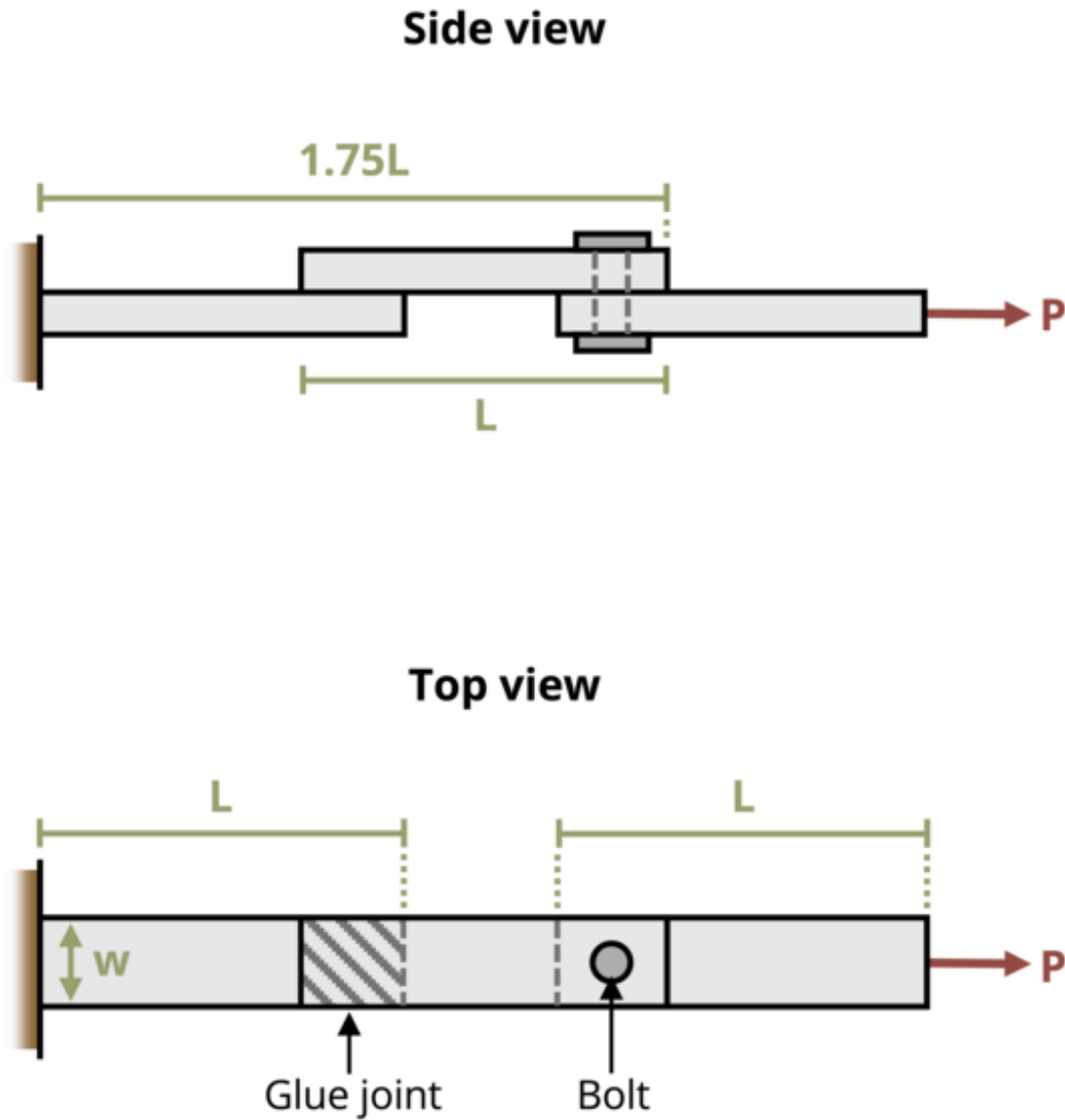
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

```

```
# App installation  
app = App(app_ui, server)
```


Problem 2.33 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]
```

```

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "167"
L = reactive.Value("__")
w = reactive.Value("__")
tau_glue = reactive.Value("__")
d = reactive.Value("__")
tau_bolt = reactive.Value("__")

attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(

    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),

    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of lb", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():

```

```

    randomize_vars()
    return [
        ui.markdown(
            f"Three bars of length L = {L()} in. and width w = {w()} in. are attached to
        )
    ]

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    L.set(random.randrange(4,10,1))
    w.set(random.randrange(10,30,1)/10)
    tau_glue.set(random.randrange(500,1000,10))
    d.set(random.randrange(3,8,1)/10)
    tau_bolt.set(random.randrange(50,150,1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.
    Pg = tau_glue()*w()*0.25*L()
    Pb = tau_bolt()*1000*((d()/2)**2*math.pi)
    if Pg >= Pb:
        instr = Pb
    else:
        instr = Pg

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

```

```

# Store the most recent encoded attempt in a reactive value so it persists across sessions
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(
    f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"
)

# Show feedback to the user.
feedback = ui.markdown(
    f"Your answer of {input.answer()} is {check}."
)
m = ui.modal(feedback, title="Feedback", easy_close=True)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = (
        session.encoded_attempt()
        if session.encoded_attempt is not None
        else "No attempts"
    )
    yield f"{final_encoded}\n\n"

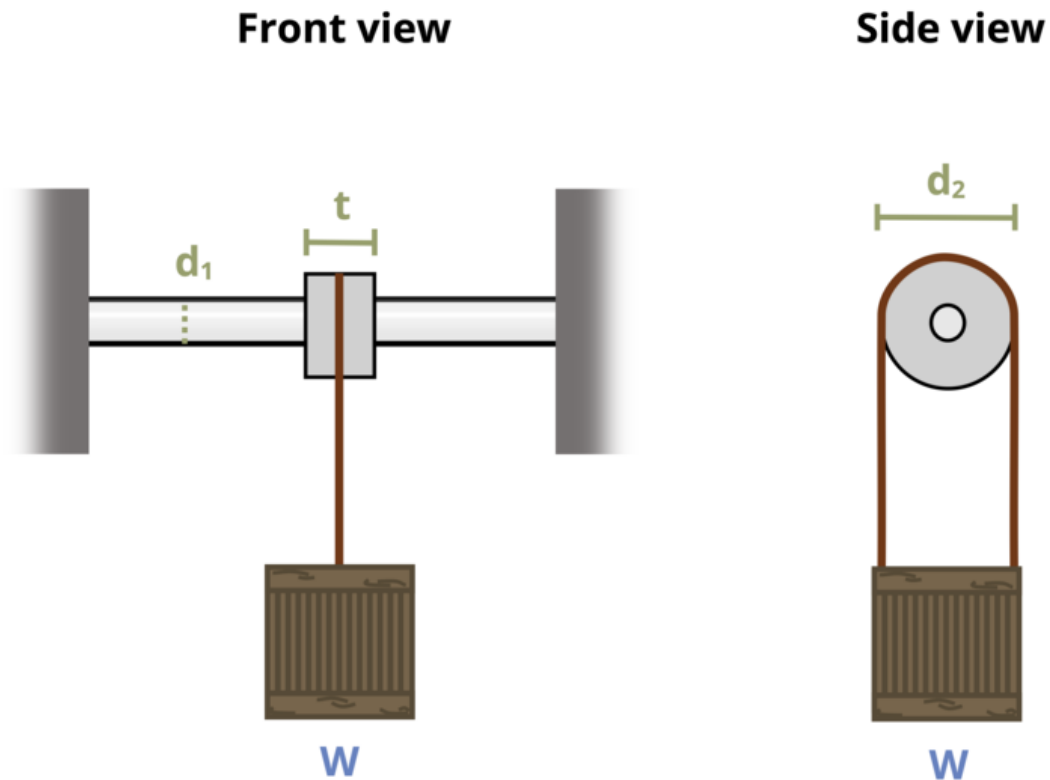
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(
            0.25
        ) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 2.38 - Bearing Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]
from shiny import App, render, ui, reactive
import random
import asyncio
import io
```

```

import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="166"
W=reactive.Value("__")
d1=reactive.Value("__")
d2=reactive.Value("__")
t=reactive.Value("__")
attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A crate of weight {W()} = lb hangs from a solid circular metal

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)

```

```

W.set(random.randrange(4000, 9000, 100))
d1.set(random.randrange(5, 30, 1)/10)
d2.set(d1()*3)
t.set(d1()*2)
@reactive.Effect
@reactive.event(input.submit)
def _():

    instr= W()/(d1()*t()*1000)

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else

```

```

yield f"{final_encoded}\n\n"

# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)

```

Problem 2.39 - Bearing Stress

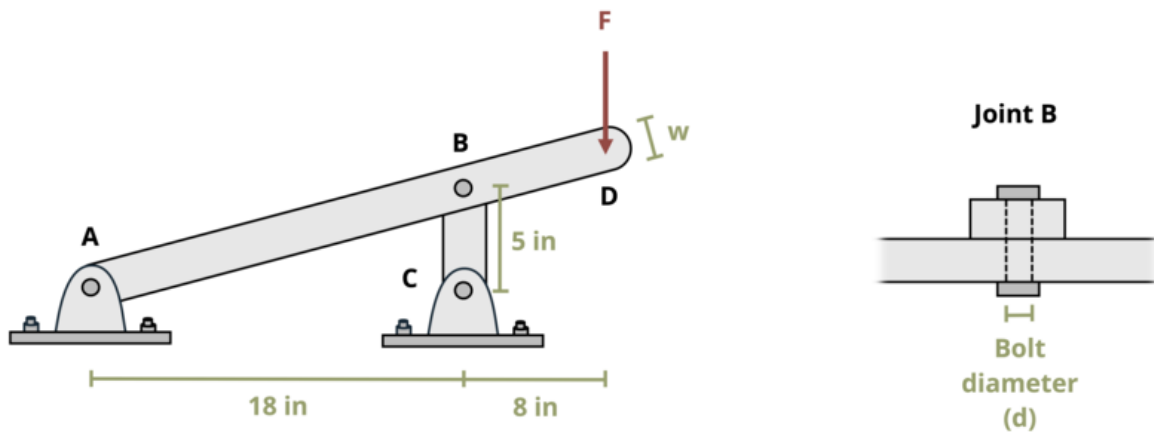


Figure 3: Figure 1: A link mechanism is connected with pins.

[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```

problem_ID="180"
F=reactive.Value("")
d=reactive.Value("")
t=reactive.Value("")
w=reactive.Value("")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A link mechanism is connected with pins of diameter d = {d()} in. The link is subjected to a tensile force of F = {F()} lb. The link has a thickness of t = {t()} in. and a width of w = {w()} in.")

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(200, 900, 10))
        d.set(random.randrange(20, 150, 10)/100)
        t.set(random.randrange(2, 10, 1)/10)
        w.set(round(d()*2, 2))

```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each
    Fb=((18+8)*F())/18
    instr= Fb/(d()*t())/1000
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

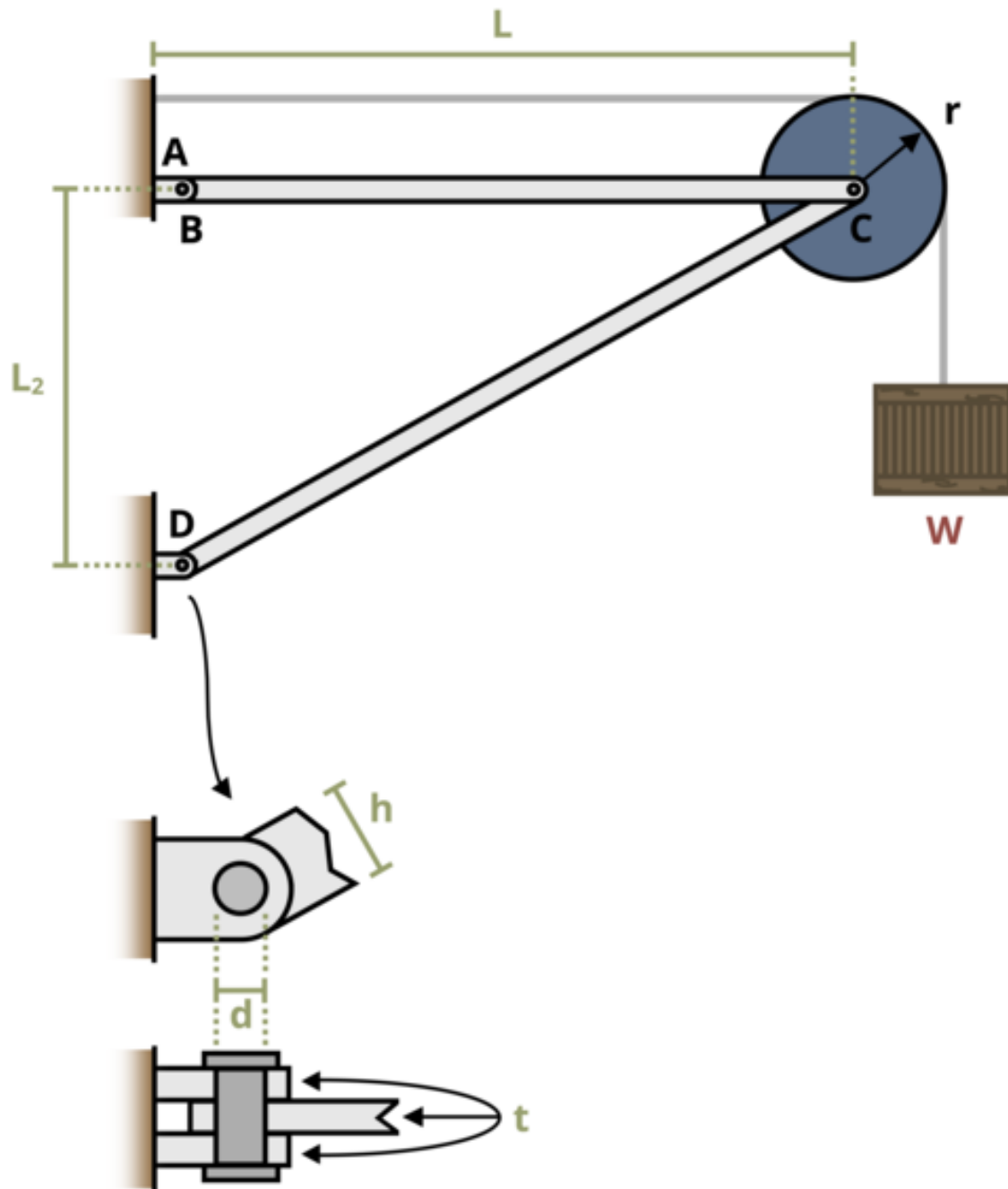
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

```

```
        # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)
```


Problem 2.40 - Bearing Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="161"
W=reactive.Value("__")
t=reactive.Value("__")
d=reactive.Value("__")
h=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui

```

```

def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A crate weighing  $W = \{W()\}$  kips is supported by the structure as

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    W.set(random.randrange(5,100,1))
    t.set(random.randrange(25, 75, 1)/100)
    d.set(t()*4)
    h.set(d()*2)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    theta = 29.05*math.pi/180
    FDC = W()/math.sin(theta)
    instr= FDC/(d()*t())
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(

```

```

        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

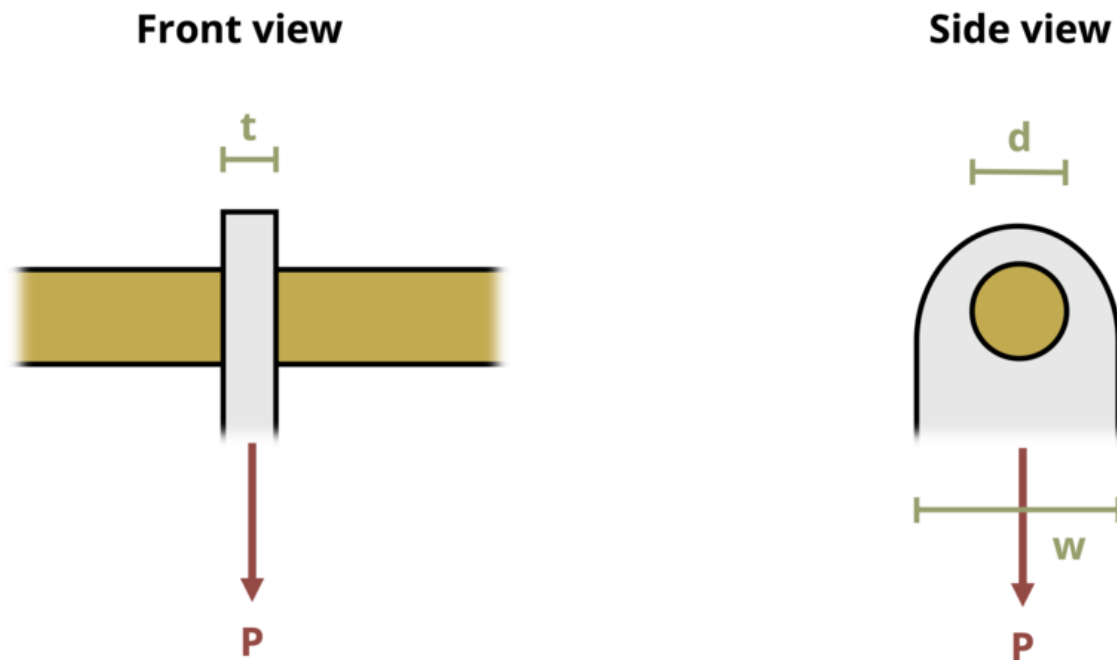
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 2.41 - Bearing Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```



```

def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each
    instr= 70*t()*d()
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header

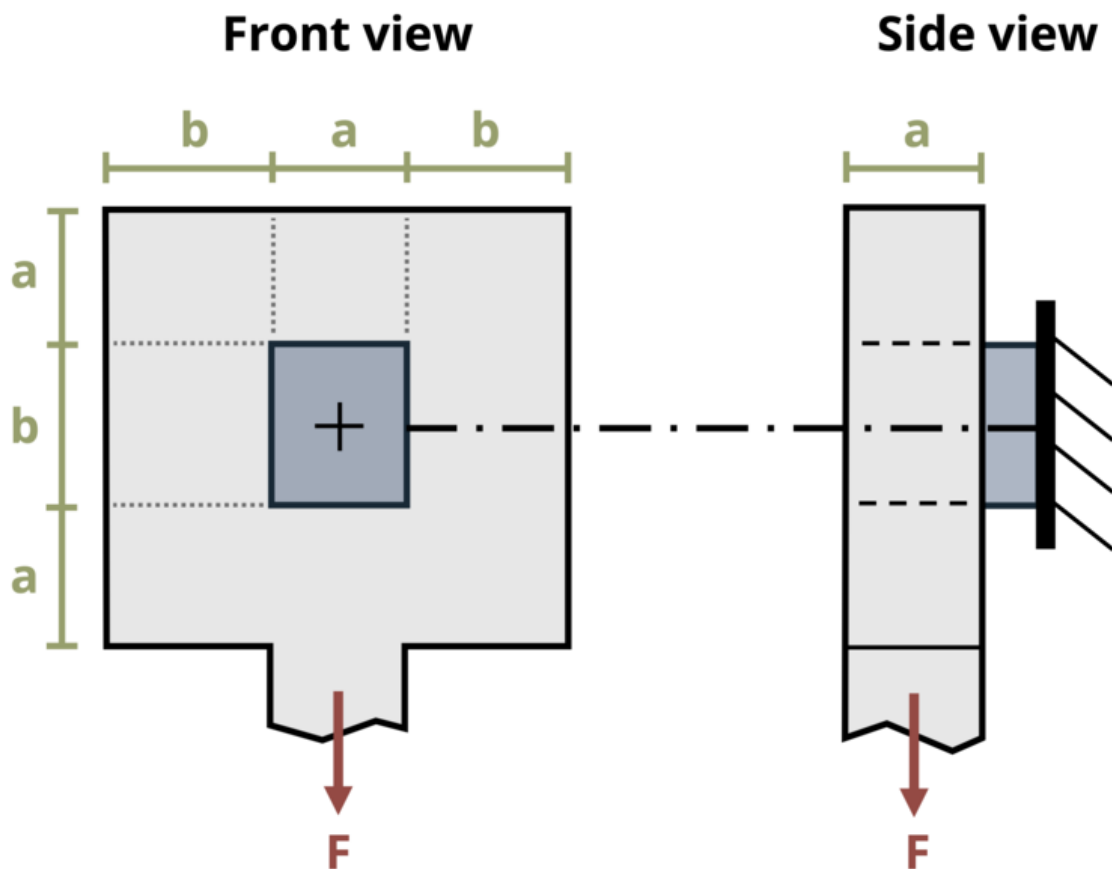
```



```
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.42 - Bearing Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
```

```

import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "172"
a = reactive.Value("__")
b = reactive.Value("__")
F = reactive.Value("__")

attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(

    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),

    ui.markdown("***Problem Statement***"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of MPa", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"A hanger is mounted to a wall with a rectangular bar as shown. Let dimensi
            )

```

```

]

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    a.set(random.randrange(100,300,2)/10)
    b.set(round(a()*1.2,1))
    F.set(random.randrange(10,70,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.
    sigmaN = F()/2/(b()*a())*10**3
    sigmaBH = F()/(a()*a())*10**3
    tauH = F()/2/(a()*a())*10**3
    tauR = F()/(a()*b())*10**3
    instr = max(sigmaN, sigmaBH, tauH, tauR)

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sessions
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(
        f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"
    )

```

```

    # Show feedback to the user.
    feedback = ui.markdown(
        f"Your answer of {input.answer()} is {check}."
    )
    m = ui.modal(feedback, title="Feedback", easy_close=True)
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = (
        session.encoded_attempt()
        if session.encoded_attempt is not None
        else "No attempts"
    )
    yield f"{final_encoded}\n\n"

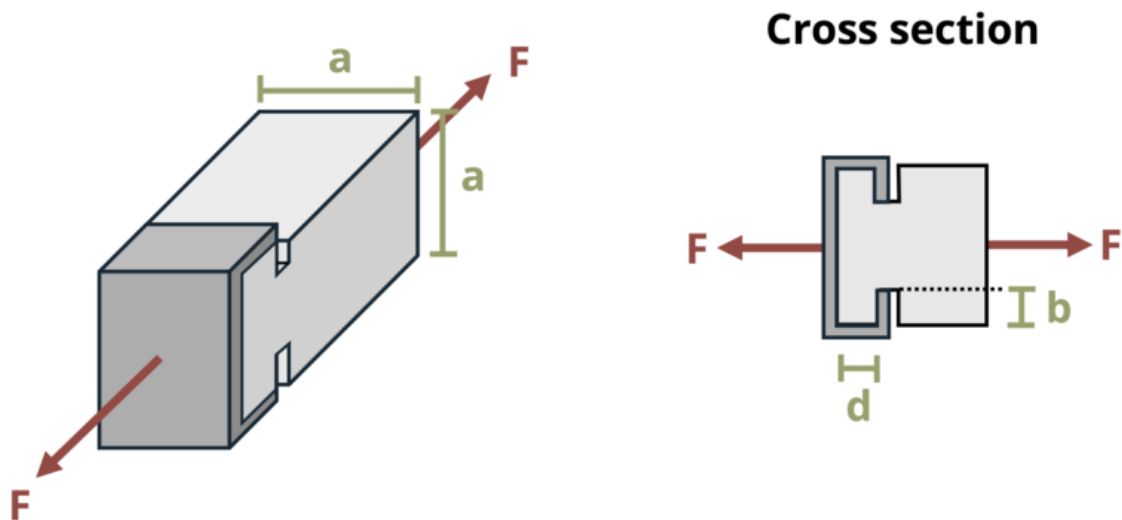
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(
            0.25
        ) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 2.43 - Bearing Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```

problem_ID = "176"
sigma_s = reactive.Value("__")
sigma_b = reactive.Value("__")
F = reactive.Value("__")
a = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer1,Answer2,Feedback\n"]

app_ui = ui.page_fluid(
  #ui.markdown("**Please enter your ID number from your instructor and click to generate y
  #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
  #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
  ui.markdown("**Problem Statement**"),
  ui.output_ui("ui_problem_statement"),
  ui.input_text("answer1", "Your Answer 1 in units of in", placeholder="Please enter your a
  ui.input_text("answer2", "Your Answer 2 in units of in", placeholder="Please enter your a
  ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
  #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
  # Initialize a counter for attempts
  attempt_counter = reactive.Value(0)

  @output
  @render.ui
  def ui_problem_statement():
    randomize_vars()
    return [
      ui.markdown(
        f"A metal end cap is secured to a wood structural member with grooves as show
      )
    ]

  #@reactive.Effect
  #@reactive.event(input.generate_problem)
  def randomize_vars():
    random.seed(101)
    sigma_s.set(random.randrange(20,50,1)/10)
    sigma_b.set(sigma_s()*2)
    F.set(random.randrange(15,40,1))
    a.set(random.randrange(50,100,1)/10)

```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

    # Calculate the instructor's answer and determine if the user's answer is correct.
    P = F()/2
    Ab = P/sigma_b()
    instr1 = Ab/a()
    As = P/sigma_s()
    instr2 = As/a()

    correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
    correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)

    if correct1 and correct2:
        check = f"{'both correct.'}"
    elif correct1:
        check = f"{'correct for answer 1 and incorrect for answer 2.'}"
    elif correct2:
        check = f"{'incorrect for answer 1 and correct for answer 2.'}"
    else:
        check = f"{'both incorrect.'}"

    correct_indicator = "JL" if correct1 and correct2 else "JG"

    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    #session.encoded_attempt = reactive.Value(encoded_attempt)
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

    feedback = ui.markdown(f"Your answers of {input.answer1()} and {input.answer2()} are
    m = ui.modal(feedback, title="Feedback", easy_close=True)
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():

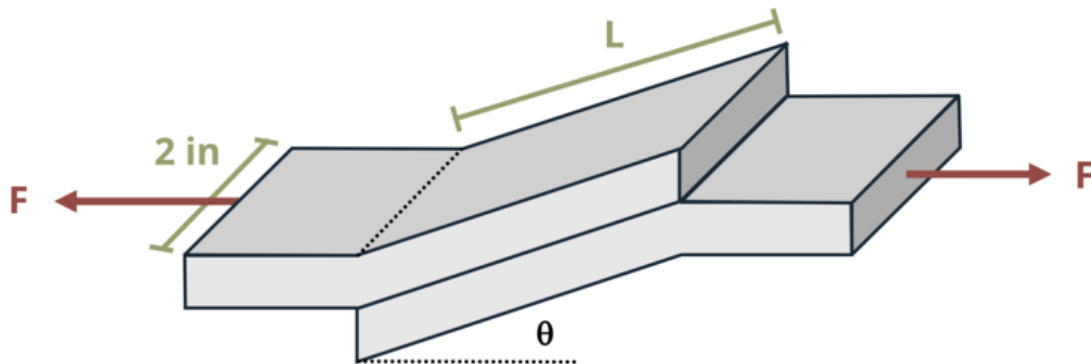
```



```
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
        yield f"{final_encoded}\n\n"
        yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
        for attempt in attempts[1:]:
            await asyncio.sleep(0.25)
            yield attempt

app = App(app_ui, server)
```

Problem 2.47 - Stress on an Inclined Plane



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="153"
F=reactive.Value("__")
L=reactive.Value("__")
```

```

θ=reactive.Value("_")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two slanted brackets are glued together as shown. If  $F = \{F()\}$ 

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(200, 800, 10))
        L.set(random.randrange(20, 80, 1)/10)
        θ.set(random.randrange(15, 30, 1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

        instr= (F()*math.cos(math.radians(θ())))/(L()*2))
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"

```

```

else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

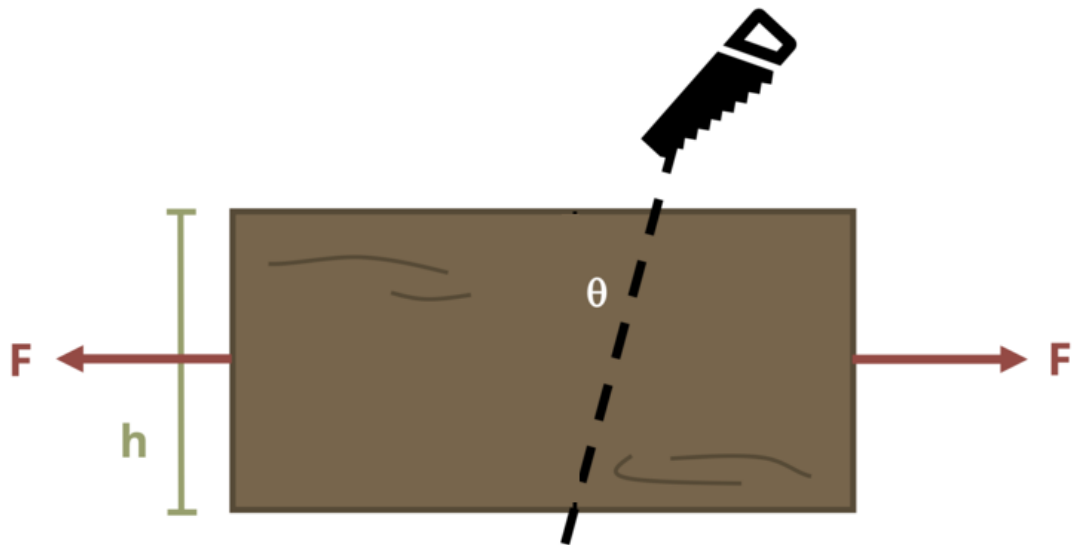
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 2.48 - Stress on an Inclined Plane



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
```

```

        return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="156"
F=reactive.Value("__")
h=reactive.Value("__")
θ=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A 2 inch thick board is cut and then glued back together along a

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(2000, 6000, 100))
        h.set(random.randrange(50, 150, 1)/10)
        θ.set(72)#(random.randrange(10, 20, 1))

    @reactive.Effect
    @reactive.event(input.submit)

```

```

def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

    instr= (F()/(2*h()))*(math.sin(math.radians(θ()))**2)
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ

```

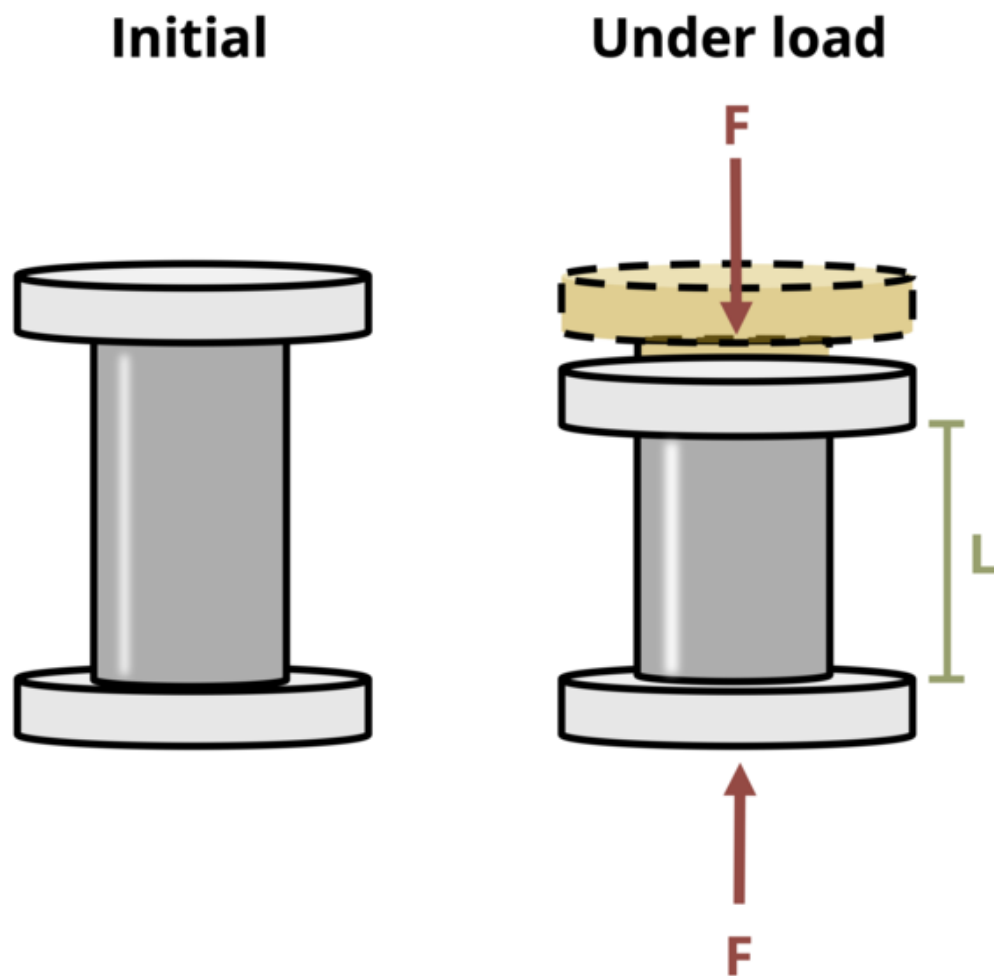
```
        for attempt in attempts[1:]: # Skip the first element which is the header
            await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```


Part IV

Chapter 3 Problems

Problem 3.1 - Normal Strain



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true  
#| viewerHeight: 600  
#| components: [viewer]
```

```

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="196"
percent=reactive.Value("__")
F=reactive.Value("__")
L=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A rubber cylinder experiences a {percent()}% reduction in length

```

```

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    percent.set(random.randrange(10,25,1))
    F.set(random.randrange(20,100,1))
    L.set(random.randrange(10,100,1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each submit
    instr = -percent()/100

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

    # Store the most recent encoded attempt in a reactive value so it persists across sessions
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

```

```

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 3.2 - Normal Strain

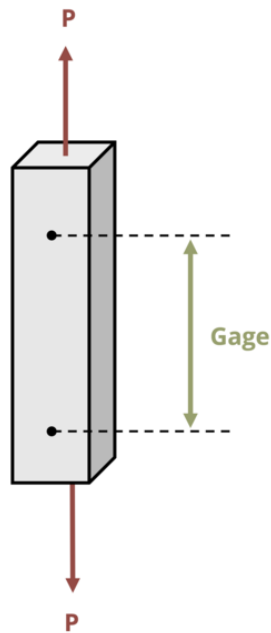


Figure 4: Figure 1: A rectangular prism is subjected to a tension test.

[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
```

```

from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="650"
L=reactive.Value("__")
Lp=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of mm/mm", placeholder="Please enter your a
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"During a tension test of a rectangular prism, the original gage

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(100, 400, 1))
        Lp.set(L()+random.randrange(100, 300, 1)/100)

```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    instr= (Lp()-L())/L()
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

```



```
        # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)
```

Problem 3.3 - Normal Strain

[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="139"
L1=reactive.Value("__")
L2=reactive.Value("__")
FA=reactive.Value("__")
FB=reactive.Value("__")
E = 30*10**6

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y",
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of inches", placeholder="Please enter your a
```

```

    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two cylinders are stacked on top of one another and two forces a

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        FA.set(random.randrange(300, 700, 10))
        FB.set(random.randrange(100, 300, 10))
        L1.set(random.randrange(2, 7, 1))
        L2.set(L1() * 1.3)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

        instr= (FA()*L1())/((math.pi*(5/2)**2)*E) + (FB()*L2())/((math.pi*(3/2)**2)*E)
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)

```

```

#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sub
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

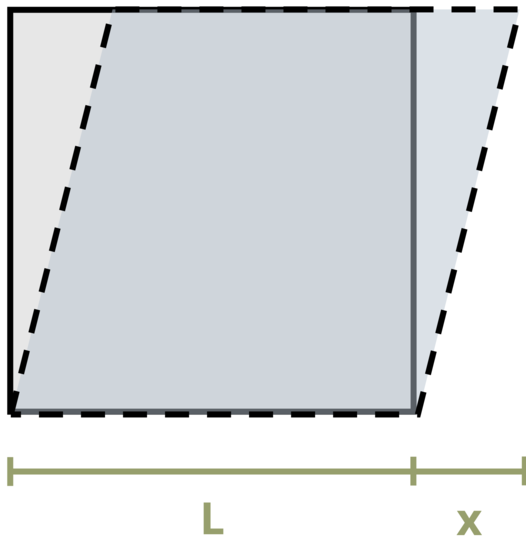
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 3.7 - Shear Strain



[Problem adapted from © Chris Galitz CC BY

NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```

problem_ID="652"
L=reactive.Value("__")
x=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of radians", placeholder="Please enter your
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A square plate is deformed due to shear with the new shape shown

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(200, 500, 10))
        x.set(random.randrange(20, 50, 1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        instr= math.atan(x())/L()
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):

```

```

        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

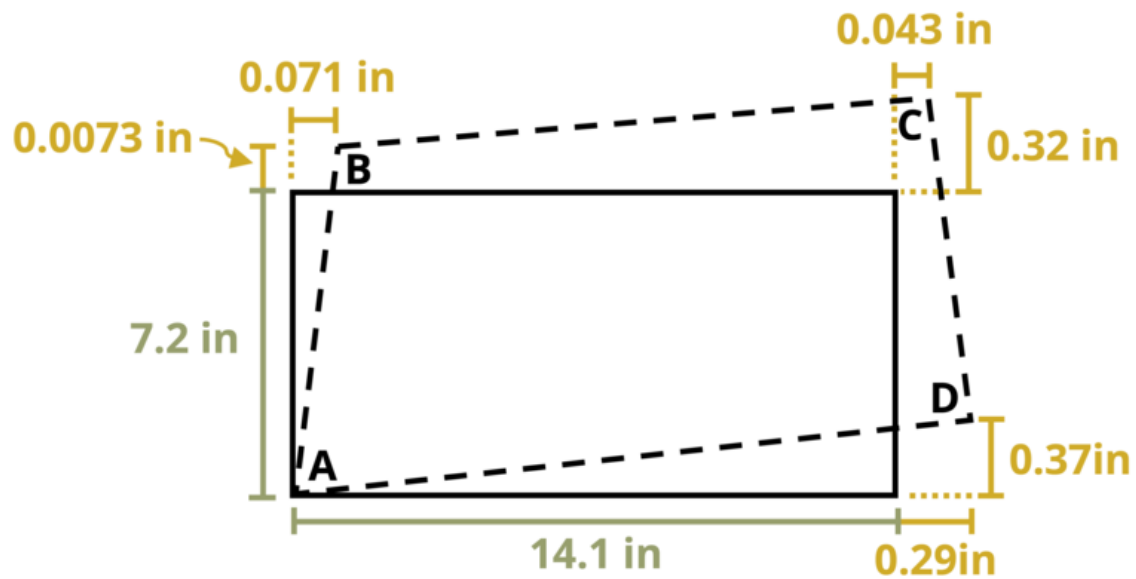
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

```

```
# App installation  
app = App(app_ui, server)
```


Problem 3.8 - Shear Strain



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
```

```

    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="653"
h=reactive.Value("__")
L=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of radians", placeholder="Please enter your
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A rectangular plate of dimensions L = {L()} in. and h = {h()} in
        )
    ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        h.set(7.2)#(random.randrange(50,100,1)/10)
        L.set(14.1)#(h()+random.randrange(50,100,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():

```

```

    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

# Calculate the instructor's answer and determine if the user's answer is correct.
alpha2 = 0.071/(h() + 0.0073)
alpha3 = 0.37/(L() + 0.29)
instr = alpha2 + alpha3
if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sub
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

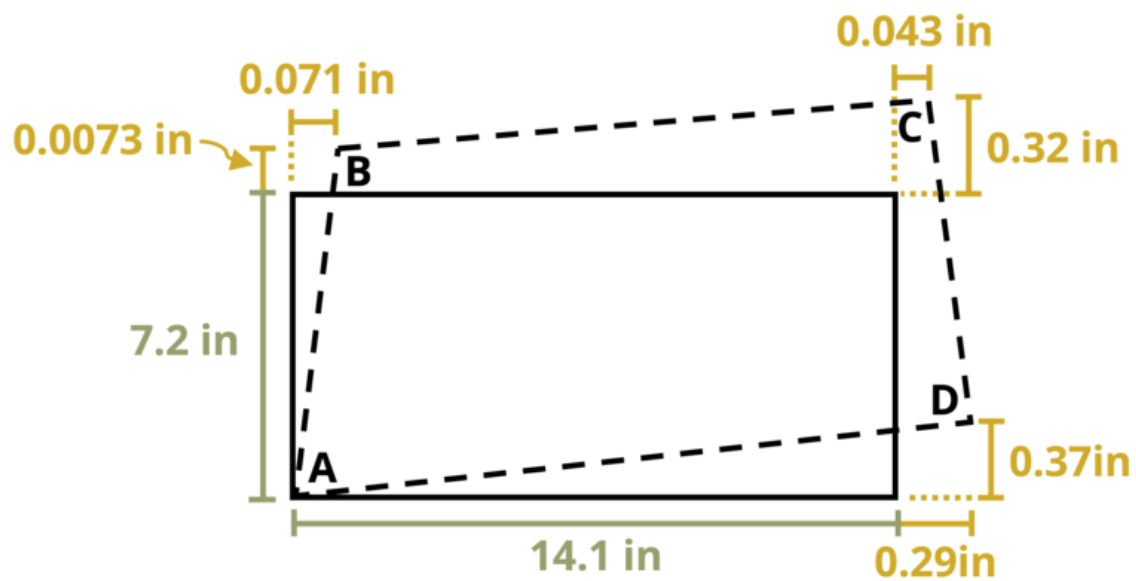
```

```
# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)
```

Problem 3.9 - Shear Strain



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
```

```

    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="654"
t=reactive.Value("__")
x=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of radians", placeholder="Please enter your
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two plates are connected by a neoprene pad of thickness  $t = \{t$ 
        )
    ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        t.set(random.randrange(10,30,1))
        x.set(random.randrange(10,30,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():

```

```

    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

    # Calculate the instructor's answer and determine if the user's answer is correct.
    y = (t()2/4-x()2/4)/x()
    r = y+x()/2
    theta = math.acos(x()/r)
    instr = math.pi/2-theta
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

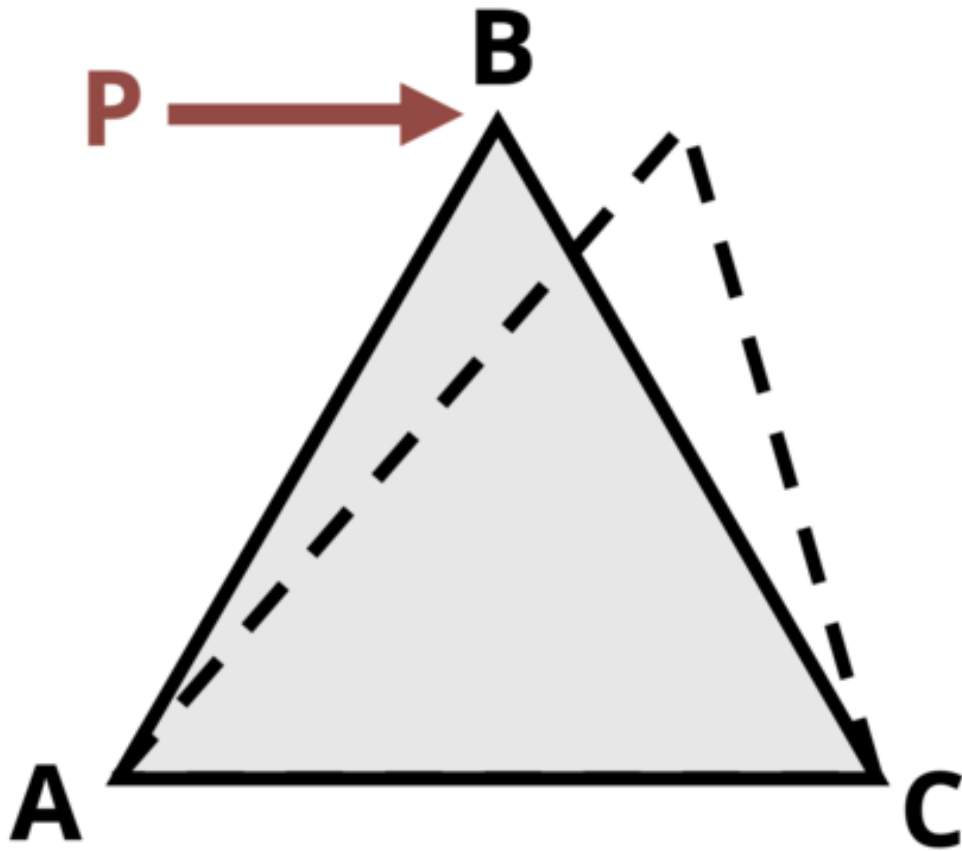
```

```
# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)
```


Problem 3.10 - Shear Strain



**Equilateral with
side length 320 mm**

[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="655"
L=reactive.Value("__")
strain=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of mm", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()

```

```

        return[ui.markdown(f"A triangular plate of side length L = {L()} mm is subjected to a
        )
    ]

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    L.set(random.randrange(100,800,10))
    strain.set(random.randrange(2,20,1)/100)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    ) # Increment the attempt counter on each submission.

    # Calculate the instructor's answer and determine if the user's answer is correct.
    ang = math.pi/3-2/3*strain()
    NewHor = 2*L()*math.sin(math.pi/3)/math.tan(ang)
    instr = NewHor-L()
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.

```

```

        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

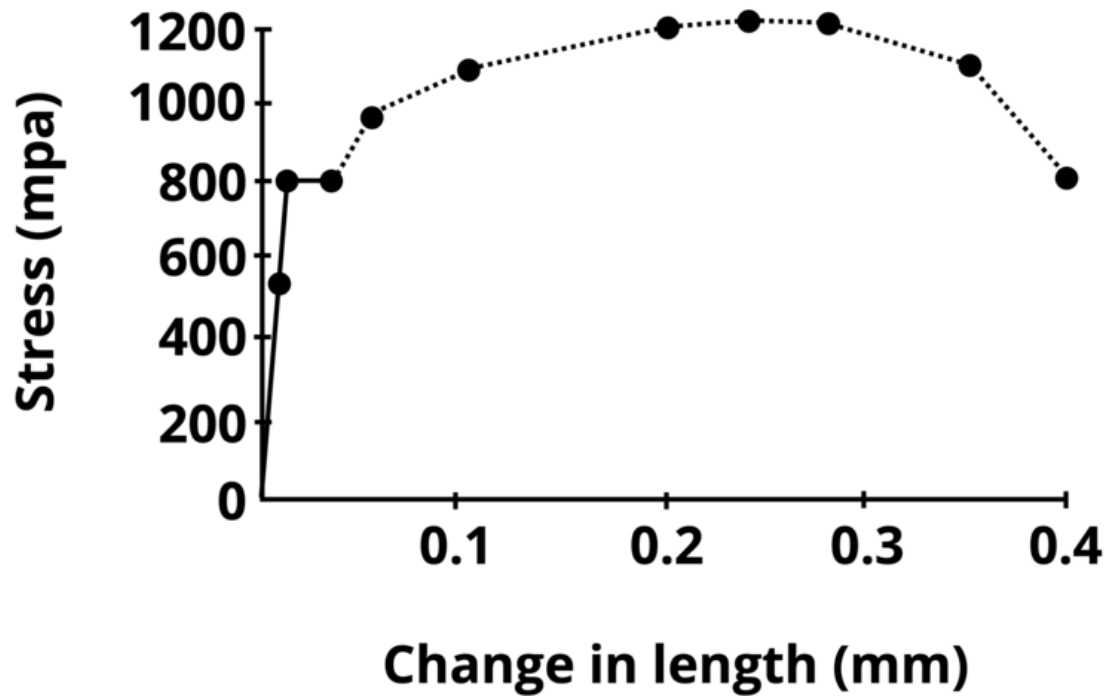
# App installation
app = App(app_ui, server)

```

Part V

Chapter 4 Problems

Problem 4.2 - Stress/Strain Diagram



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
```

```

import pandas as pd
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="656"
L=reactive.Value("__")
P=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of mm", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"The stress-strain diagram for a steel specimen is shown. A rect

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(500,2000,10))
        P.set(random.randrange(180,675,1))

```



```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each
    sigma=P()/900*1000
    e = sigma/200000
    instr = L()*e

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

```

```
# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)
```

Problem 4.5 - Hooke's Law

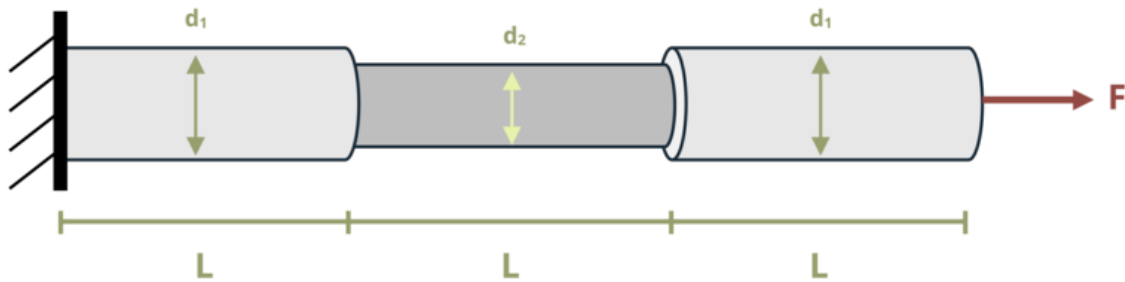


Figure 5: Figure 1: A single force pulls on three cylindrical rods that are fixed to a wall.

[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="188"
F=reactive.Value("__")
L=reactive.Value("__")
```

```

d1=reactive.Value("__")
d2=reactive.Value("__")
Esteel = 29000
Ealuminum = 10000

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of inches/inches", placeholder="Please enter
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A single force F = {F()} kips pulls on three cylindrical rods, e

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(20, 300, 1)/10)
        L.set(random.randrange(50, 200, 1)/10)
        d1.set(random.randrange(10, 50, 1)/10)
        d2.set(round(d1()/2, 2))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():

```

```

attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each
instr=(F()/(((d2()/2)**2)*math.pi))/Esteel
if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across su
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

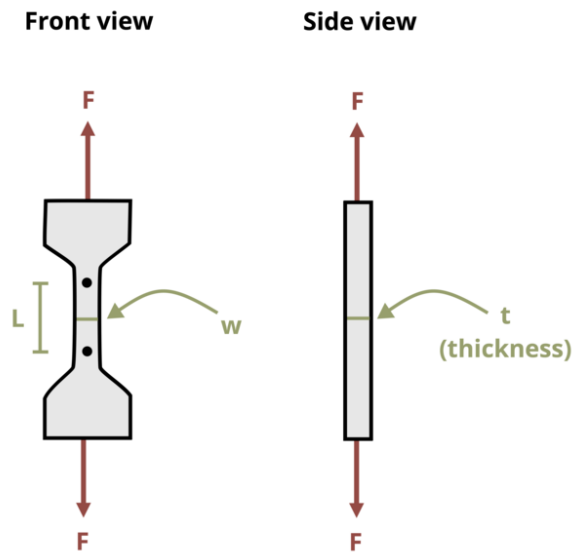
    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed

```

```
yield attempt
```

```
# App installation  
app = App(app_ui, server)
```

Problem 4.6 - Hooke's Law



[Problem adapted from © Kurt Gramoll CC

BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```

problem_ID="194"
F=reactive.Value("__")
L=reactive.Value("__")
w=reactive.Value("__")
t=reactive.Value("__")
dL=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A polymer test specimen is subjected to an axial load of  $F = \{F\}$ ")]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(100, 500, 1)/10)
        L.set(random.randrange(50, 150, 1)/10)
        w.set(round(L()*0.375, 2))
        t.set(random.randrange(10, 50, 1)/100)
        dL.set(random.randrange(3, 9, 1)/100)

```



```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    instr= (F()*L())/(dL()*(w()*t()))
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

```

```
        # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)
```

Problem 4.7 - Hooke's Law

[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="657"
d=reactive.Value("__")
L=reactive.Value("__")
P=reactive.Value("__")
elong=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
```

```

    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A round bar of diameter d = {d()} in. and length L = {L()} in. :

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        d.set(random.randrange(3,15,1)/10)
        L.set(random.randrange(100,200,1)/10)
        P.set(random.randrange(750,1500,10))
        elong.set(random.randrange(50,200,1)/1000)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each
        A = math.pi*(d()*2)/4
        sigma = P()/A
        epsilon = elong()/L()
        instr= sigma/epsilon/1000
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

```

```

# Store the most recent encoded attempt in a reactive value so it persists across sub
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 4.10 - Poisson's Ratio



Figure 6: Figure 1: A circular rod is placed in tension with an axial load.

[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="206"
P=reactive.Value("__")
E=reactive.Value("__")
L=reactive.Value("__")
```

```

d=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A circular rod of an unknown metallic alloy is placed in tension

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        P.set(random.randrange(20, 200, 1)/10)
        L.set(random.randrange(10, 20, 1))
        E.set(random.randrange(20, 40, 1)/100)
        d.set(round((.00014*L())/(.0035*E()), 1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

        instr= -(-.00014/d())/(.0035/L())
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"

```

```

        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

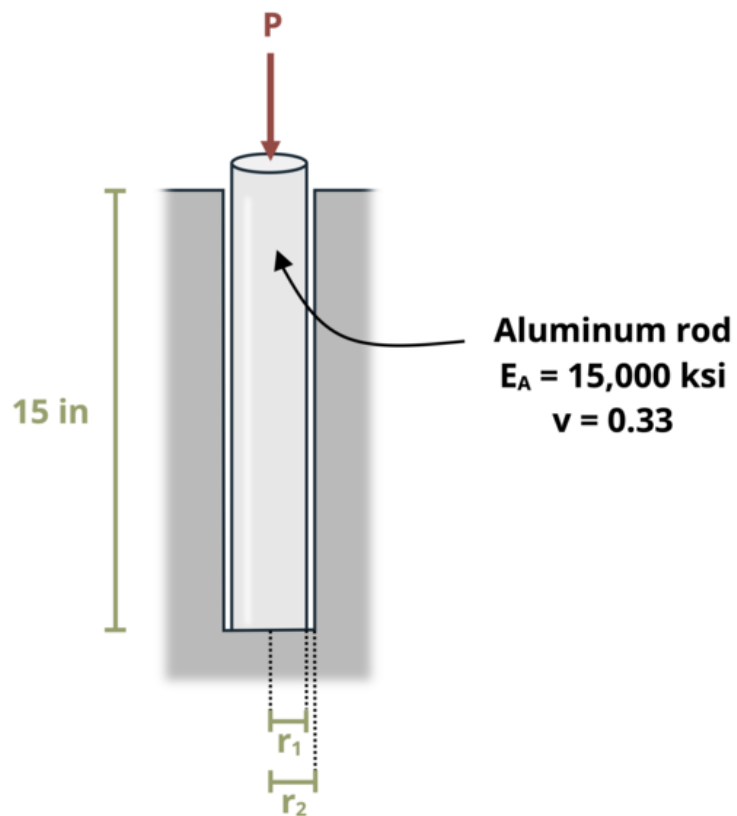
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 4.11 - Poisson's Ratio



[Problem adapted from © Kurt

Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
```

```

import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="213"
r1=reactive.Value("__")
r2=reactive.Value("__")
E=15000
v=0.33

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kip", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"An aluminum circular rod of radius  $r_1$  in is

    #@reactive.Effect
    #@reactive.event(input.generate_problem)

```

```

def randomize_vars():
    random.seed(101)
    r1.set(random.randrange(10, 50, 1)/10)
    r2.set((r1() + random.randrange(1, 10, 1)/1000))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    Ex = (r1() - r2())/r1()
    sigmay = (Ex*E)/(-v)
    instr= sigmay*math.pi*r1()**2
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():

```

```

# Start the CSV with the encoded attempt (without label)
final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
yield f"{final_encoded}\n\n"

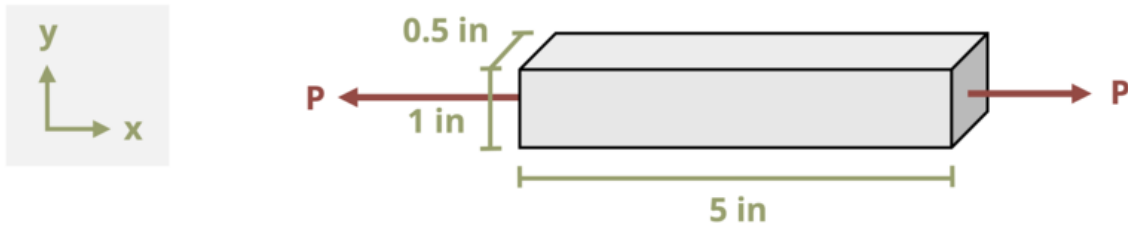
# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)

```

Problem 4.12 - Poisson's Ratio



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="216"
d1=reactive.Value("__")
d2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
```

```

    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A rectangular bar is pulled in tension by a load P in the x-dir

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        d1.set(random.randrange(20, 50, 1)/1000)
        d2.set(random.randrange(10, 20, 1)/10000)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        instr= -(-d2()/1)/(d1()/5)
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)

```

```

random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across su
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

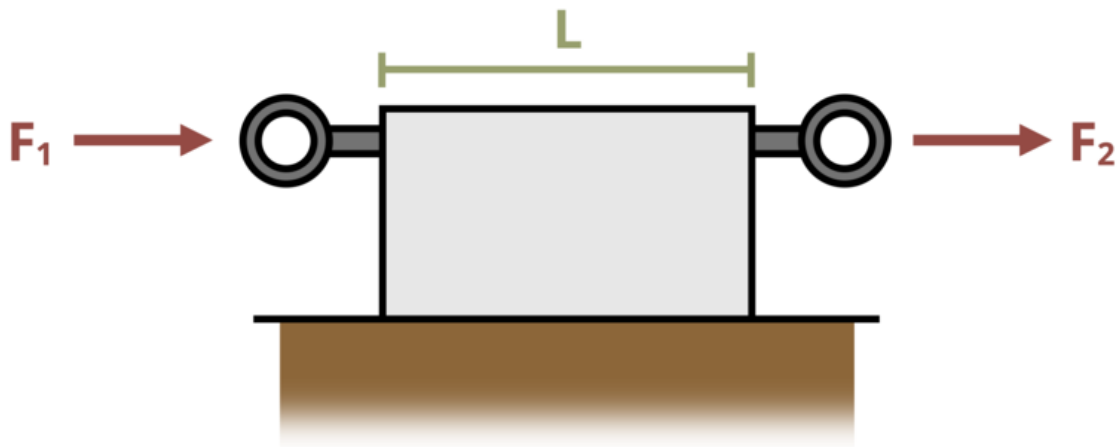
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 4.17 - Shear Stress/Strain



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="218"
```



```

F2=reactive.Value("__")
F1=reactive.Value("__")
L=reactive.Value("__")
d=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of radians", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Force  $F_1$  lb and  $F_2$  lb are applied to a spring with constant  $k = 100$  lb/ft. The spring is initially at its natural length. The displacement of the spring is  $x$  ft. The mass of the spring is  $m$  lb. The damping coefficient is  $c$  lb/ft-sec. The equation of motion is  $m\ddot{x} + c\dot{x} + kx = F_1 + F_2$ . The initial conditions are  $x(0) = 0$  and  $\dot{x}(0) = 0$ . The solution is  $x(t) = \frac{F_1 + F_2}{k} (1 - e^{-\frac{c}{2m}t}) + \frac{F_1 - F_2}{k} e^{-\frac{c}{2m}t}$ .")]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F2.set(random.randrange(10,50,1))
        F1.set(F2()+random.randrange(5,20,1))
        L.set(random.randrange(30,60,1))
        d.set(random.randrange(30,60,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each submit
        tau = (F1()+F2())/(L()*d())

```

```

G = 500/(2*(1+0.49))
instr = tau/G
if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

# Store the most recent encoded attempt in a reactive value so it persists across sessions
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written again
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed

```

```
        yield attempt

# App installation
app = App(app_ui, server)
```

Problem 4.18 - Thermal Strain

[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="658"
L=reactive.Value("__")
Ti=reactive.Value("__")
Tf=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of m/m", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
```

```

)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A copper pipe ( = 17x10-6 /°C) of length L = {L()} m

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(50, 100, 1)/10)
        Ti.set(random.randrange(30, 50, 1))
        Tf.set(Ti()-random.randrange(10, 25, 1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        instr= 17*10**-6*(Tf()-Ti())
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        # Store the most recent encoded attempt in a reactive value so it persists across sul
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt

```

```

        attempts.append(f"{datetime.now()}", {attempt_counter()}, {input.answer()}, {check}\n

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 4.19 - Thermal Strain

[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="659"
L=reactive.Value("__")
Ti=reactive.Value("__")
Tf=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of in.", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
```

```

)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Upon plant startup, a steel steam pipe ( = 6.5 x 10-6/sup>-6</sup>

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(50, 200, 5))
        Ti.set(random.randrange(50, 75, 1))
        Tf.set(random.randrange(300, 500, 1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        instr= 6.5*10**-6*(Tf()-Ti())*L()*12
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        # Store the most recent encoded attempt in a reactive value so it persists across sul
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt

```



```

        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

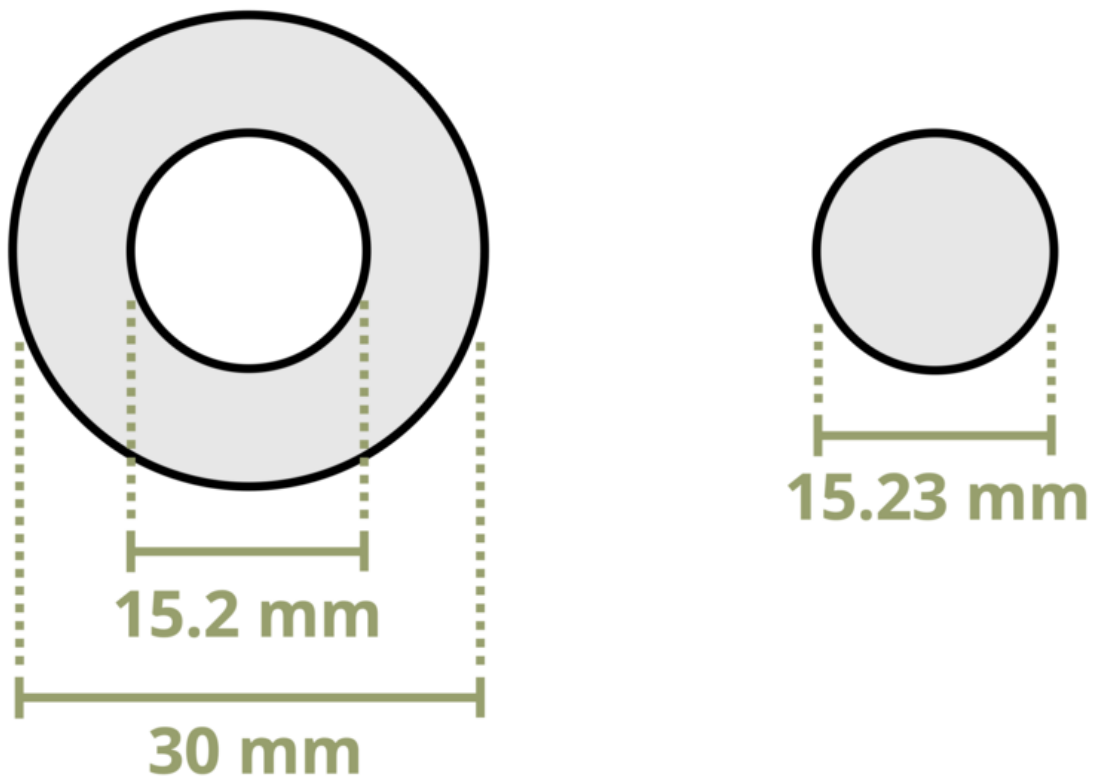
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 4.20 - Thermal Strain



{fig-
alt="Picture with a coin made of a nickel outer core and a copper inner core. The nickel is a ring with a outer diameter of 30 mm and an inner diameter of 15.2 mm. The copper is circular with a diameter of 15.23 mm." [Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
```

```

import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="660"
Ti=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of \u00b0C", placeholder="Please enter your
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A coin is to be made with a nickel outer ring and a copper ( =

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        Ti.set(random.randrange(10,20,1))

```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    instr= Ti()-115.87
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

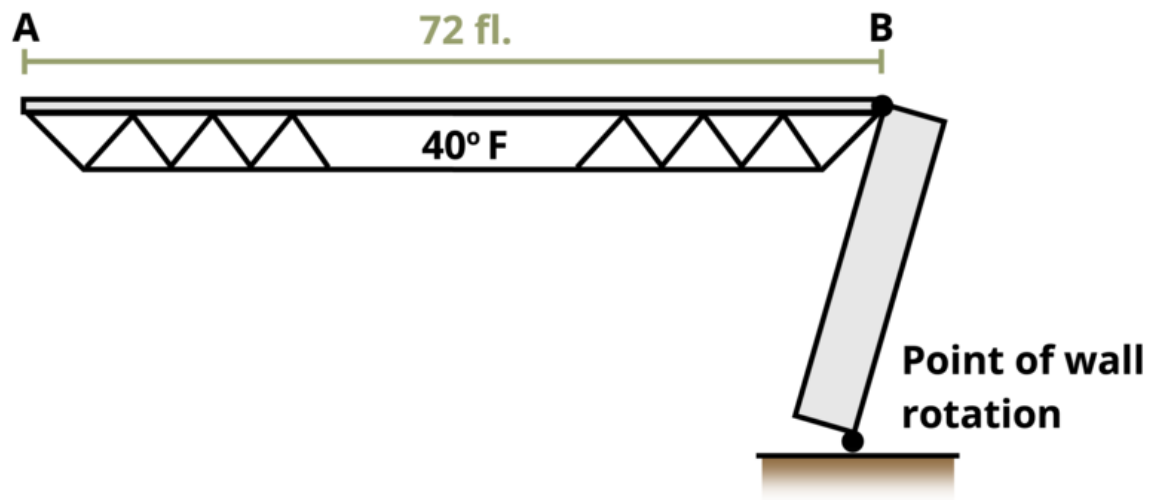
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

```

```
        # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)
```

Problem 4.21 - Thermal Strain



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```

problem_ID="661"
L=reactive.Value("")
Ti=reactive.Value("")
Tf=reactive.Value("")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of in", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A gymnasium roof of length L = {L()} ft is supported by open-wel

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(50,100,1))
        Ti.set(random.randrange(30,50,1))
        Tf.set(random.randrange(100,150,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        instr=6.5*10**-6*(Tf()-Ti())*L()*12
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):

```

```

        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

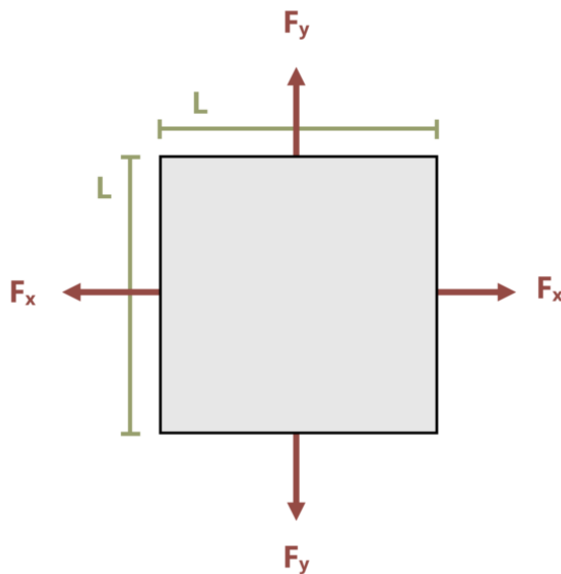
# App installation

```



```
app = App(app_ui, server)
```

Problem 4.23 - Multiaxial Hooke's Law



[Problem adapted from © Kurt Gramoll CC

BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```

problem_ID="204"
L=reactive.Value("__")
t=reactive.Value("__")
Fx=reactive.Value("__")
Fy=reactive.Value("__")
E=29000
v=0.29

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of inches", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A square steel plate of side length L = {L()} in. and thickness t = {t()} in.")]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(50, 150, 1)/10)
        t.set(random.randrange(2, 10, 1)/10)
        Fx.set(random.randrange(100, 500, 1)/10)
        Fy.set(random.randrange(100, 500, 1)/10)

    @reactive.Effect
    @reactive.event(input.submit)

```

```

def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    sigmax = Fx()/(L()*t())
    sigmay = Fy()/(L()*t())
    instr= t()*(-v/E)*(sigmax+sigmay)
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

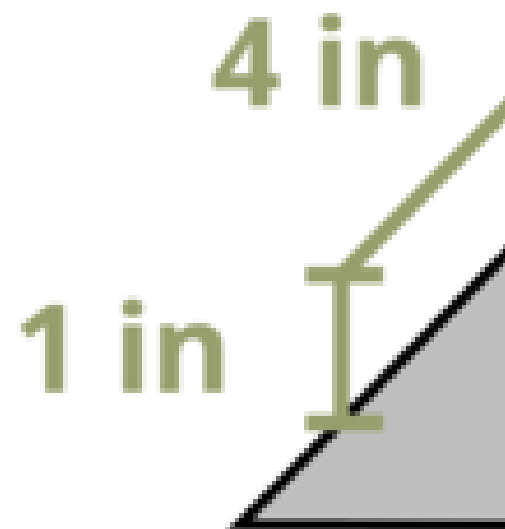
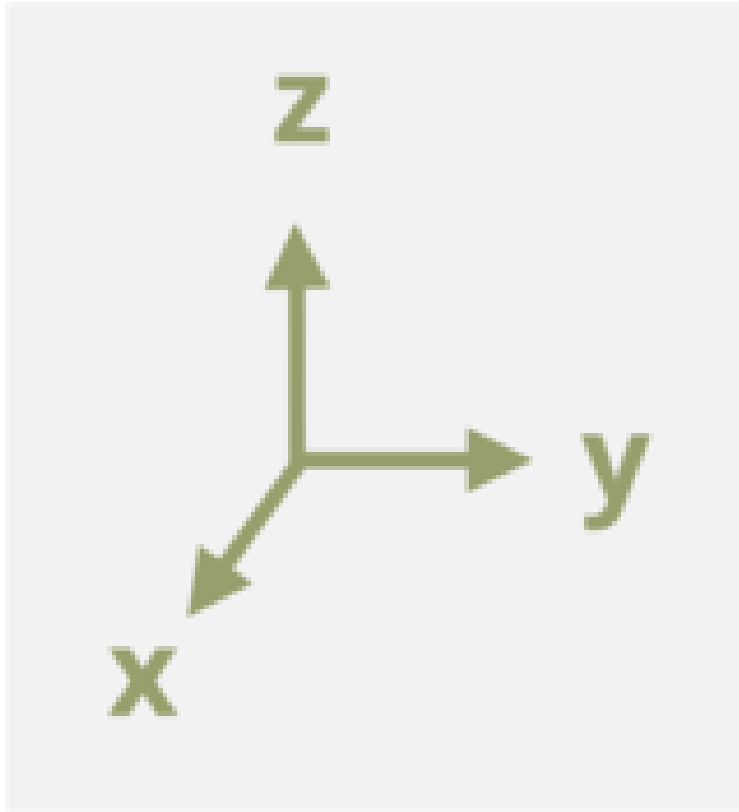
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

```

```
    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)
```


Problem 4.24 - Multiaxial Hooke's Law



```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="205"
P1=reactive.Value("__")
E=reactive.Value("__")
v=reactive.Value("__")
SG=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in percent", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui

```



```

def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A strain gauge is placed on a polymer test sample with an elast.

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    P1.set(random.randrange(20, 200, 1)/10)
    E.set(random.randrange(5, 20, 1))
    v.set(random.randrange(20, 40, 1)/100)
    SG.set(round((v()*P1()*1000)/(12*E()),1)-(random.randrange(3, 20, 1)/10))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    sigmaz = (-P1()*1000)/(4*3)
    Ex = ((-v()*(sigmaz))/(E()*10**6))
    instr= ((Ex - SG()*10**-6)/Ex)*100
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(

```

```

        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

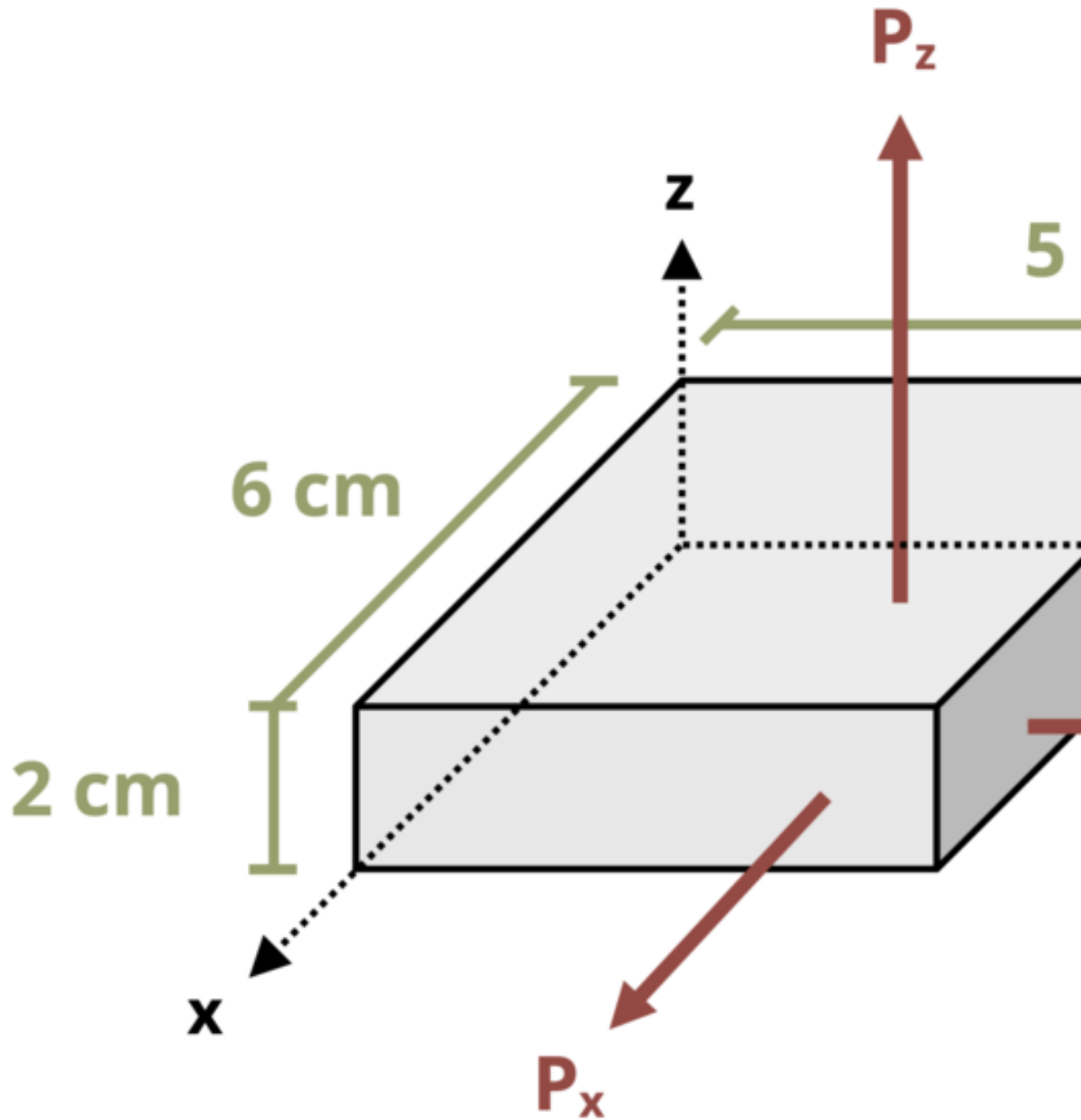
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 4.25 - Multiaxial Hooke's Law



```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="208"
Px=reactive.Value("__")
Py=reactive.Value("__")
Pz=reactive.Value("__")
L=reactive.Value("__")
E=reactive.Value("__")
v=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in percent", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts

```

```

attempt_counter = reactive.Value(0)

@output
@render.ui
def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A block is pulled in all three directions ( $P_x = \{P_x\}$ 

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    Px.set(random.randrange(1, 20, 1))
    Py.set(random.randrange(1, 20, 1))
    Pz.set(random.randrange(1, 20, 1))
    E.set(random.randrange(1000, 2000, 100))
    v.set(random.randrange(20, 40, 1)/100)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    sigmax = (Px()/(5*2))*10
    sigmay = (Py()/(6*2))*10
    sigmaz = (Pz()/(5*6))*10
    Ex = (sigmax - v()*(sigmay+sigmaz))/(E())
    Ey = (sigmay - v()*(sigmaz+sigmax))/(E())
    Ez = (sigmaz - v()*(sigmax+sigmay))/(E())
    Lx = 6 + 6*Ex
    Ly = 5 + 5*Ey
    Lz = 2 + 2*Ez
    instr= (((Lx*Ly*Lz)-60)/60)*100
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)

```

```

random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across su
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

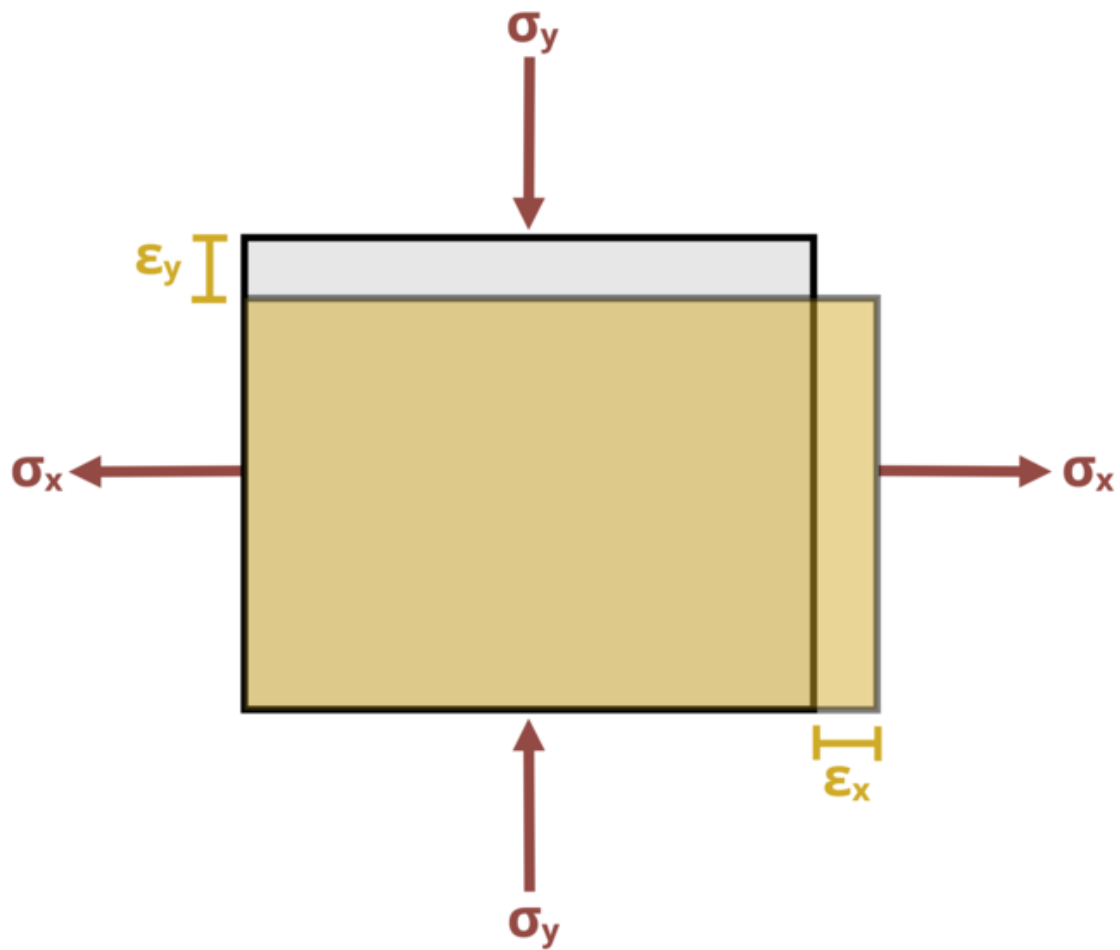
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 4.26 - Multiaxial Hooke's Law



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]
```



```

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="209"
sigma_x=reactive.Value("__")
sigma_y=reactive.Value("__")
e_x=reactive.Value("__")
e_y=reactive.Value("__")
placeholder = reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Pressure loading is applied to a plate in both the x and y directions")]

```

```

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    sigma_x.set(random.randrange(5, 20, 1))
    sigma_y.set(random.randrange(5, 20, 1))
    e_x.set(random.randrange(2100, 2500, 10))
    placeholder.set(random.randrange(15,40,1)/100)
    e_y.set(round(e_x()*(sigma_y()-placeholder()*sigma_x()/(sigma_x()+placeholder()*sigma_x()))/(sigma_x()+placeholder()*sigma_x()))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each submit

    instr= -(sigma_x()/e_x()+sigma_y()/-e_y())/(-sigma_y()/e_x()-sigma_x()/-e_y())
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{random_end}"

    # Store the most recent encoded attempt in a reactive value so it persists across sessions
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )

```

```

        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

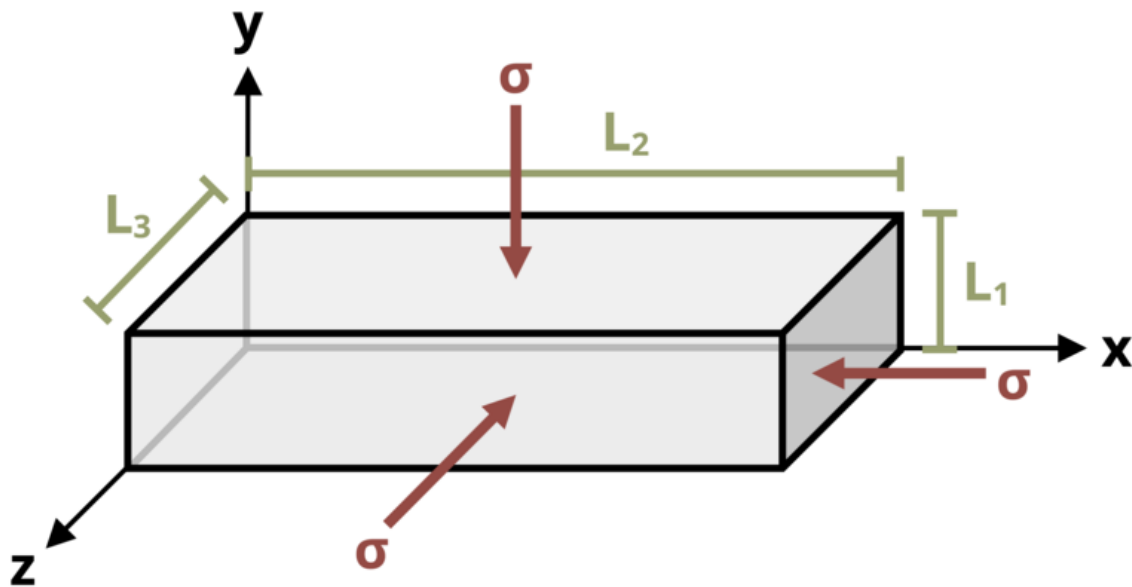
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 4.27 - Multiaxial Hooke's Law



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
```

```

    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="210"
x=reactive.Value("__")
E=reactive.Value("__")
v=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A rigid block is subjected to uniform hydrostatic pressure, , o

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        x.set(random.randrange(15,100,1)/10)
        E.set(random.randrange(75,150,1))
        v.set(random.randrange(20,40,1)/100)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():

```

```

    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

    instr= x()/30*E()/(1-2*v())
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

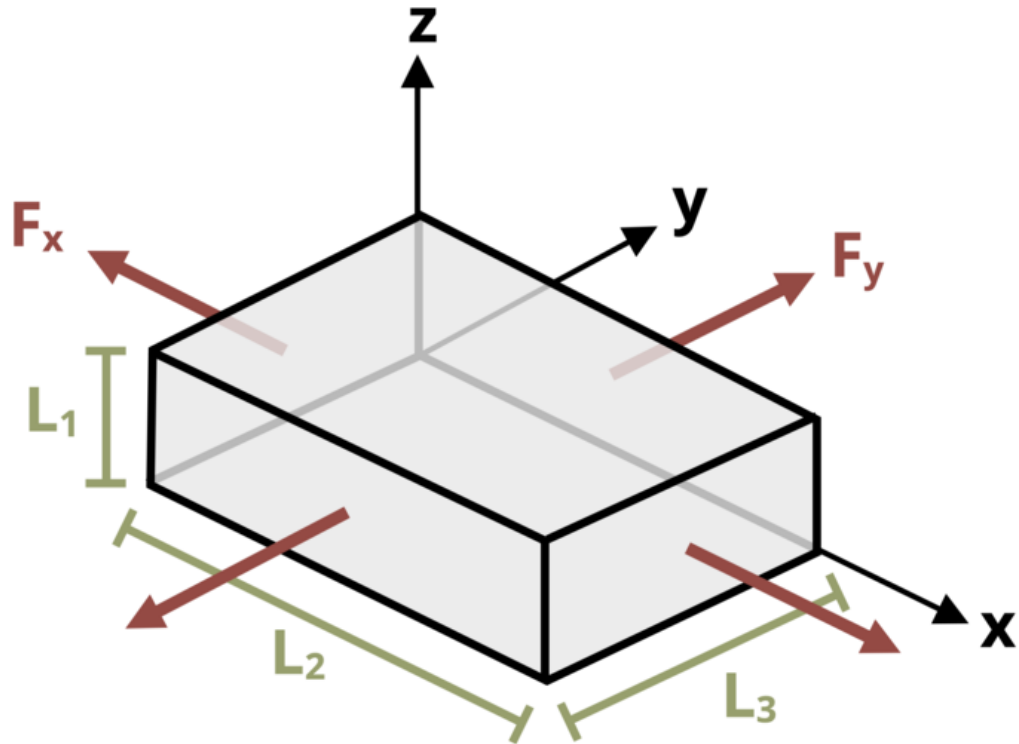
    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header

```

```
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)
```

Problem 4.28 - Multiaxial Hooke's Law



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
```



```

    Fy.set(random.randrange(50,200,1)/10)
    E.set(random.randrange(75,150,1)/10)
    v.set(random.randrange(20,40,1)/100)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    sigmax = Fx()/4
    sigmay=Fy()/5
    strainx = (sigmax-v()*sigmay)/(E()*1000)
    strainy = (sigmay-v()*sigmax)/(E()*1000)
    strainz = (-v()*(sigmax+sigmay))/(E()*1000)
    LxN = 10+10*strainx
    LyN = 8+8*strainy
    LzN = 0.5+0.5*strainz
    VN = LxN*LyN*LzN
    Vo = 10*8*0.5
    instr= VN-Vo
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,

```

```

        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

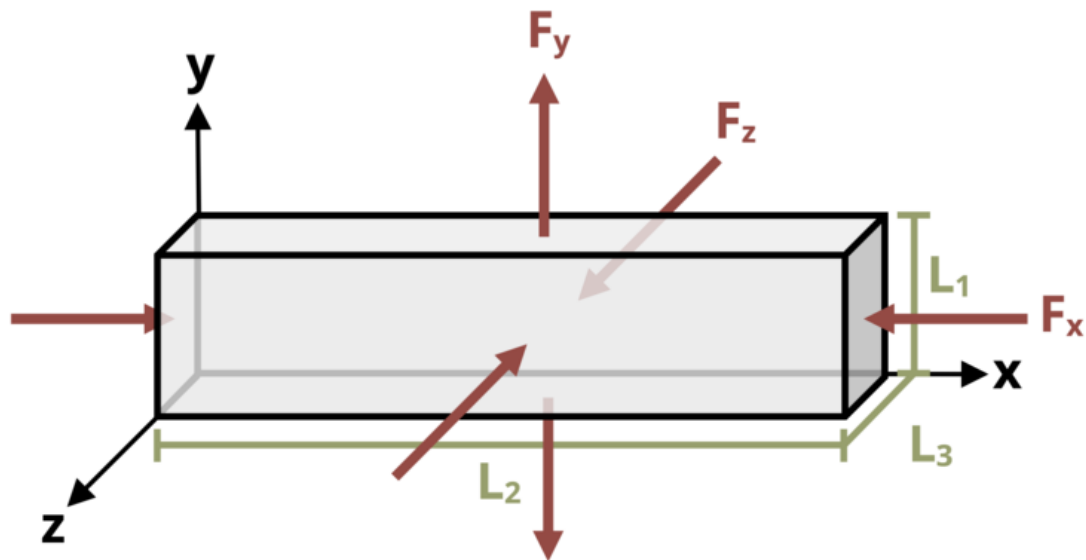
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 4.29 - Multiaxial Hooke's Law



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```



```

    sigmax = Fx()/4
    sigmay=Fy()/5
    strainx = (sigmax-v()*sigmay)/(E()*1000)
    strainy = (sigmay-v()*sigmax)/(E()*1000)
    strainz = (-v()*(sigmax+sigmay))/(E()*1000)
    LxN = 10+10*strainx
    LyN = 8+8*strainy
    LzN = 0.5+0.5*strainz
    VN = LxN*LyN*LzN
    Vo = 10*8*0.5
    instr= VN-Vo
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)

```

```

    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

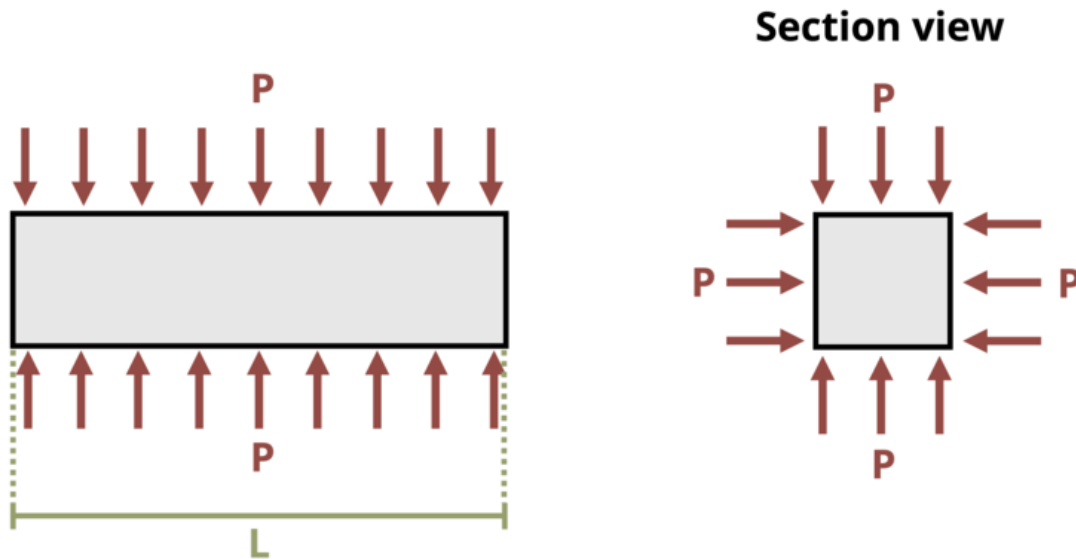
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 4.30 - Multiaxial Hooke's Law



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
```



```

        return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="215"
L=reactive.Value("__")
P=reactive.Value("__")
E=reactive.Value("__")
v=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of mm", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A bar of length L = {L()} mm is subjected to a pressure P = {P()}

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(250,750,10))
        P.set(random.randrange(1,10,1)/10)
        E.set(random.randrange(400,900))
        v.set(random.randrange(30,45,1)/100)

    @reactive.Effect
    @reactive.event(input.submit)

```

```

def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    strainz = (v()*2*P())/E()
    instr= L()*strainz
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

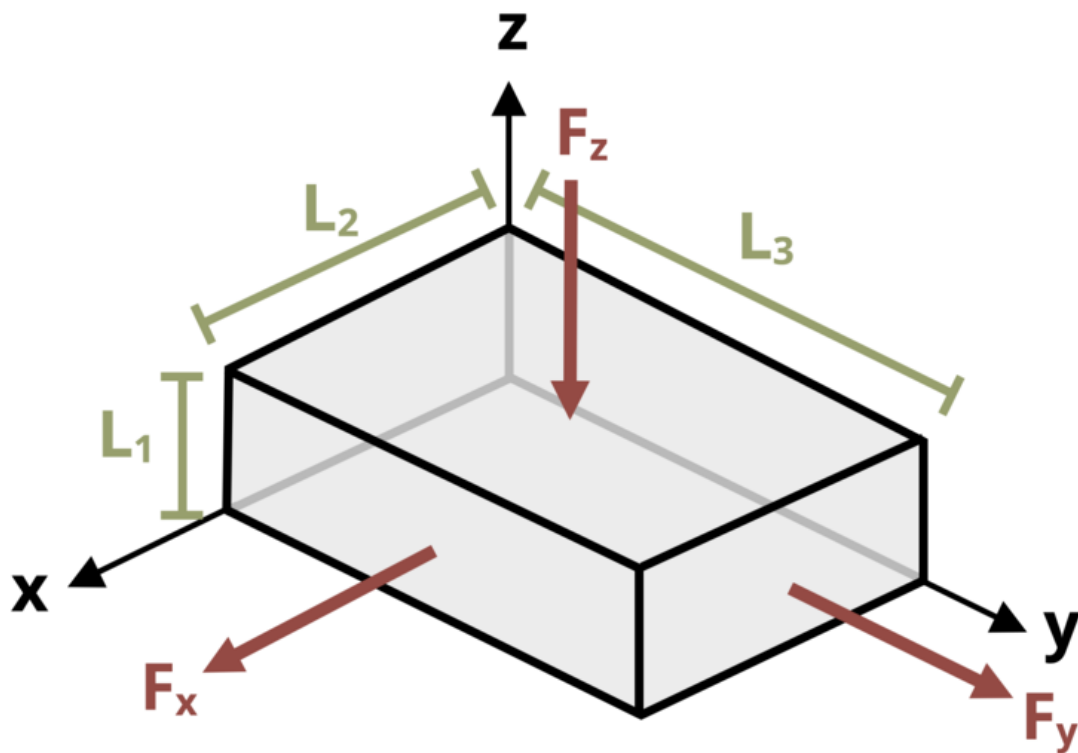
    # Write the attempts data, ensure that the header from the attempts list is not writ

```

```
        for attempt in attempts[1:]: # Skip the first element which is the header
            await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 4.31 - Multiaxial Hooke's Law



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
```

```

import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="217"
Fx=reactive.Value("__")
Fy=reactive.Value("__")
Fz=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of in", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A small brass sample is subjected to loads on each of its three

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        Fx.set(random.randrange(50,200,1)/10)
        Fy.set(random.randrange(50,200,1)/10)

```

```

Fz.set(random.randrange(50,200,1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    sigmax = Fx()/3
    sigmay = Fy()/2
    sigmaz = Fz()/6
    instr = (-sigmaz-0.34*(sigmax+sigmay))/(15*10**3)
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else

```

```

yield f"{final_encoded}\n\n"

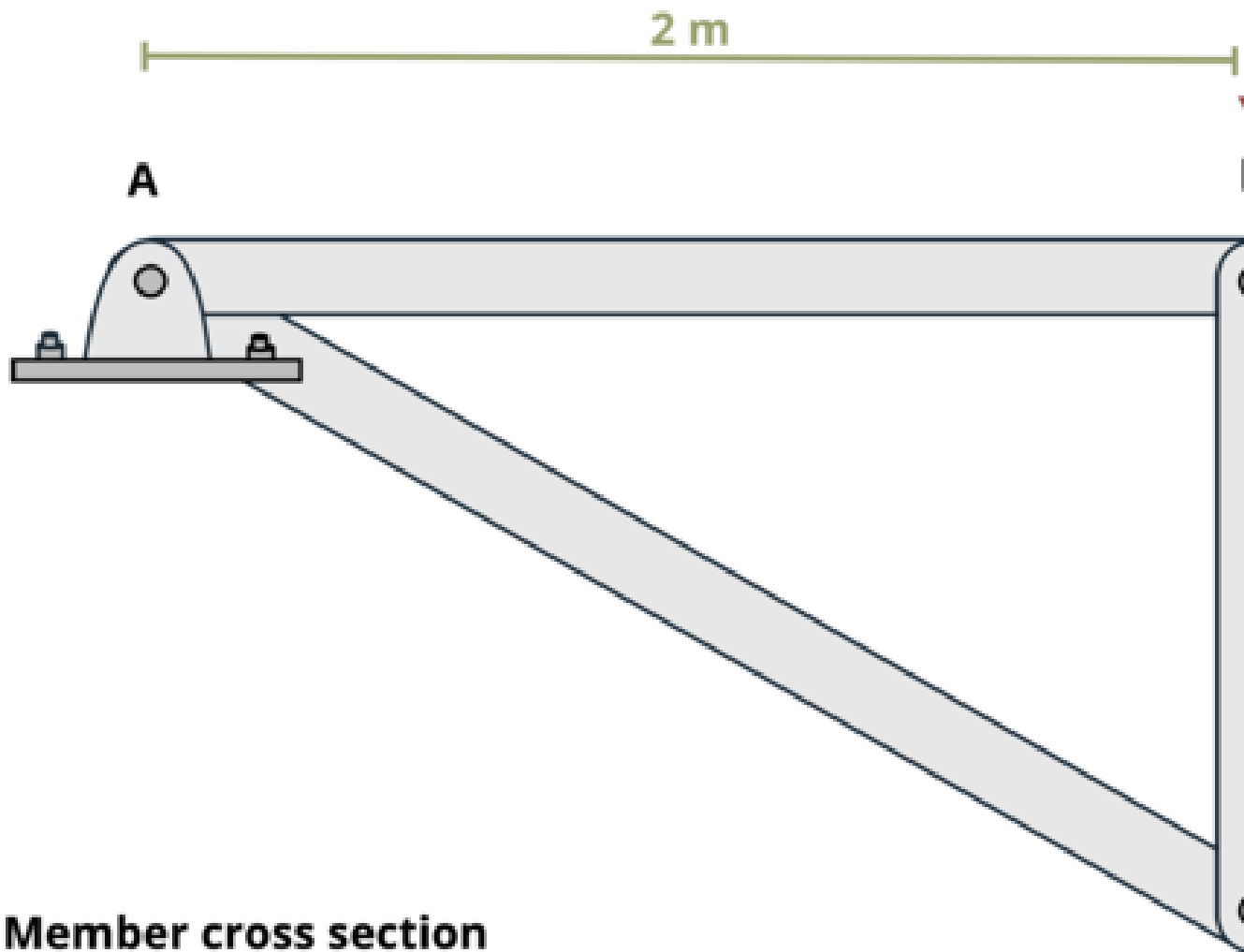
# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

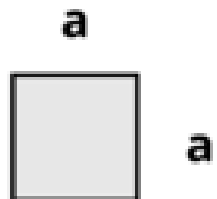
# App installation
app = App(app_ui, server)

```


Problem 4.35 - Allowable Stress/Safety Factor



Member cross section



```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="157"
F=reactive.Value("__")
FS=reactive.Value("__")
fail=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of centimeters", placeholder="Please enter y
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui

```

```

def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A small truss is constructed with solid square wood members and

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    F.set(random.randrange(15, 50, 1))
    FS.set(random.randrange(15, 40, 1)/10)
    fail.set(random.randrange(40, 60, 1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

    dl = FS()*F()
    A = (dl/( fail()*10**3))*100*100
    instr= math.sqrt(A)
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")

```

```

        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Part VI

Chapter 5 Problems

Problem 5.1 - Axial Stress

[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="662"
L=reactive.Value("__")
P=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)
```

```

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A W18 x 50 structural steel column of length L = {L()} ft is su

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(10,25,1))
        P.set(random.randrange(10,100,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        Weight=(50*L())/1000
        LoadTot=P()+Weight
        instr=LoadTot/14.7
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        # Store the most recent encoded attempt in a reactive value so it persists across su
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

```

```

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

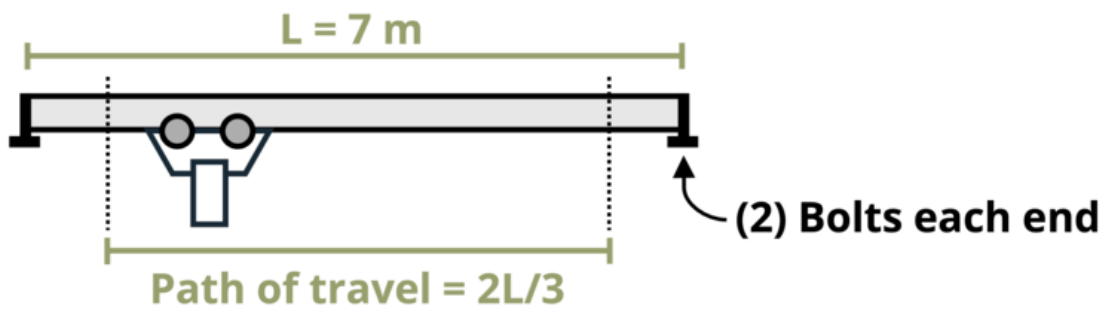
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 5.2 - Axial Stress



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="663"
cap=reactive.Value("__")
L=reactive.Value("__")
sigmay=reactive.Value("__")
FS=reactive.Value("__")
```

```

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of mm", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A chain hoist with a labeled capacity of {cap()} kN tavel's along

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        cap.set(random.randrange(50,150,5))
        L.set(random.randrange(5,20,1))
        sigmay.set(random.randrange(400,700,10))
        FS.set(random.randrange(15,30,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        Fbolt=cap()*5/6/2
        Fdes=Fbolt*FS()
        Amin=Fdes/(sigmay()/1000)
        instr=math.sqrt(4*Amin/math.pi)
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"

```

```

        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

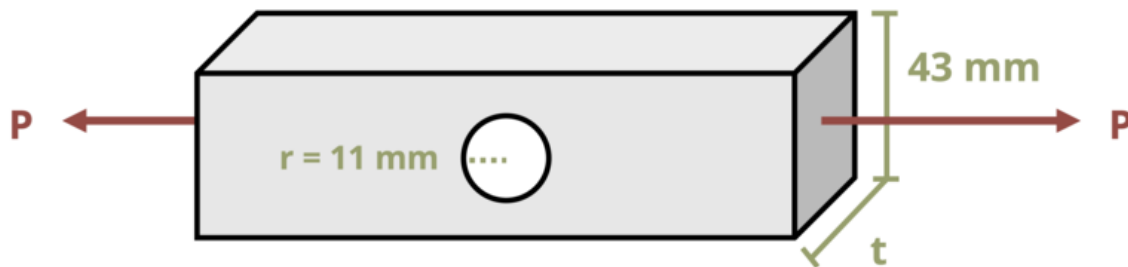
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.6 - Stress Concentrations



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="664"
t=reactive.Value("__")
P=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]
```

```

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A flat bar of thickness t = {t()} mm contains a hole as shown.

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        t.set(random.randrange(10, 25, 1))
        P.set(random.randrange(10, 100, 1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        sigma_norm = P()/(t()*(43-22))*1000
        instr= 2.15*(sigma_norm)
        if math.isclose(float(input.answer()), instr, rel_tol=0.1):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.

```

```

random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

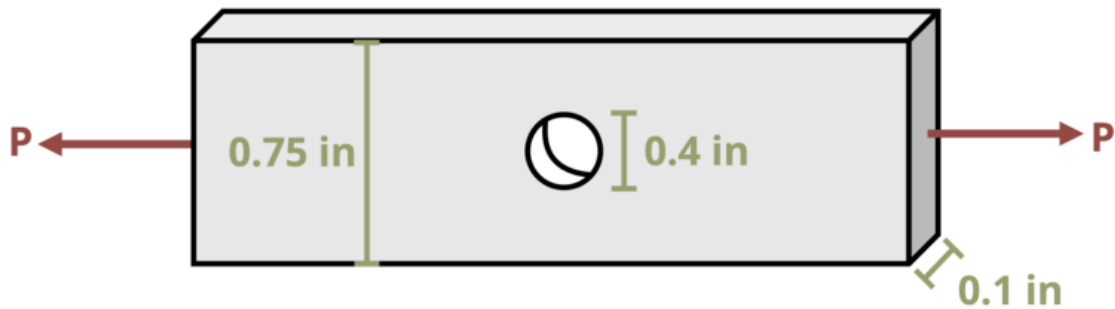
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.7 - Stress Concentrations



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="665"
t=reactive.Value("__")
sigmay=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]
```

```

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of lb", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A flat bar of thickness t = {t()} in. contains a hole as shown.

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        t.set(random.randrange(10,50,1)/100)
        sigmay.set(random.randrange(20,30,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        instr=sigmay()*t()*0.35/2.14*1000
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)

```



```

random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across su
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.8 - Stress Concentrations

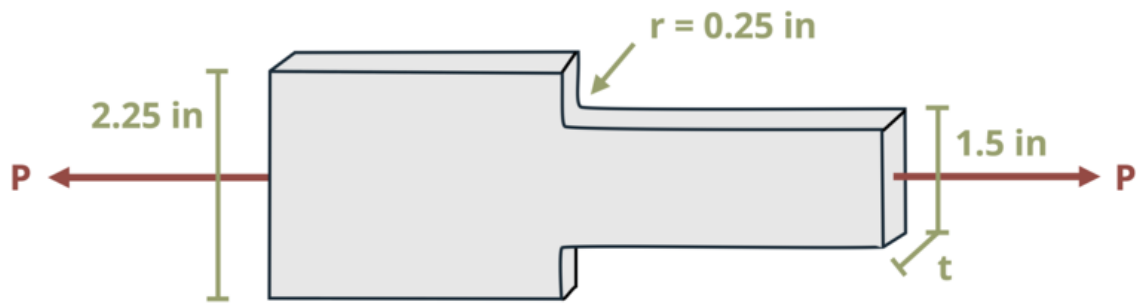


Figure 7: Figure 1: A bar narrows in width.

[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="666"
t=reactive.Value("__")
```



```

        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

    # Store the most recent encoded attempt in a reactive value so it persists across sessions
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

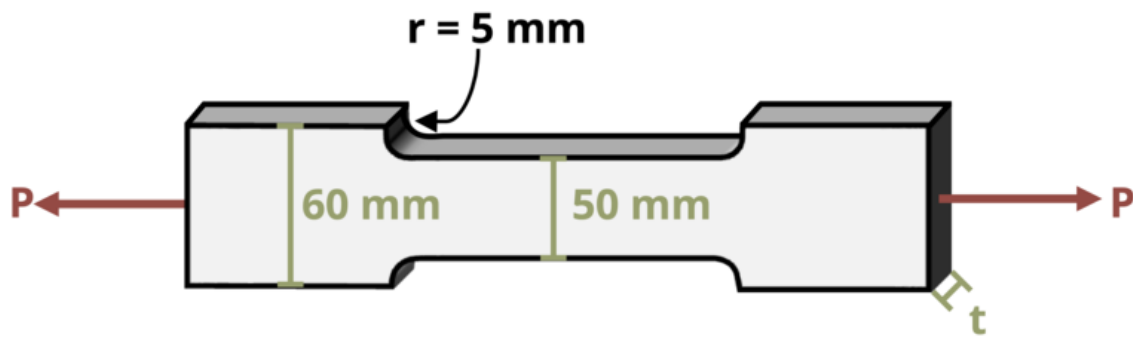
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written again
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.9 - Stress Concentrations



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="667"
t=reactive.Value("__")
P=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]
```

```

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A dogbone specimen of thickness  $t = \{t()\}$  mm is being prepared

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        t.set(random.randrange(70,200,1)/10)
        P.set(random.randrange(20,50,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        A = 0.05*t()
        sigmaavg=P()/A
        instr= sigmaavg*1.59
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

```

```

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.10 - Stress Concentrations

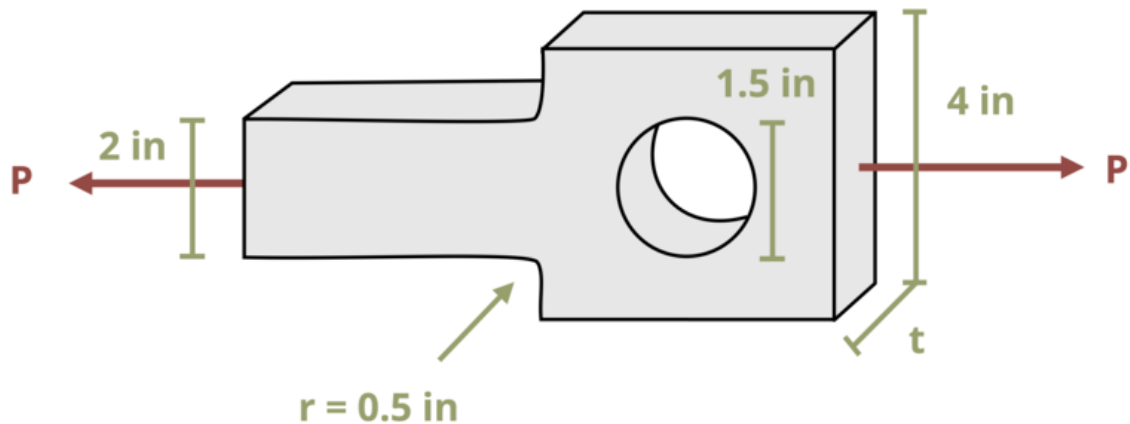


Figure 8: Figure 1: A linkage is subjected to a loading as shown.

[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```



```

problem_ID="668"
t=reactive.Value("__")
P=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"The linkage of thickness  $t = \{t()\}$  in. shown is subjected to loa

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        t.set(random.randrange(25, 75, 5)/100)
        P.set(random.randrange(20,45,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        sigma1 = 1.82*P()/(2*t())
        sigma2 = 2.28*P()/(2.5*t())
        instr = max(sigma1,sigma2)

```

```

if math.isclose(float(input.answer()), instr, rel_tol=0.1):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

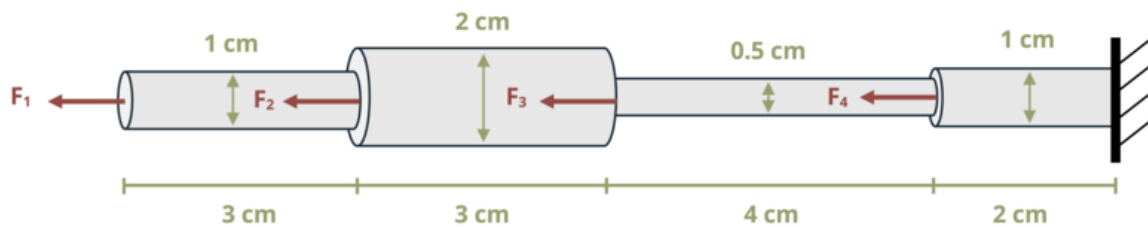
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

```

```
# App installation
app = App(app_ui, server)
```

Problem 5.11 - Axial Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

problem_ID="183"
F1=reactive.Value("__")
F2=reactive.Value("__")
F3=reactive.Value("__")
F4=reactive.Value("__")
E=210

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

attempts=["Timestamp,Attempt,Answer,Feedback\n"]
```

```

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of mm", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A series of solid, steel, circular bars are loaded with forces a

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(10, 100, 1)/10)
        F2.set(random.randrange(10, 100, 1)/10)
        F3.set(random.randrange(10, 100, 1)/10)
        F4.set(random.randrange(10, 100, 1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        instr= ((F1()*3)/(E*math.pi*0.5**2))+((F1()+F2()*3)/(E*math.pi*1**2))+((F1()+F2()+F3(
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

```

```

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

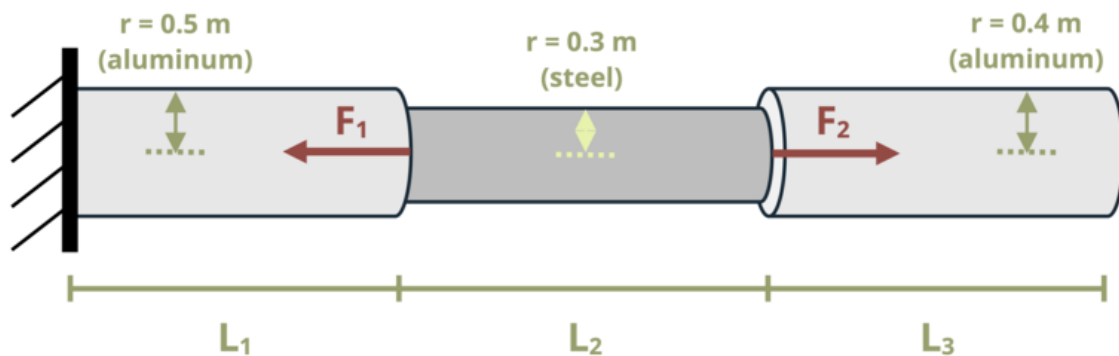
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.12 - Axial Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="184"
F1=reactive.Value("__")
F2=reactive.Value("__")
L1=reactive.Value("__")
```

```

L2=reactive.Value("__")
L3=reactive.Value("__")
Esteel = 210
Ealuminum = 70

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of micrometers", placeholder="Please enter y
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two forces,  $F_1$  = {F1()} kN and  $F_2$  = {F2()} kN")]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(100, 300, 1)/10)
        F2.set(round(F1()/1.5, 2))
        L1.set(random.randrange(20, 80, 1)/10)
        L2.set(round(L1()*0.6, 2))
        L3.set(round(L1()*0.8, 2))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

```



```

instr= ((F2()*10**3*L2())/(math.pi*0.3**2*Esteel*10**9) + ((F2()-F1())*1000*L1()/(m
if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across su
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

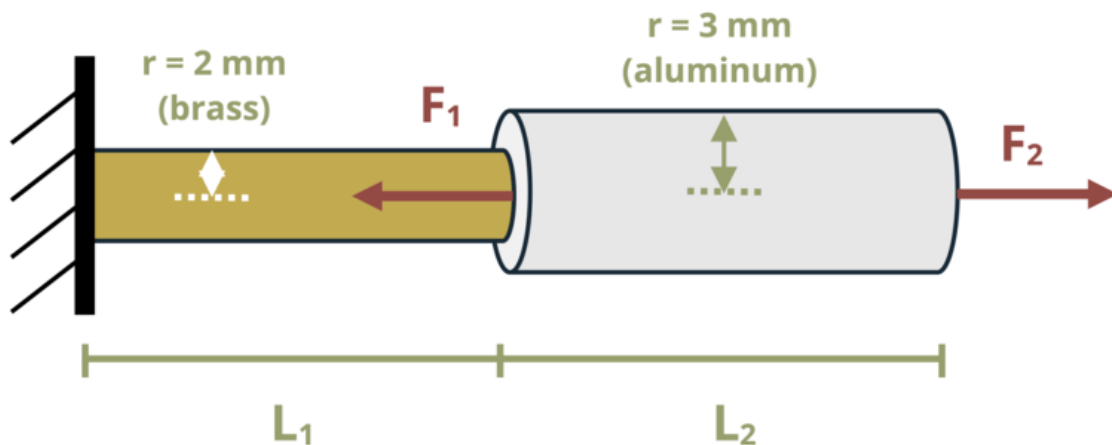
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

```

```
# App installation  
app = App(app_ui, server)
```

Problem 5.13 - Axial Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="185"
F1=reactive.Value("__")
```

```

F2=reactive.Value("__")
L1=reactive.Value("__")
L2=reactive.Value("__")
Ebrass = 100
Ealuminum = 70

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of mm", placeholder="Please enter your answe
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two forces,  $F_1$  = {F1()} kN and  $F_2$  = {F2()}

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(10, 100, 1)/10)
        F2.set(round(F1()*2, 2))
        L1.set(random.randrange(50, 150, 1))
        L2.set(round(L1()*1.5))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

```

```

instr= (((F2()-F1())*L1()/(math.pi*.002**2*Ebrass*10**6)) + (F2()*L2()/(math.pi*.0
if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

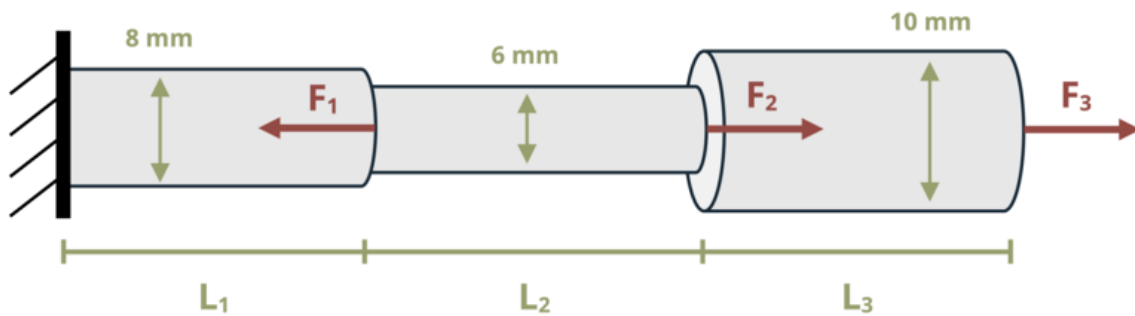
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

```

```
# App installation
app = App(app_ui, server)
```

Problem 5.14 - Axial Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="186"
F1=reactive.Value("__")
F2=reactive.Value("__")
F3=reactive.Value("__")
L1=reactive.Value("__")
L2=reactive.Value("__")
```



```

dAB = R*L1()/100/(210*10**9*0.004**2*math.pi)
dBC = FBC*L2()/100/(210*10**9*0.003**2*math.pi)
dCD = F3()*L3()/100/(210*10**9*0.005**2*math.pi)
instr= abs((dAB+dBC+dCD)*10**3)
if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sessions
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

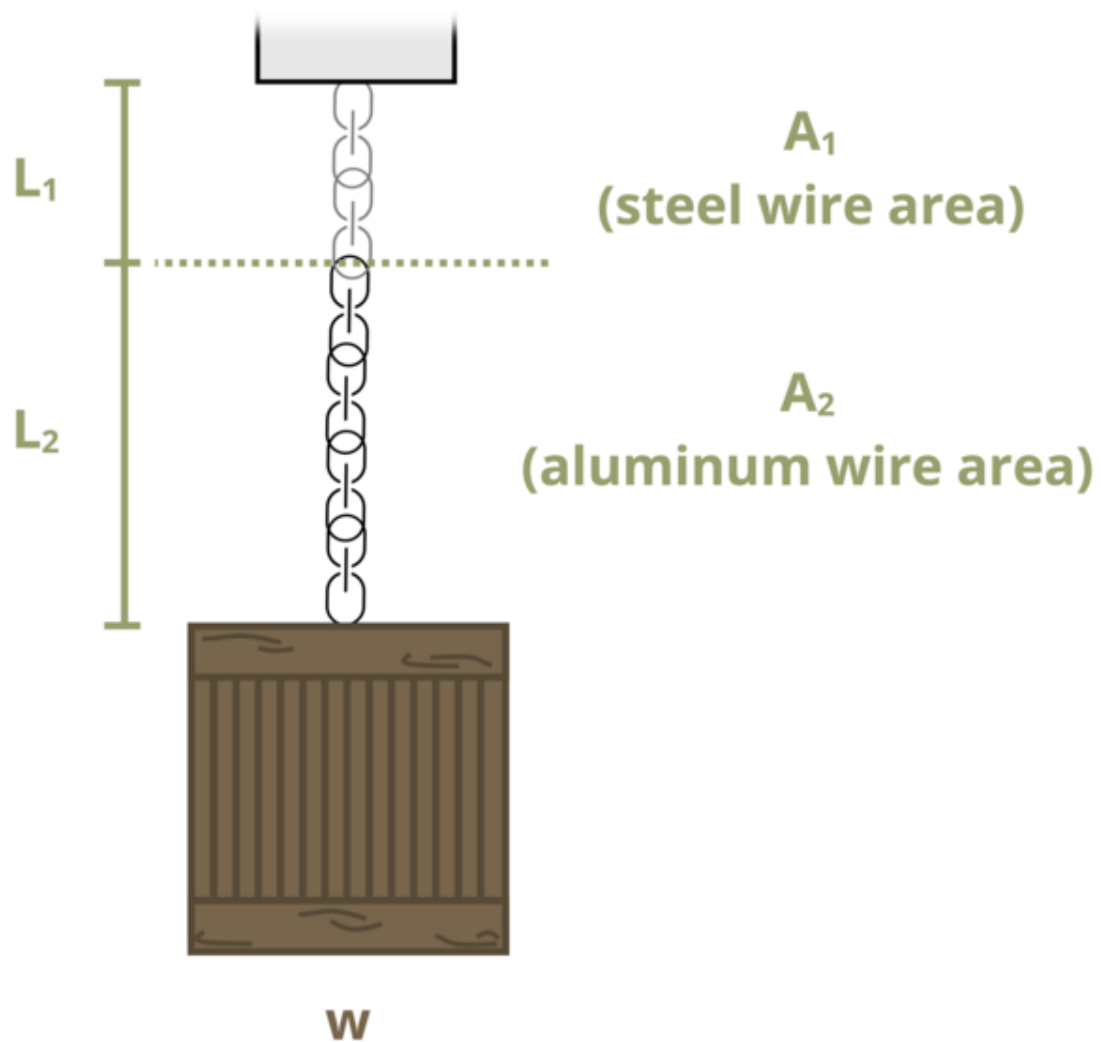
    # Write the attempts data, ensure that the header from the attempts list is not written

```

```
        for attempt in attempts[1:]: # Skip the first element which is the header
            await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 5.15 - Axial Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="187"
W=reactive.Value("__")
L1=reactive.Value("__")
L2=reactive.Value("__")
A1=reactive.Value("__")
A2=reactive.Value("__")
Esteel = 29000
Ealuminum = 10000

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of inches", placeholder="Please enter your a
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

```

```

@output
@render.ui
def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A crate weight W = {W()} lb is attached to a cable constructed

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    W.set(random.randrange(50, 250, 1))
    L1.set(random.randrange(10, 30, 1))
    L2.set(round(L1()*2, 2))
    A1.set(random.randrange(1, 5, 1)/100)
    A2.set(random.randrange(1, 5, 1)/100)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

    instr=(W()*L1()/(A1()*Esteel*1000) + (W()*L2()/(A2()*Ealuminum*1000)))
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

    # Show feedback to the user.

```

```

        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

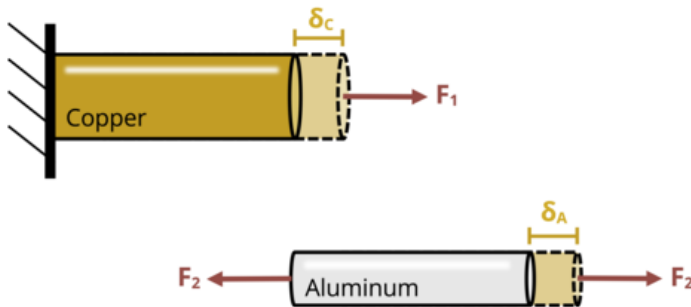
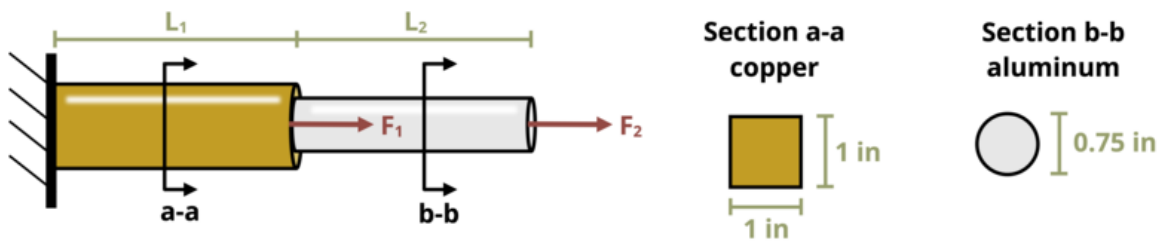
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.16 - Axial Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
```

```

from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
problem_ID="190"
F1=reactive.Value("__")
F2=reactive.Value("__")
L1=reactive.Value("__")
L2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of in", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A solid square rod and round bar are pulled in series as shown.

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(10,30,1))
        F2.set(random.randrange(10,30,1))
        L1.set(random.randrange(6,25,1))

```



```

L2.set(round(L1()*1.4,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    Pc = (F1()+F2())*1000
    Ec = 16*10**6
    Ac = 1
    Pa = F2()*1000
    Ea = 10*10**6
    Aa = 0.375**2*math.pi
    instr = Pc*L1()/(Ec*Ac)+Pa*L2()/(Ea*Aa)

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

```

```

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

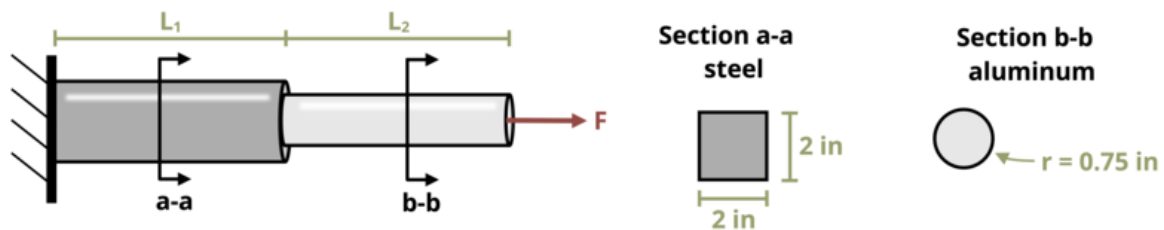
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written again
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.17 - Axial Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="190"
F1=reactive.Value("__")
F2=reactive.Value("__")
L1=reactive.Value("__")
L2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]
```

```

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of in", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A solid square rod and round bar are pulled in series as shown.

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(150,400,10))
        L1.set(random.randrange(6,25,1))
        L2.set(round(L1()*1.4,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        Ps = F1()*1000
        Es = 29*10**6
        As = 4
        Pa = F1()*1000
        Ea = 10*10**6
        Aa = 0.75**2*math.pi
        instr = Ps*L1()/(Es*As)+Pa*L2()/(Ea*Aa)

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):

```

```

        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

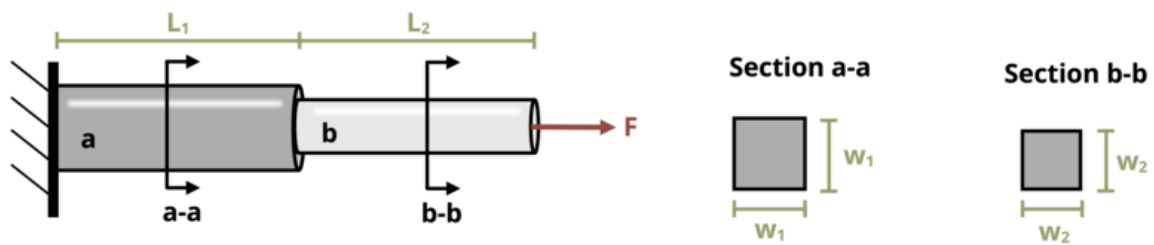
    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation

```

```
app = App(app_ui, server)
```

Problem 5.18 - Axial Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="197"
w2=reactive.Value("__")
w1=reactive.Value("__")
F=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
```

```

    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in terms of 10 ", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two solid square cross-sections with dimensions  $w_1$  and  $w_2$  are connected by a bolt of thickness  $t$ . The bolt is subjected to a tensile force  $F$ . Determine the average normal stress in the bolt. Assume  $w_1 = 10$  mm,  $w_2 = 20$  mm,  $t = 5$  mm, and  $F = 1000$  N. Express your answer in MPa to three significant figures."
)]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        w2.set(random.randrange(25,80,1))
        w1.set(round(w2()*1.25))
        F.set(random.randrange(200,800,10))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each submission
        delb = F()*0.6/(200*10**6*(w2()/1000)**2)
        instr = delb/0.6*10**6

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

```



```

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across su
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

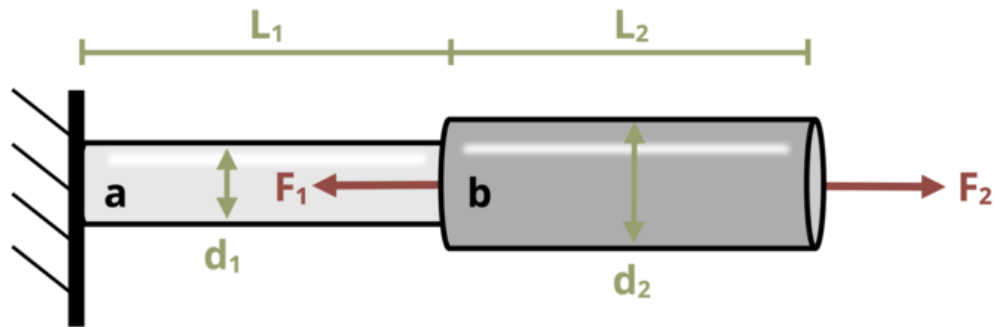
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.19 - Axial Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="198"
F1=reactive.Value("__")
F2=reactive.Value("__")
L2=reactive.Value("__")
L1=reactive.Value("__")
```



```

if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

# Store the most recent encoded attempt in a reactive value so it persists across sessions
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

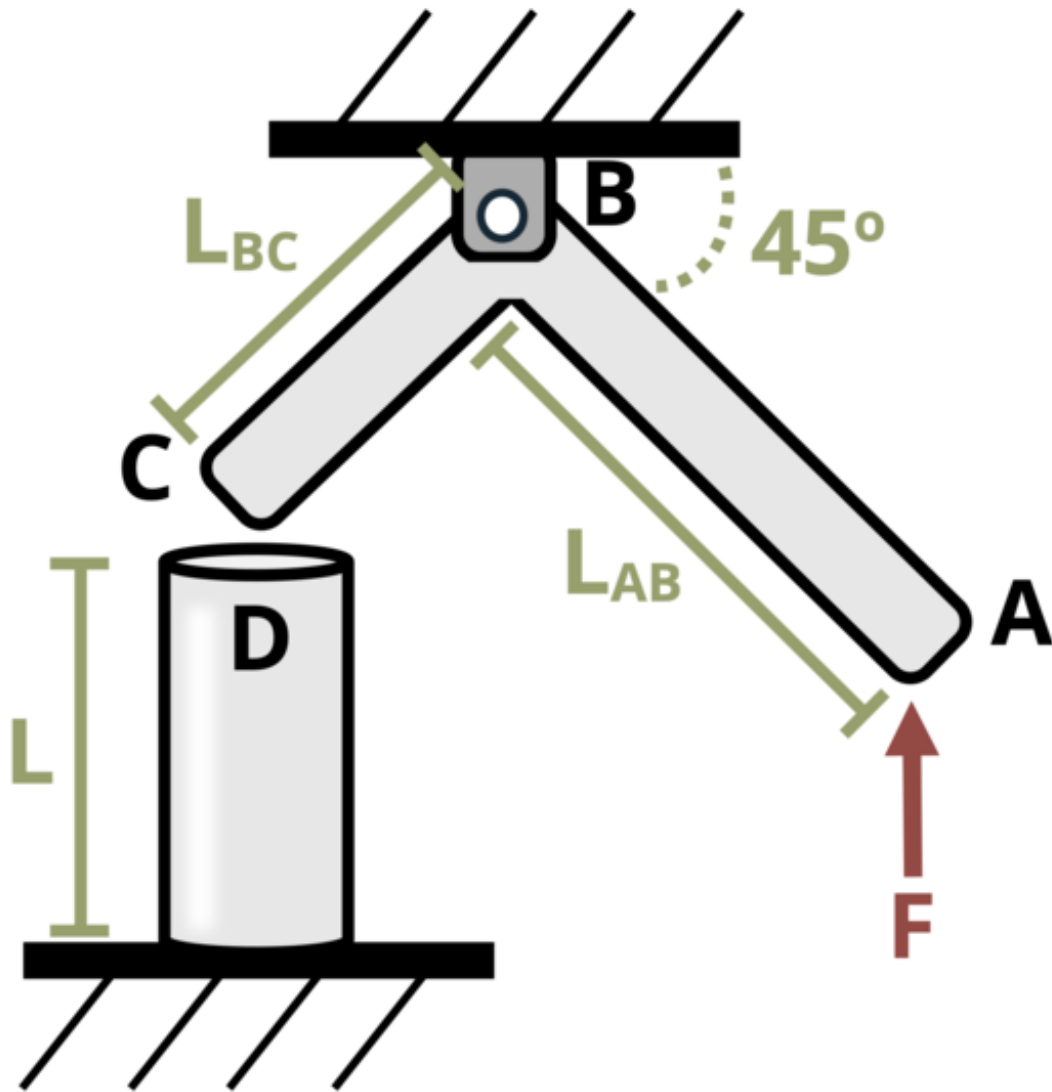
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

```

```
# App installation  
app = App(app_ui, server)
```

Problem 5.20 - Axial Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="199"
F=reactive.Value("__")
LBC=reactive.Value("__")
LAB=reactive.Value("__")
r=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of in", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui

```

```

def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"Force  $F = \{F()\}$  kips on lever ABC causes a force at D to compress

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    F.set(random.randrange(100,400,10))
    LBC.set(random.randrange(5,20,1))
    LAB.set(LBC()*1.7)
    r.set(random.randrange(5,15,1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each submit
    Fc = F()*LAB()*math.cos(45*math.pi/180)/(LBC()*math.cos(45*math.pi/180))
    E = 10000
    instr = Fc*6/(r()*2*math.pi*E)

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

    # Store the most recent encoded attempt in a reactive value so it persists across sessions
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")

```



```

        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

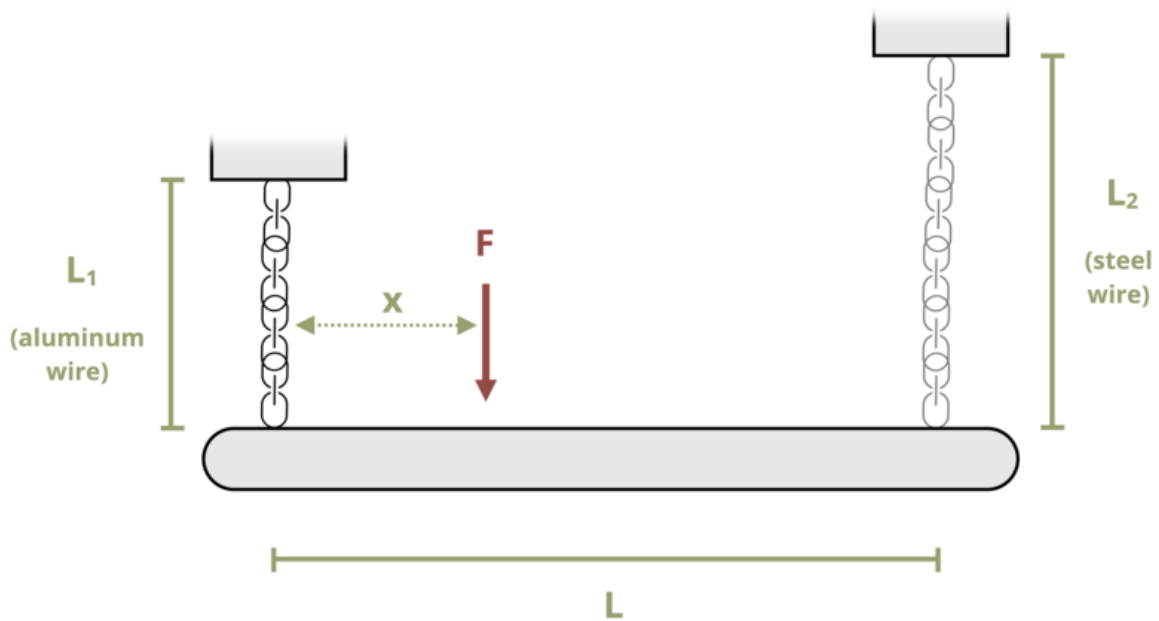
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.27 - Deformation in Systems of Bars



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
```

```

    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="191"
L1=reactive.Value("__")
L2=reactive.Value("__")
F=reactive.Value("__")
A=reactive.Value("__")
L=reactive.Value("__")
Esteel = 29000
Ealuminum = 10000

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of inches", placeholder="Please enter your a
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A bar is attached to two wires, one steel and one aluminum. If t

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L1.set(random.randrange(50, 150, 1)/10)

```

```

L2.set(round(L1()*2, 2))
F.set(random.randrange(30, 150, 1)/10)
A.set(random.randrange(2, 25, 1)/100)
L.set(random.randrange(10, 20, 1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    PsPa = (L1()*Esteel*A())/(L2()*Ealuminum*A())
    Ps = (F()/(PsPa+1))*PsPa
    instr=(Ps*L())/F()
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():

```

```

# Start the CSV with the encoded attempt (without label)
final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
yield f"{final_encoded}\n\n"

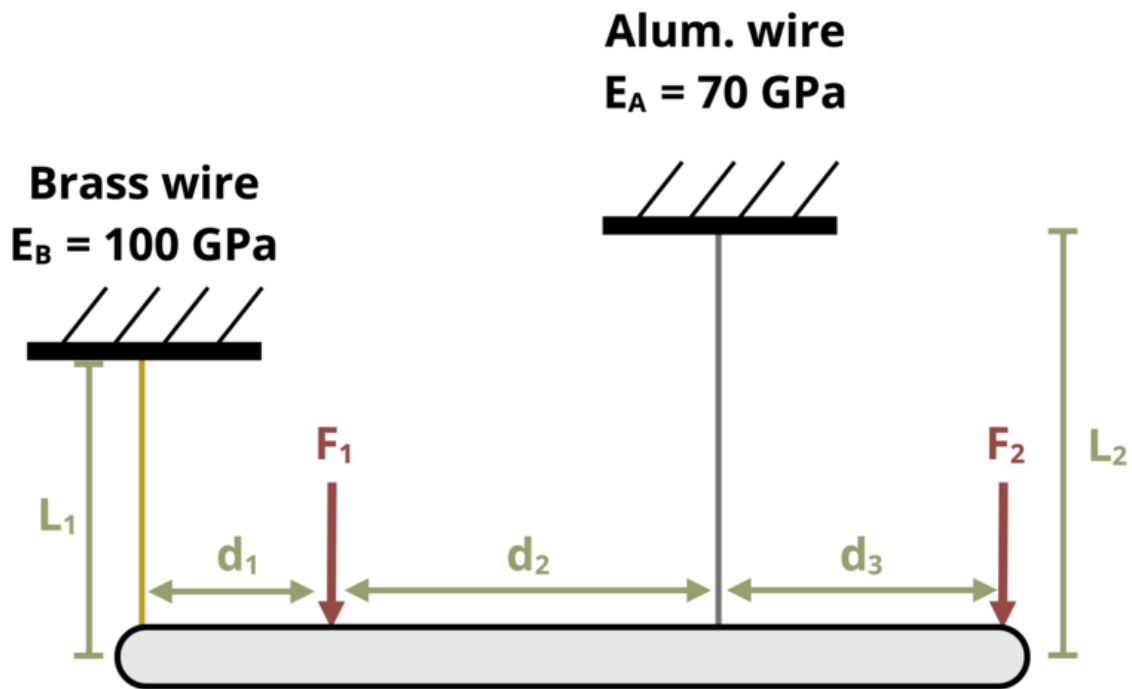
# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.28 - Deformation in Systems of Bars



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```



```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    FA = (2*F1()+10*F2())/7
    FB = F1()+F2()-FA
    EB = 100*10**9
    EA = 70*10**9
    delB = FB*1000*0.04/((db()/2000)**2*math.pi*EB)
    delA = FA*1000*0.06/((da()/2000)**2*math.pi*EA)
    instr = math.atan((delA-delB)/0.07)*180/math.pi

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():

```



```

# Start the CSV with the encoded attempt (without label)
final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
yield f"{final_encoded}\n\n"

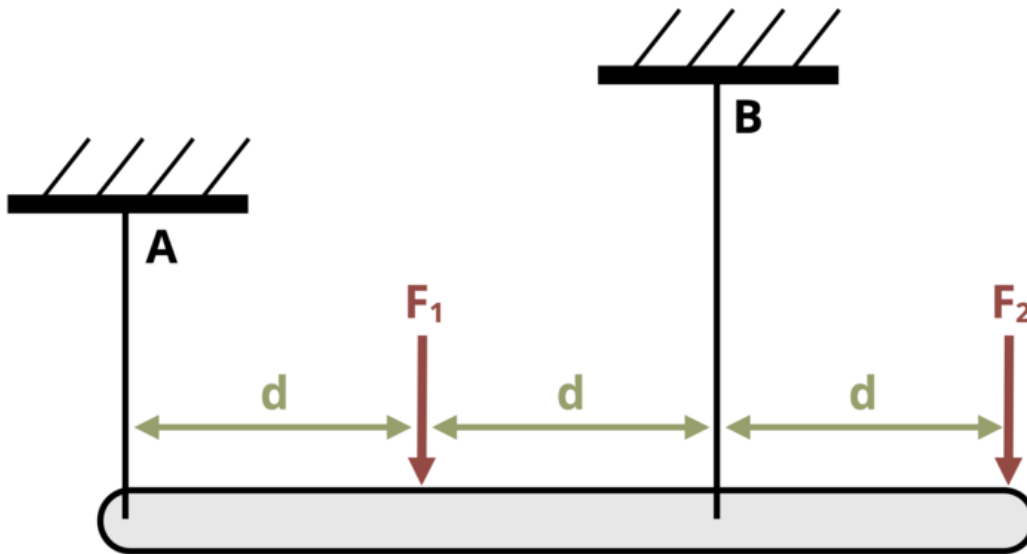
# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.33 - Deformation in Systems of Bars



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
```

```

        return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
problem_ID="189"
F1=reactive.Value("__")
F2=reactive.Value("__")
AA=reactive.Value("__")
AB=reactive.Value("__")
EA=reactive.Value("__")
EB=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of degrees", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two loads,  $F_1 = \{F1()\}$  lb and  $F_2 = \{F2()\}$  lb")]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(200,800,10))
        F2.set(random.randrange(200,800,10))
        AA.set(random.randrange(5,10,1)/100)
        AB.set(random.randrange(2,5,1)/100)
        EA.set(random.randrange(15,25,1))
        EB.set(random.randrange(5,14,1))

```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    FB = (F1()*8+F2()*24)/16
    FA = F1()+F2()-FB
    delB = FB*12/(EB()*10**6*AB())
    delA = FA*6/(EA()*10**6*AA())
    delta = delB-delA
    instr = math.atan(delta/16)*180/math.pi

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)

```

```

    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

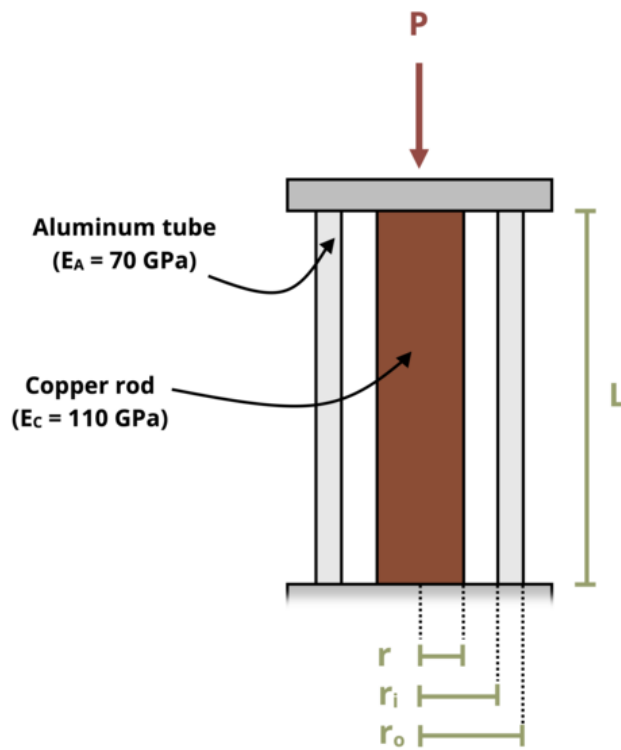
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.34 - Statically Indeterminate Axial Loads



[Problem adapted from © Kurt

Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
```

```

import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="192"
r=reactive.Value("__")
ri=reactive.Value("__")
ro=reactive.Value("__")
L=reactive.Value("__")
dL=reactive.Value("__")
Ecopper = 110
Ealuminum = 70

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kN", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A copper circular rod of radius  $r = \{r()\}$  cm is inserted into a

    #@reactive.Effect

```

```

#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    r.set(random.randrange(20, 60, 1)/10)
    ri.set(round(r()*1.25, 2))
    ro.set(round(r()*1.5, 2))
    L.set(random.randrange(150, 300, 1)/10)
    dL.set(random.randrange(10, 50, 1)/100)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    instr=((dL()*Ecopper*r())**2*math.pi)/L() + ((dL()*Ealuminum*(ro()-ri())**2)*math
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

```



```

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

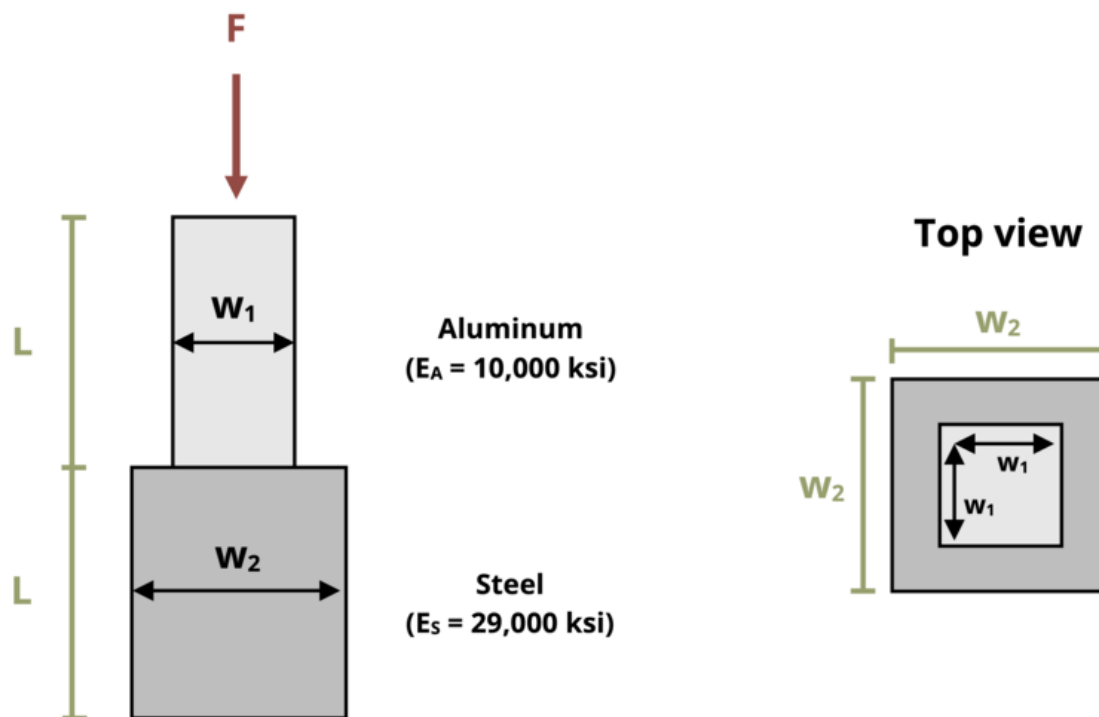
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.35 - Statically Indeterminate Axial Loads



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
```

```

import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="193"
F=reactive.Value("__")
w1=reactive.Value("__")
w2=reactive.Value("__")
L=reactive.Value("__")
Esteel = 29000
Ealuminum = 10000

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of inches", placeholder="Please enter your a
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two blocks with square cross-sections are stacked as shown, with

```

```

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    F.set(random.randrange(20, 100, 1)/10)
    w1.set(random.randrange(15, 50, 1)/10)
    w2.set(round(w1()*1.5))
    L.set(random.randrange(50, 200, 1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    instr=F()*L()*((1/((w1())**2)*Ealuminum)) + (1/((w2())**2)*Esteel)))
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

```

```

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

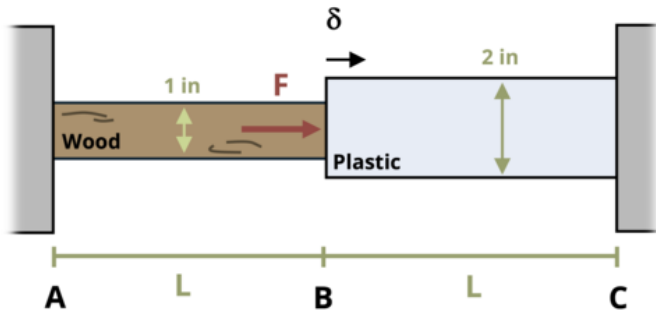
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

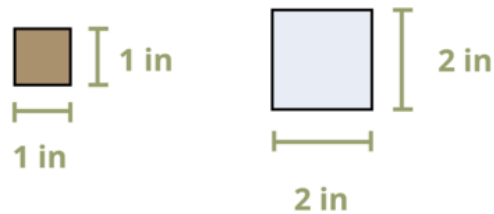
# App installation
app = App(app_ui, server)

```

Problem 5.36 - Statically Indeterminate Axial Loads



Cross sections



[Problem adapted from © Kurt

Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
```

```

import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="246"
d=reactive.Value("__")
L=reactive.Value("__")
Eplastic=reactive.Value("__")
Ewood=1750

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of lb", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two square members are attached to two fixed walls as shown. For

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():

```

```

    random.seed(101)
    d.set(random.randrange(2, 8, 1)/1000)
    L.set(random.randrange(4, 15, 1))
    Eplastic.set(random.randrange(350, 900, 10))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    instr= (d()*1*Ewood*1000)/L() - (-d()*4*Eplastic()*1000)/L()
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else

```



```

yield f"{final_encoded}\n\n"

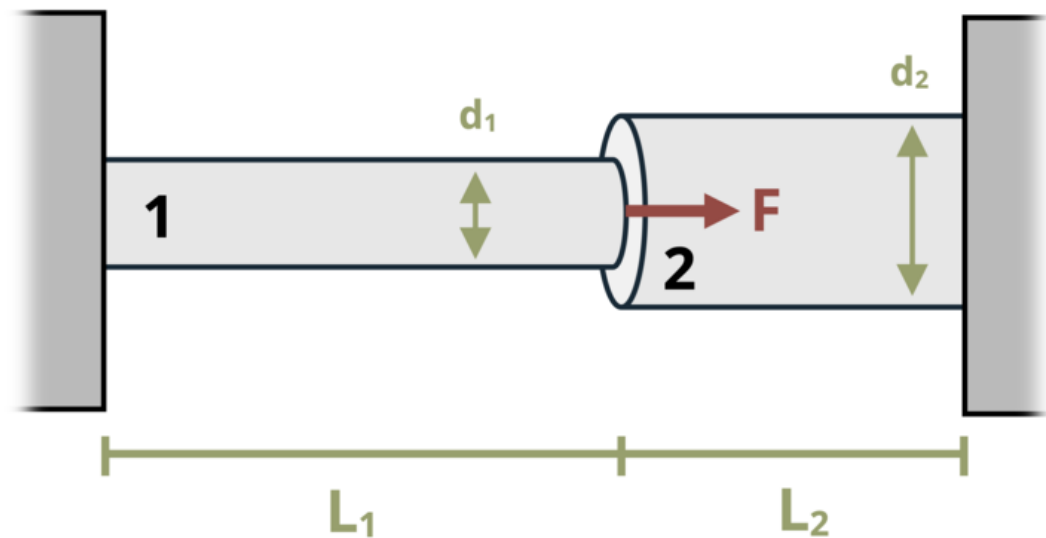
# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.37 - Statically Indeterminate Axial Loads



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```

```

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="247"
F=reactive.Value("__")
d1=reactive.Value("__")
d2=reactive.Value("__")
L1=reactive.Value("__")
L2=reactive.Value("__")
Ewood=70

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two aluminum circular rods are attached to two fixed walls as sl

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(20, 100, 1))
        d1.set(random.randrange(15, 50, 1))

```

```

d2.set(round(d1()*1.5))
L1.set(random.randrange(200, 800, 10))
L2.set(round(L1()*(2/3)))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    LHS = (L1()/1000)/(math.pi*(d1()/2000)**2) + (L2()/1000)/(math.pi*(d2()/2000)**2)
    RHS = (F()*L2())/(math.pi*(d2()/2)**2)
    instr= (RHS/LHS)/(math.pi*(d1()/2000)**2)
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():

```

```

# Start the CSV with the encoded attempt (without label)
final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
yield f"{final_encoded}\n\n"

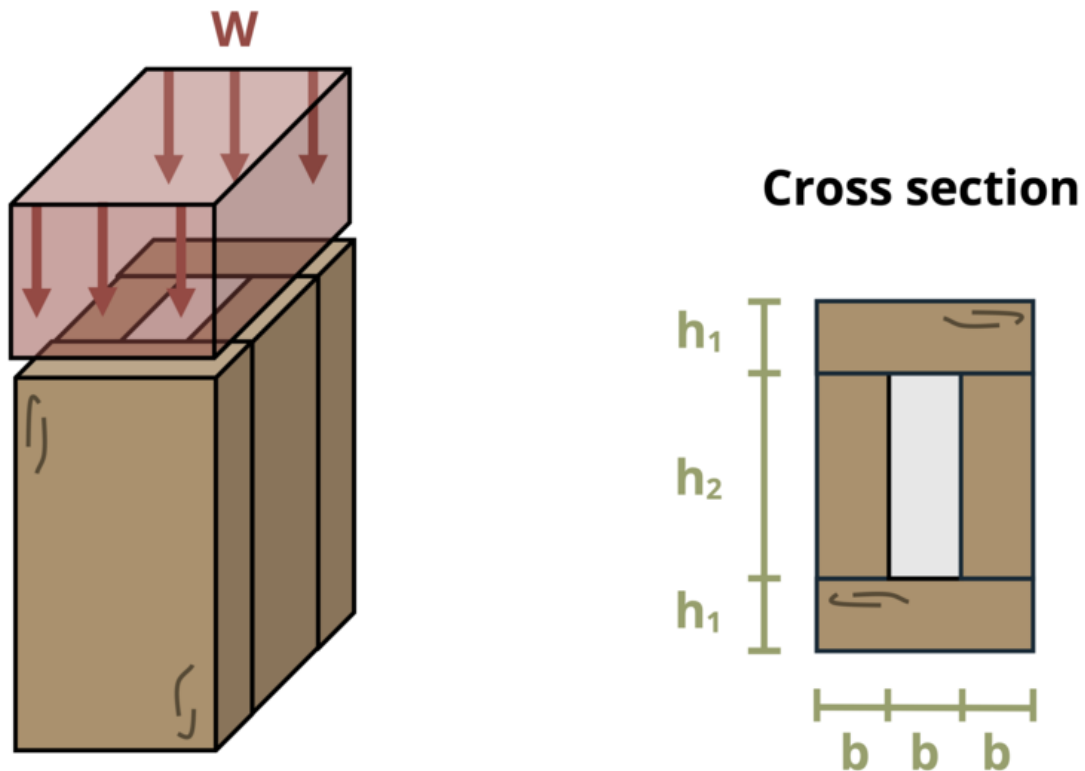
# Write the header for the remaining CSV data once
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.38 - Statically Indeterminate Axial Loads



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
```

```

import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="248"
w=reactive.Value("__")
b=reactive.Value("__")
h1=reactive.Value("__")
h2=reactive.Value("__")
Econcrete=25
Ewood=12

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kN", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A distributed load  $w = \{w()\}$  N/cm2 is applied to a sl

```

```

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    w.set(random.randrange(50, 750, 10))
    b.set(random.randrange(20, 100, 1)/10)
    h1.set(b()*1)
    h2.set(b()*2)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    instr= (2.0833*h2()*w()*(2*h1()+h2())*3*b())/(6*h1()+4.0833*h2())/1000
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

```



```

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

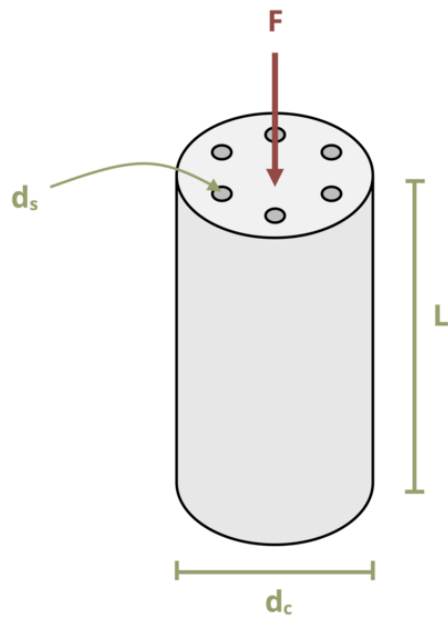
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.39 - Statically Indeterminate Axial Loads



[Problem adapted from © Kurt Gramoll CC BY NC-SA]

4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```

```

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="250"
F=reactive.Value("__")
L=reactive.Value("__")
dc=reactive.Value("__")
ds=reactive.Value("__")
Econcrete=25
Esteel=200

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A concrete post of length L = {L()} m and diameter d<sub>c</sub>")]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(10, 50, 1)/10)
        dc.set(random.randrange(100, 500, 10))

```

```

        ds.set(round(dc()/12))
        F.set(random.randrange(100, 500, 10))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    As = (math.pi*6*(ds()/20)**2)
    Ac = (math.pi*(dc()/20)**2) - As
    Cside = Ac*Econcrete
    Sside = As*Esteel
    LHS = F()*Cside
    RHS = Cside+Sside
    instr= ((LHS/RHS)/(Ac/100**2))/10**3
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

```

```

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

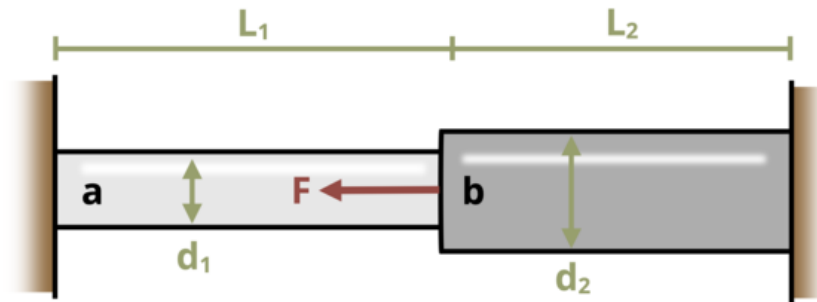
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.40 - Statically Indeterminate Axial Loads



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="252"
L2=reactive.Value("__")
L1=reactive.Value("__")
```

```

d1=reactive.Value("__")
d2=reactive.Value("__")
F=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two circular rods are attached together and fixed between two w

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L2.set(random.randrange(20,80,5)/100)
        L1.set(round(L2()+(random.randrange(5,20,5)/100))
        d1.set(random.randrange(20,50,1))
        d2.set(d1()+random.randrange(5,15,1))
        F.set(random.randrange(10,100,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        Fa = F()*L2()/((d2()/2000)**2*math.pi)/(L1()/((d1()/2000)**2*math.pi)+L2()/((d2()/2000)**2*math.pi))

```

```

Fb = F()-Fa
sigmaA = Fa/((d1()/2000)**2*math.pi)/1000
sigmaB = Fb/((d2()/2000)**2*math.pi)/1000
instr = max(sigmaA,sigmaB)
if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ

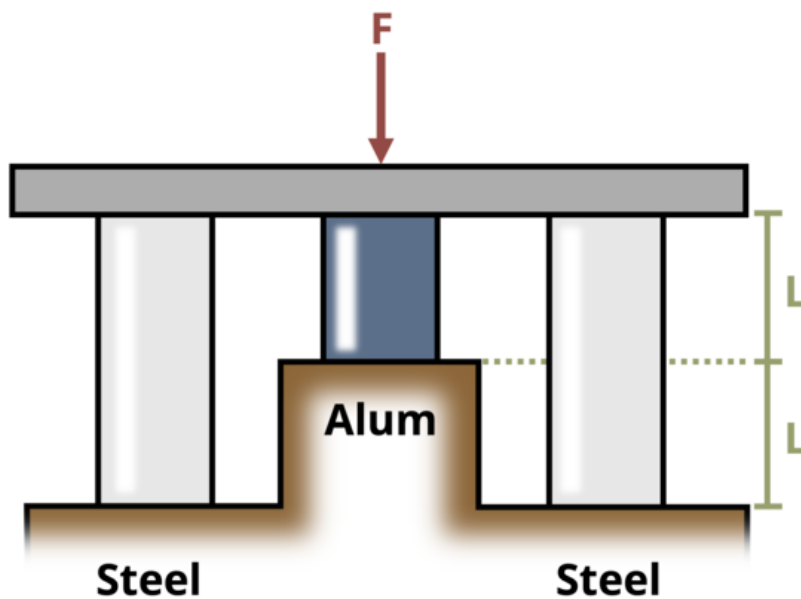
```



```
        for attempt in attempts[1:]: # Skip the first element which is the header
            await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 5.41 - Statically Indeterminate Axial Loads



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
```

```

from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="254"
F=reactive.Value("__")
L=reactive.Value("__")
d=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Three cylinders support a load  $F = \{F()\}$  kN. If length  $L = \{L()\}$ 

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(100,500,10))
        L.set(random.randrange(5,50,1)/10)
        d.set(random.randrange(25,100,1))

```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    FB = F()/((2*L()*200)/(2*L()*70)+1)
    instr = FB/(math.pi*(d()/2000)**2)
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across su
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

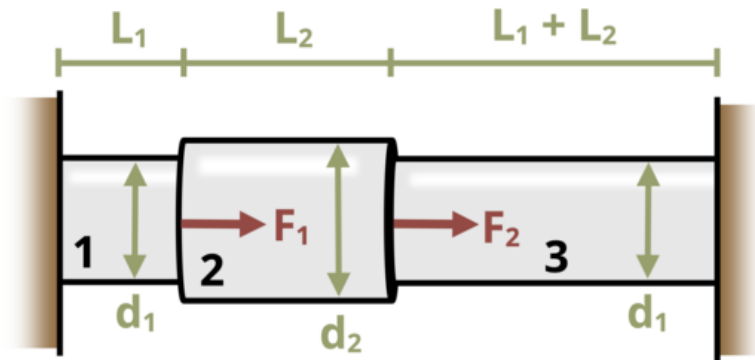
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

```

```
        # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)
```

Problem 5.42 - Statically Indeterminate Axial Loads



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="255"
```

```

F1=reactive.Value("__")
F2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Three circular steel rods (E = 29,000 ksi) are placed between two rigid plates. The rods are of different diameters and lengths. The top plate is fixed to the ceiling and the bottom plate is fixed to the floor. A downward force P is applied to the bottom plate. Determine the reaction forces at the top and bottom plates and the deflection of the bottom plate."
)]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(10,50,1))
        F2.set(random.randrange(10,50,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each submission
        A1 = math.pi*0.5**2
        A2 = math.pi*0.75**2
        A3 = A1
        L1 = 1
        L2 = 2

```

```

L3 = 3
FA = (F1()*L2/A2+F1()*L3/A3+F2()*L3/A3)/(L1/A1+L2/A2+L3/A3)
FR3 = FA-F1()-F2()
instr = FR3*1000/A3
if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sub
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ

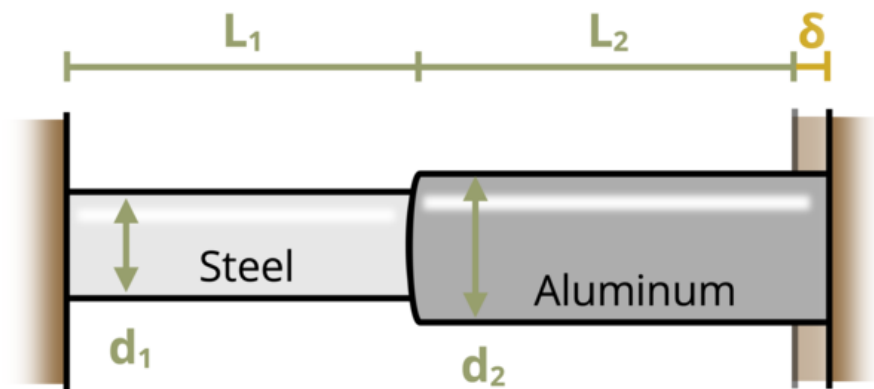
```



```
        for attempt in attempts[1:]: # Skip the first element which is the header
            await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 5.43 - Statically Indeterminate Axial Loads



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```

problem_ID="261"
delta=reactive.Value("__")
L1=reactive.Value("__")
L2=reactive.Value("__")
r1=reactive.Value("__")
r2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kip", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two circular rods are attached together and placed between two :

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        delta.set(random.randrange(1,5,1)/100)
        L1.set(random.randrange(5,20,1))
        L2.set(round(L1()*1.3,1))
        r1.set(random.randrange(20,200,5)/100)
        r2.set(round(r1()*1.5,1))

    @reactive.Effect
    @reactive.event(input.submit)

```

```

def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

    instr = delta()/(L1()/(29*10**6*r1()**2*math.pi)+L2()/(10*10**6*math.pi*r2()**2))/100
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

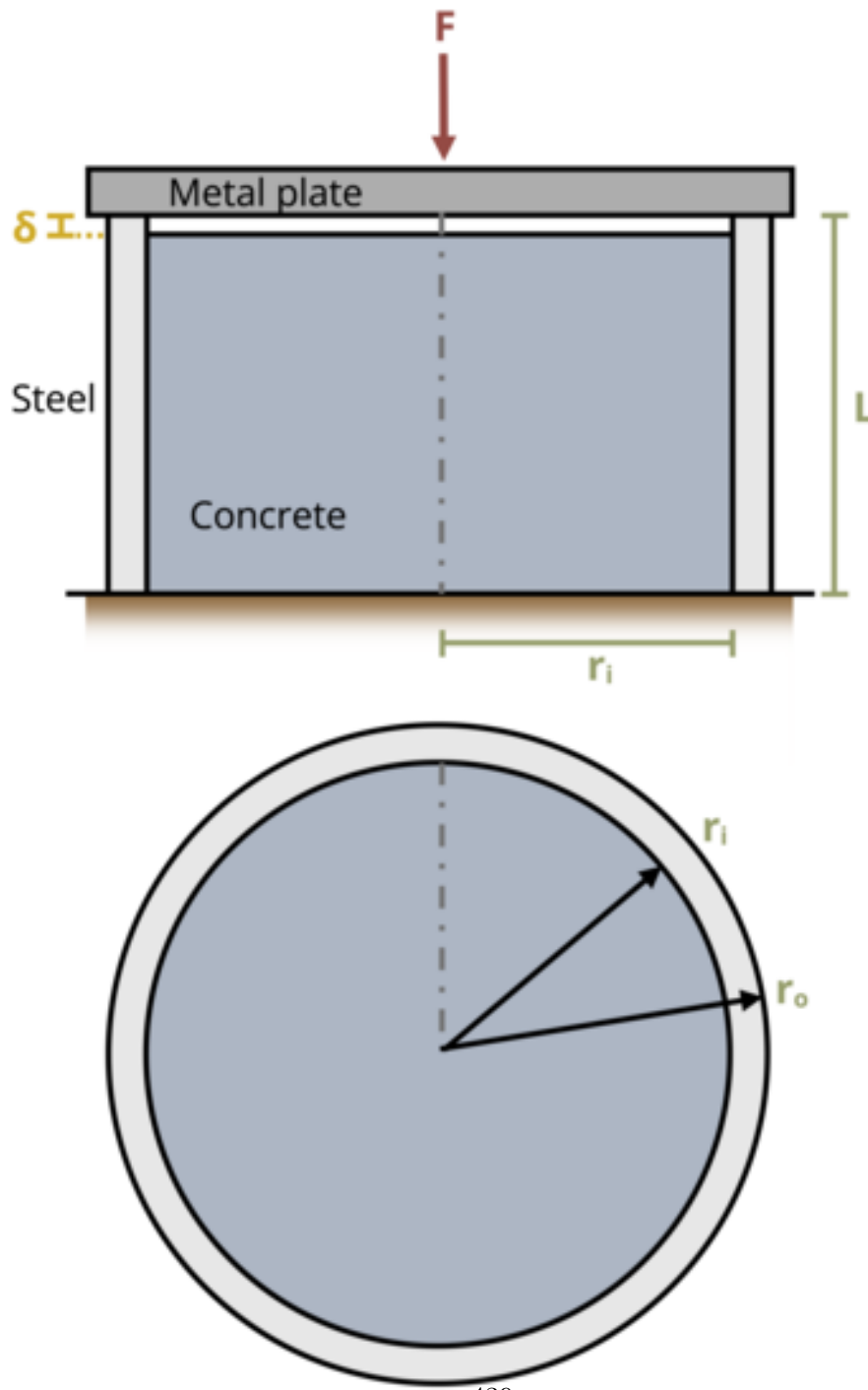
    # Write the attempts data, ensure that the header from the attempts list is not writ

```

```
        for attempt in attempts[1:]: # Skip the first element which is the header
            await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```


Problem 5.44 - Statically Indeterminate Axial Loads



439

```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="262"
ri=reactive.Value("__")
ro=reactive.Value("__")
L=reactive.Value("__")
delta=reactive.Value("__")
F=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output

```



```

@render.ui
def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A steel tube of inner radius  $r_{i}$  in., outer r

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    ri.set(random.randrange(25,50,1)/10)
    ro.set(round(ri())+random.randrange(1,5,1)/10)
    L.set(random.randrange(30,60,1))
    delta.set(random.randrange(1,5,1)/1000)
    F.set(random.randrange(100,300,10))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    As = (ro()2-ri()2)*math.pi
    Ac = ri()2*math.pi
    Ec = 4000
    Es = 29000
    Lc = L()-delta()
    Fc = (F()*L()/(As*Es)-delta())/(Lc/(Ac*Ec)+L()/(As*Es))
    Fs = F()-Fc
    instr = Fs/As
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

```

```

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

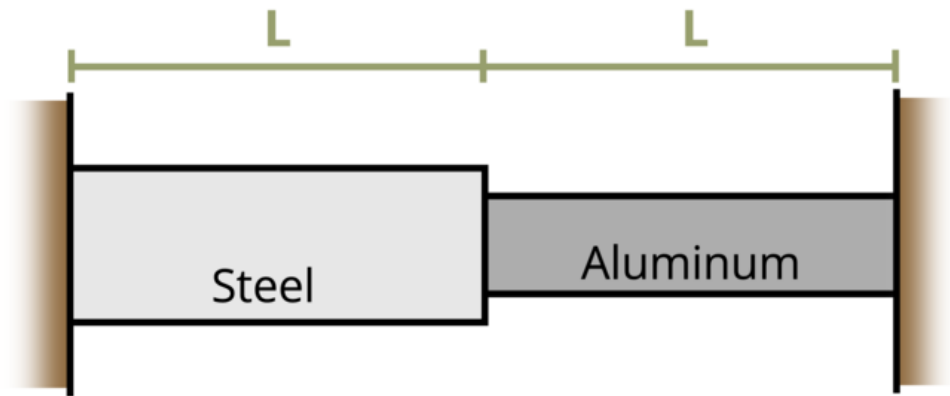
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.45 - Statically Indeterminate Axial Loads



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```

problem_ID="264"
L=reactive.Value("__")
delT=reactive.Value("__")
Aa=reactive.Value("__")
As=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two members, each of length L = {L()} m., are placed between two

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(5,50,1)/10)
        delT.set(random.randrange(50,150,1))
        Aa.set(random.randrange(100,500,10))
        As.set(Aa()*2)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s

```

```

F = (12*10**-6*L()*delT()+23*10**-6*L()*delT()/(L()/(As()*200*10**3)+L()/(Aa()*70*10**3)))/L()
instr = F/Aa()
if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

# Store the most recent encoded attempt in a reactive value so it persists across sessions
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

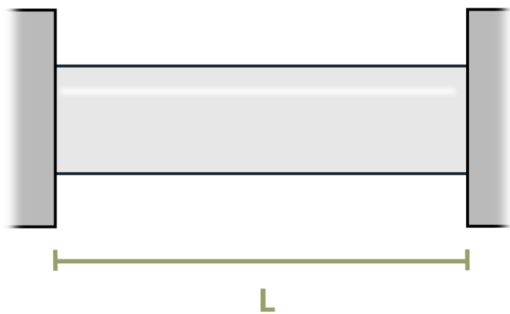
    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed

```

```
        yield attempt

# App installation
app = App(app_ui, server)
```

Problem 5.50 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC

BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="222"
stress=reactive.Value("__")
L=reactive.Value("__")
E=29000
alpha=6.5*10**-6
```



```

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sub
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

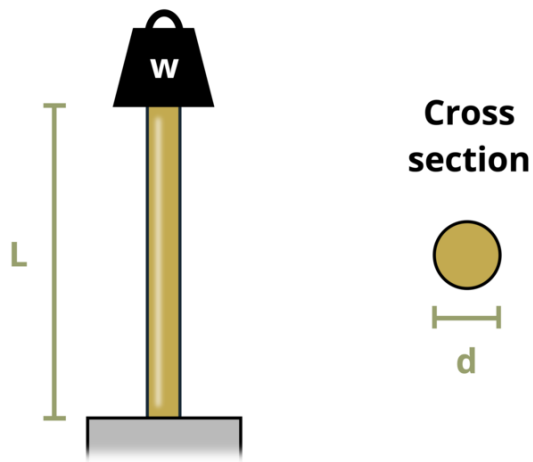
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.51 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC

BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="223"
```

```

W=reactive.Value("__")
d=reactive.Value("__")
L=reactive.Value("__")
TC=reactive.Value("__")
E=100*10**9
alpha=10*10**-6

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of mm", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"The W = {W()} kg weight is placed on a L = {L()} m tall brass b

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        W.set(random.randrange(500, 2000, 100))
        L.set(random.randrange(10, 50, 1)/10)
        d.set(random.randrange(15, 40, 1)/10)
        TC.set(random.randrange(20, 150, 5))

    @reactive.Effect

```

```

@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    deltaT = L()*alpha*TC()
    deltaM = (W()*9.81*L()/(E*math.pi*(d()/200)**2)) #200 converts from diameter in cm to
    instr= (deltaT - deltaM)*1000 # *1000 to get answer in mm
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

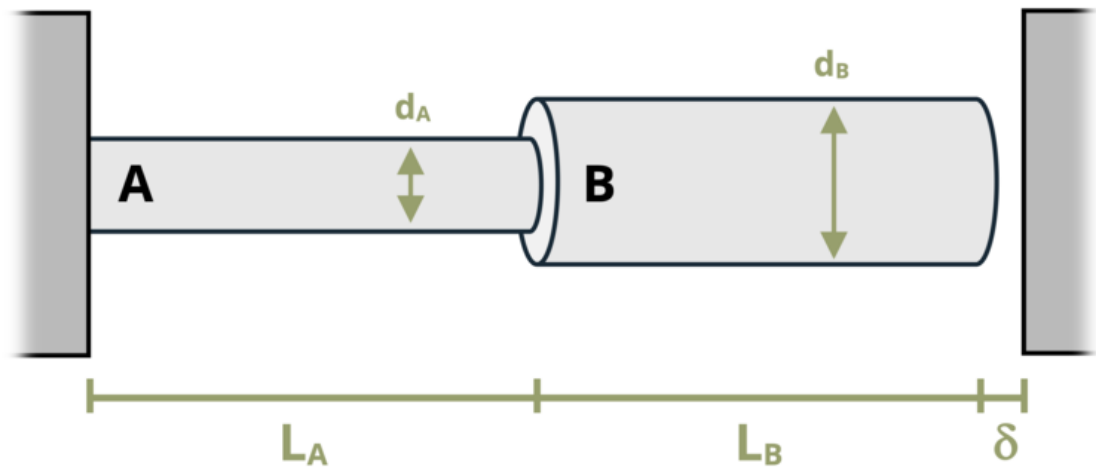
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

```

```
        # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)
```

Problem 5.52 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```

problem_ID="224"
d=reactive.Value("__")
rA=reactive.Value("__")
rB=reactive.Value("__")
L1=reactive.Value("__")
L2=reactive.Value("__")
alphaA=6*10**-6
alphaB=10*10**-6

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of °F", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two cylindrical rods are heated until they expand, just closing

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        d.set(random.randrange(1, 20, 1)/100)
        rA.set(random.randrange(5, 20, 1)/10)
        rB.set(round(rA()*1.6, 2))
        L1.set(random.randrange(5, 20, 1))
        L2.set(round(L1()*0.6, 2))

```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    instr= d()/(alphaA*L1()+alphaB*L2())
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

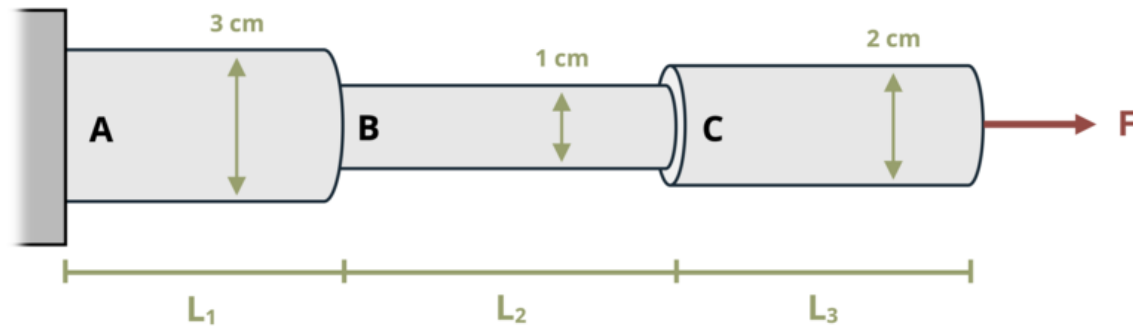
```



```
        # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)
```

Problem 5.53 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="225"
L1=reactive.Value("__")
L2=reactive.Value("__")
L3=reactive.Value("__")
F=reactive.Value("__")
```

```

dT=reactive.Value("__")
alphaA=10*10**-6
alphaB=5*10**-6
alphaC=7*10**-6
EA=40*10**9
EB=120*10**9
EC=80*10**9

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of mm", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Three cylindrical rods of lengths  $L_1 = \{L1()\}$  m,  $L_2 = \{L2()\}$  m,  $L_3 = \{L3()\}$  m,  $E = \{E()\}$  Pa,  $d = \{dT()\}$  m")]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L1.set(random.randrange(10, 40, 1)/10)
        L2.set(round((L1()*0.8),1))
        L3.set(round((L1()*2/3),1))
        F.set(random.randrange(5, 50, 1))
        dT.set(random.randrange(100, 300, 10))

```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    deltaL = (F()*L1()*1000)/(EA*math.pi*.015**2) + (F()*L2()*1000)/(EB*math.pi*.005**2)
    deltaT = alphaA*dT()*L1() + alphaB*dT()*L2() + alphaC*dT()*L3()
    instr= (deltaL + deltaT)*1000
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once

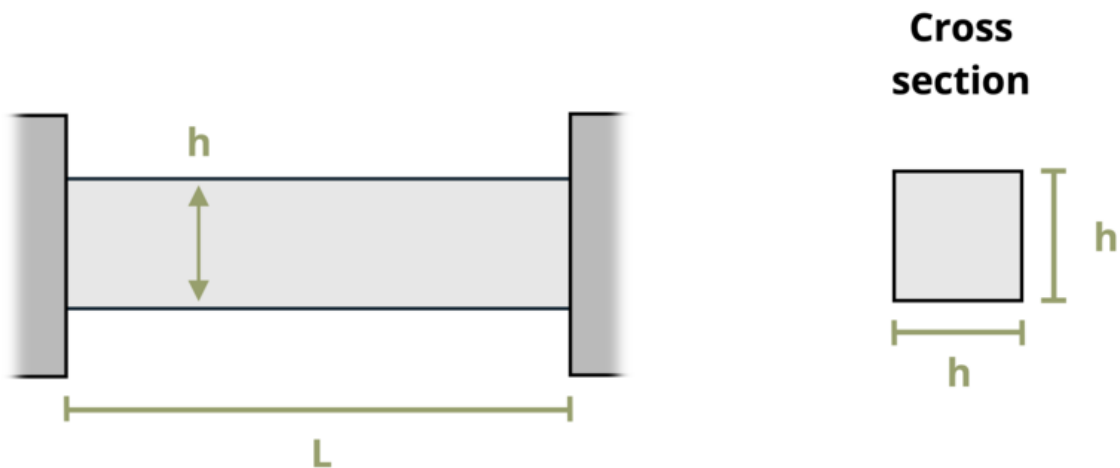
```

```
yield "Timestamp,Attempt,Answer,Feedback\n"

# Write the attempts data, ensure that the header from the attempts list is not written
for attempt in attempts[1:]: # Skip the first element which is the header
    await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
    yield attempt

# App installation
app = App(app_ui, server)
```

Problem 5.54 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="226"
```

```

T1=reactive.Value("__")
T2=reactive.Value("__")
L=reactive.Value("__")
h=reactive.Value("__")
alpha=20*10**-6
E=100*10**9

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A square brass bar is placed between two fixed walls and heated

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        T1.set(random.randrange(5, 20, 1))
        T2.set(round(T1() + random.randrange(20, 50, 1), 2))
        L.set(random.randrange(250, 750, 10))
        h.set(random.randrange(10, 30, 1))

    @reactive.Effect

```

```

@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each
    instr= (E*alpha*(T2()-T1()))/10**6
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ

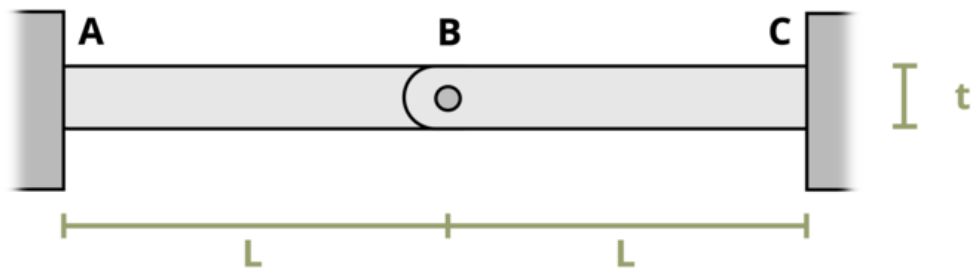
```



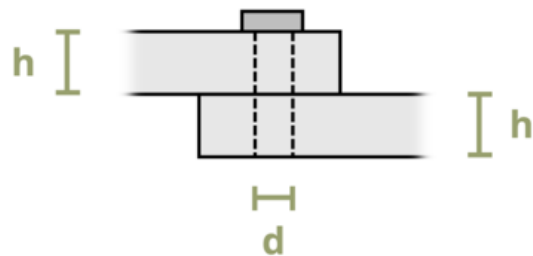
```
        for attempt in attempts[1:]: # Skip the first element which is the header
            await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 5.55 - Thermal Deformation



Bar cross section



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
```

```

from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="249"
L=reactive.Value("__")
d=reactive.Value("__")
t=reactive.Value("__")
h=reactive.Value("__")
dT=reactive.Value("__")
E=200
v=0.32
a=11.7*10**-6

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Bars AB and BC are pinned at joint B. Both bars are made from tl

    #@reactive.Effect
    #@reactive.event(input.generate_problem)

```

```

def randomize_vars():
    random.seed(101)
    L.set(random.randrange(300, 900, 10))
    t.set(random.randrange(30, 80, 1))
    h.set(t()/2)
    d.set(round(t()/3,2))
    dT.set(random.randrange(10, 40, 1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    deltaM = a*dT()*(t()/1000)*(h()/1000)*E*10**9
    instr= (deltaM/(math.pi*(d()/2000)**2))/10**6
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

```

```

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

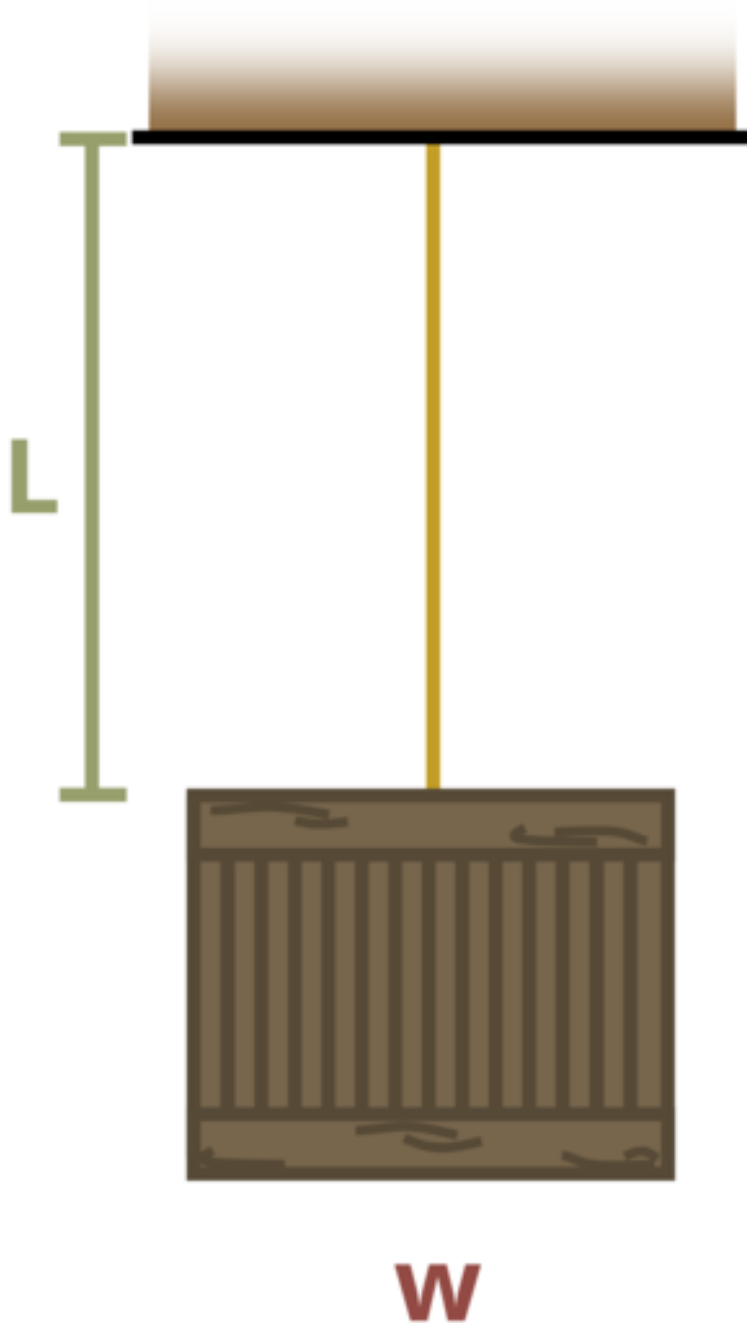
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written again
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 5.56 - Thermal Deformation



```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="227"
W=reactive.Value("__")
L=reactive.Value("__")
A=reactive.Value("__")
T1=reactive.Value("__")
T2 =reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of inches", placeholder="Please enter your a
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

```



```

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

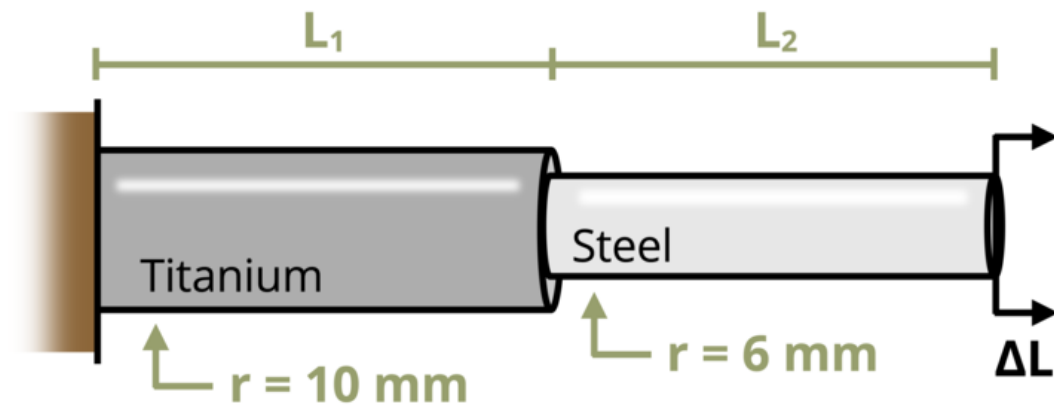
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.57 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="228"
L2=reactive.Value("__")
L1=reactive.Value("__")
```



```

        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

    # Store the most recent encoded attempt in a reactive value so it persists across sessions
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

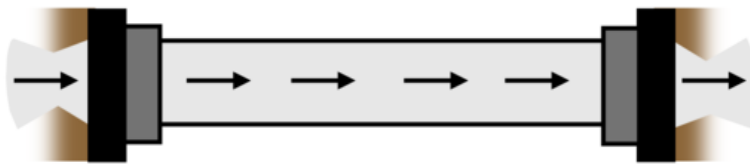
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written again
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

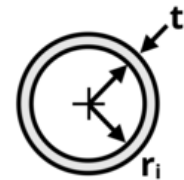
# App installation
app = App(app_ui, server)

```

Problem 5.58 - Thermal Deformation



Section view



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import math
import io
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="229"
ri=reactive.Value("__")
t=reactive.Value("__")
T1=reactive.Value("__")
T2=reactive.Value("__")
L=reactive.Value("__")
```



```

else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

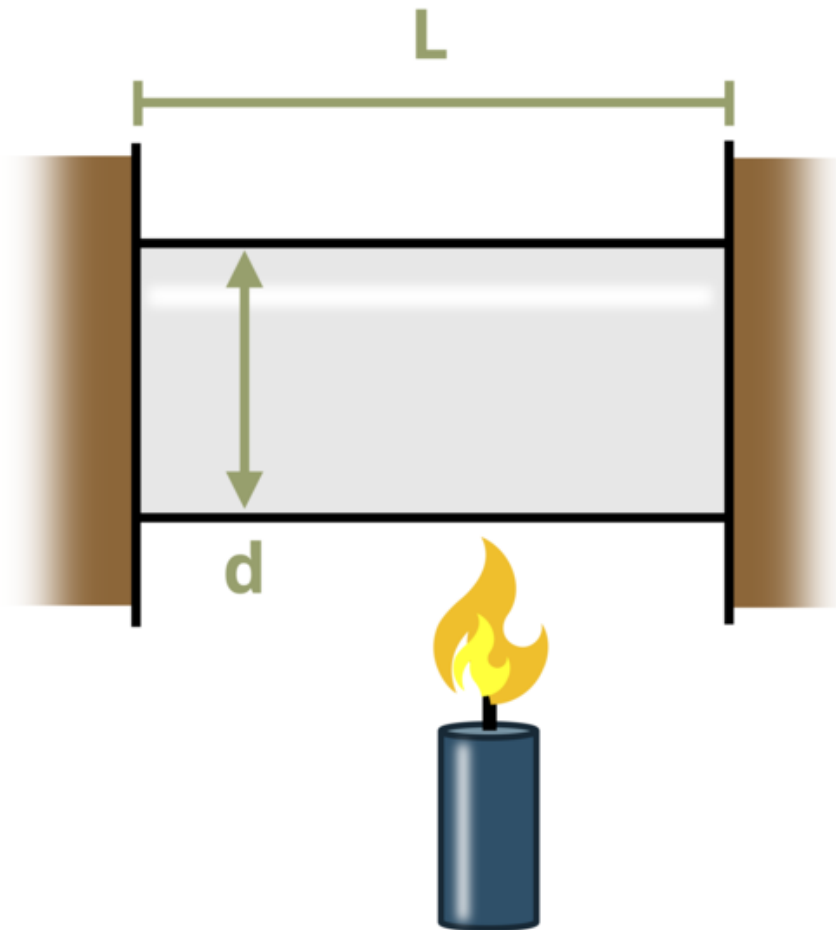
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 5.59 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true  
#| viewerHeight: 600  
#| components: [viewer]
```

```

from shiny import App, render, ui, reactive
import random
import asyncio
import math
import io
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="230"
L=reactive.Value("__")
d=reactive.Value("__")
T1=reactive.Value("__")
T2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kip", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"An aluminum cylinder of length L = {L()} in. and diameter d = {d()} in.")]

```

```

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    L.set(random.randrange(5,20,1))
    d.set(random.randrange(15,50,1)/10)
    T1.set(random.randrange(20,50,1))
    T2.set(T1()+random.randrange(30,100,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    delT = 13*10**(-6)*(T2()-T1())*L()
    instr = delT*(math.pi*(d()/2)**2)*10*10**6/L()/1000
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

```

```

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

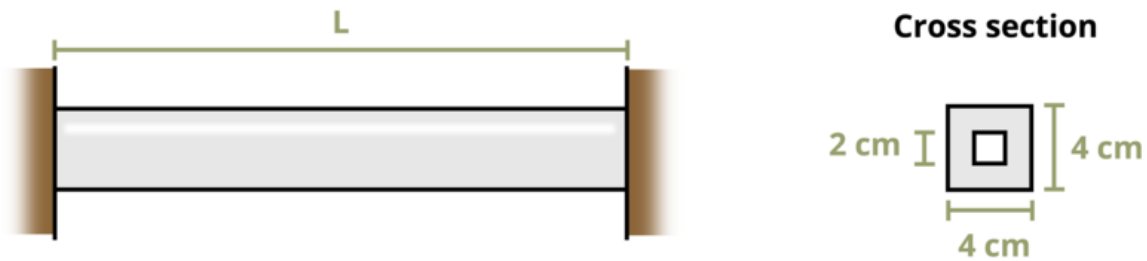
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.60 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import math
import io
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="231"
L=reactive.Value("__")
T1=reactive.Value("__")
T2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]
```

```

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A square PVC plastic rod of length L = {L()} mm is placed between

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(500,1500,10))
        T1.set(random.randrange(0,30,1))
        T2.set(T1()+random.randrange(80,150,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        instr = 2800*50*10**(-6)*(T2()-T1())
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.

```

```

random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

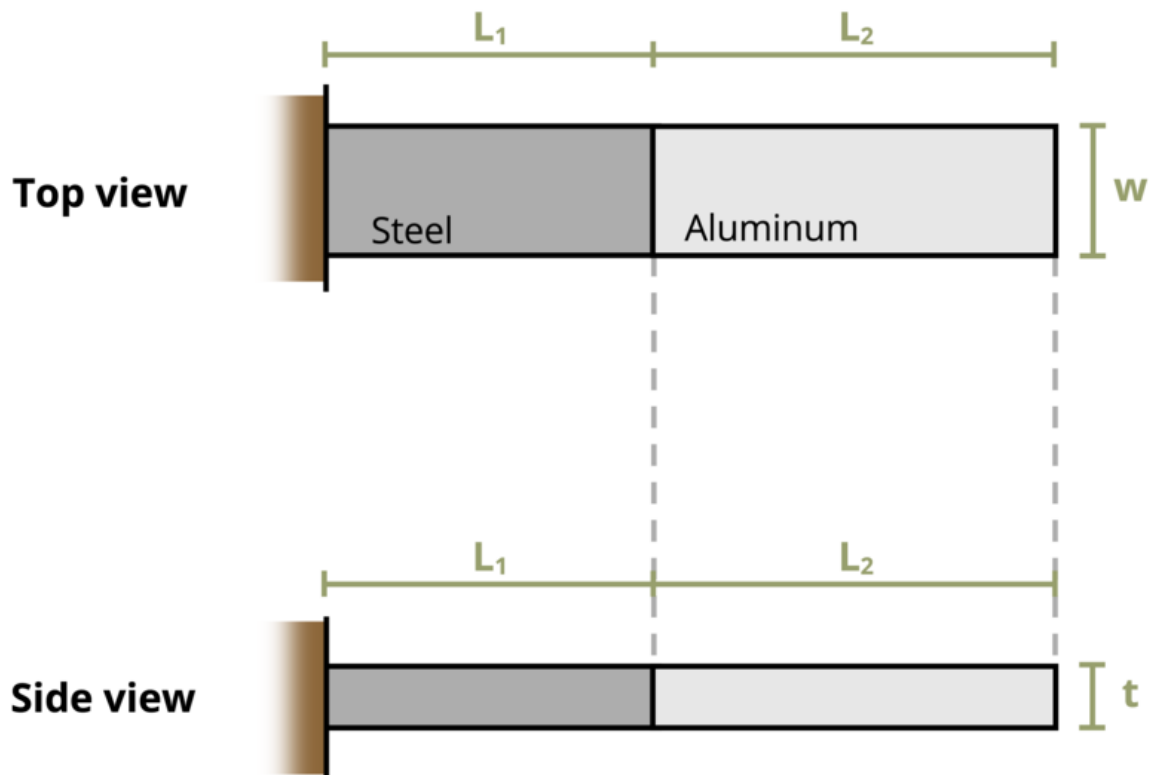
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.61 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import math
import io
```



```

import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="232"
L1=reactive.Value("__")
L2=reactive.Value("__")
delT=reactive.Value("__")
t=reactive.Value("__")
w=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of in", placeholder="Please enter your answe
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two solid bars of length  $L_1$  in. and  $L_2$ 

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)

```

```

L1.set(random.randrange(50,100,1)/10)
L2.set(L1()+random.randrange(20,50,1)/10)
delT.set(random.randrange(100,300,1))
t.set(random.randrange(10,20,1)/10)
w.set(t()*2)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    instr = (L1()*6.5*10**-6+L2()*13*10**-6)*delT()
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)

```

```

    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

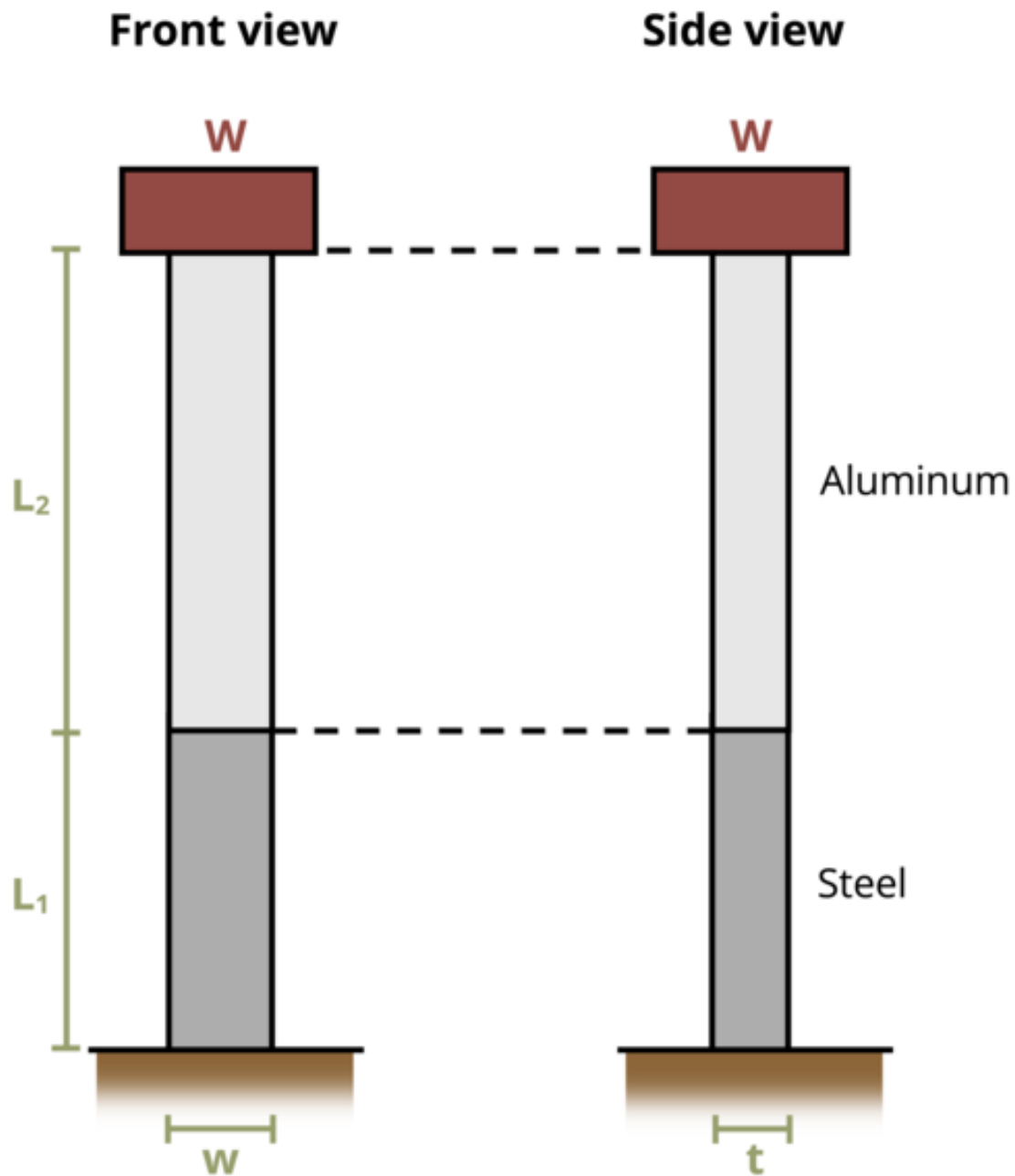
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 5.62 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import math
import io
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="233"
L1=reactive.Value("__")
L2=reactive.Value("__")
t=reactive.Value("__")
w=reactive.Value("__")
delT=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kip", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output

```

```

@render.ui
def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"Two solid bars of length  $L_1$  in. and  $L_2$ 

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    L1.set(random.randrange(50,100,1)/10)
    L2.set(L1()+random.randrange(20,50,1)/10)
    t.set(random.randrange(10,20,1)/10)
    w.set(t()*2)
    delT.set(random.randrange(50,100,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    dTherm = L1()*6.5*10**-6*delT()+L2()*13*10**-6*delT()
    dM = (L1()/29000+L2()/10000)/(w()*t())
    instr = dTherm/dM
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.

```

```

        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

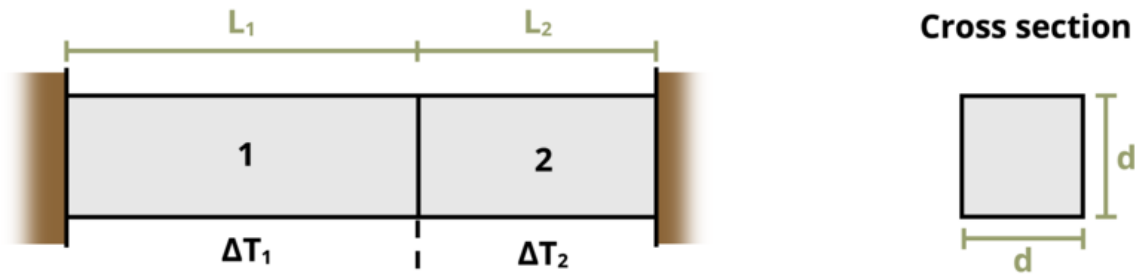
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 5.63 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import math
import io
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="235"
L2=reactive.Value("__")
L1=reactive.Value("__")
delT1=reactive.Value("__")
delT2=reactive.Value("__")
```

```

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A square steel 1 inch bar is placed between two fixed walls. Th

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L2.set(random.randrange(40,100,1)/10)
        L1.set(L2()+random.randrange(20,60,1)/10)
        delT1.set(random.randrange(50,100,1))
        delT2.set(delT1()+random.randrange(50,100,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        dTherm = 6.5*10**-6*(L1()*delT1()+L2()*delT2())
        instr = dTherm*29000/(L1()+L2())
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:

```

```

        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

    # Store the most recent encoded attempt in a reactive value so it persists across sessions
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

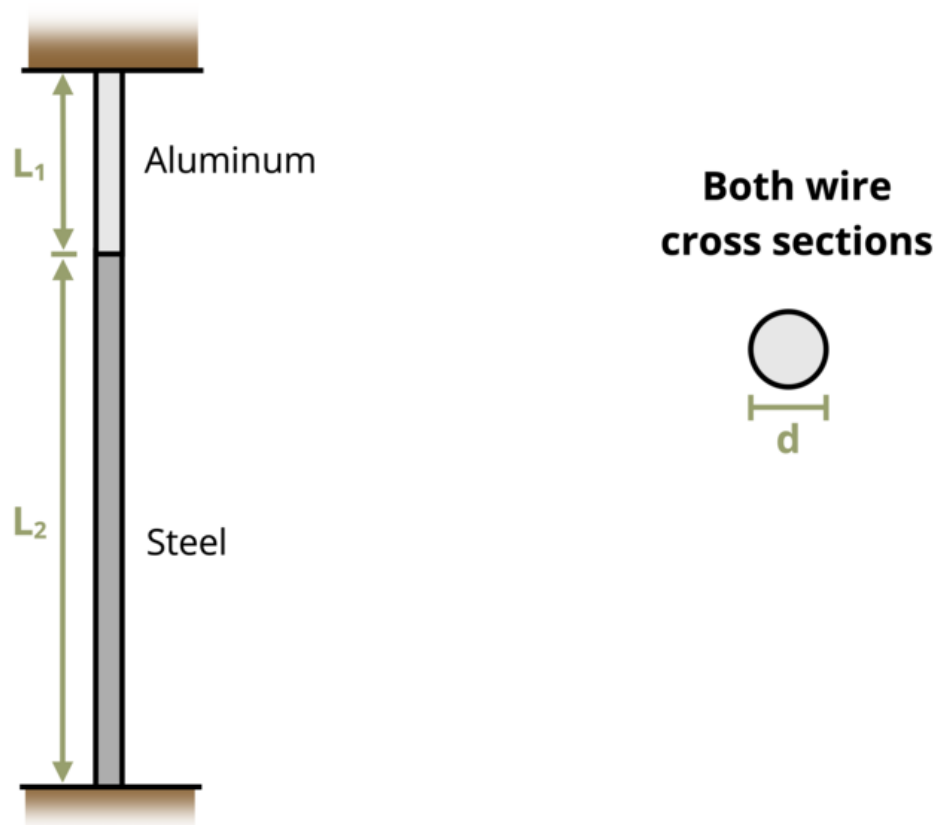
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written again
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.64 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import math
```

```

import io
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="236"
L1=reactive.Value("__")
L2=reactive.Value("__")
delT=reactive.Value("__")
d=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two wires of length  $L_1 = \{L1()\}$  m and  $L_2 =$ 

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)

```

```

L1.set(random.randrange(10,50,1)/10)
L2.set(L1()*3)
delT.set(random.randrange(20,50,1))
d.set(random.randrange(5,10,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    A = (d()/2)**2*math.pi*10**-6
    dTherm = 12*10**-6*delT()*L2()+23*10**-6*delT()*L1()
    F = dTherm/(L1()/(70*10**9*A)+L2()/(190*10**9*A))
    instr = F/A/10**6
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")

```

```

async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

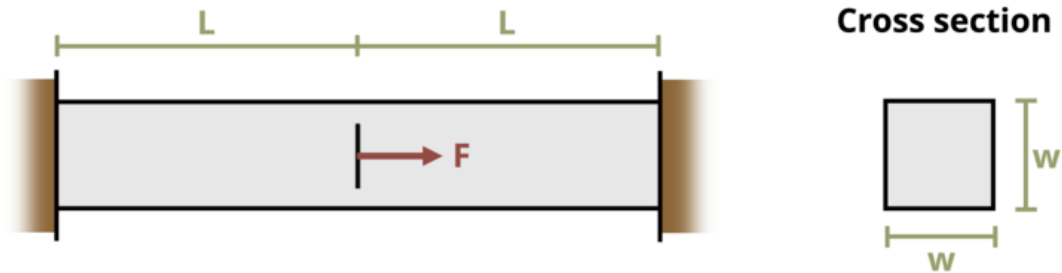
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.65 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import math
import io
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="237"
w=reactive.Value("__")
delT=reactive.Value("__")
F=reactive.Value("__")
L=reactive.Value("__")
```



```

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kN", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A square steel bar with cross-section side length w = {w()} mm :

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        w.set(random.randrange(10,50,1))
        delT.set(random.randrange(250,500,10))
        F.set(random.randrange(50,200,1))
        L.set(random.randrange(10,50,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
        dTherm = 12*10**-6*2*L()*delT()
        FLR = dTherm/(L()/(200*10**9*w()*w()*10**-6))*10**-3
        instr = (FLR+F())/2
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"

```

```

else:
    check = "*Not Correct*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

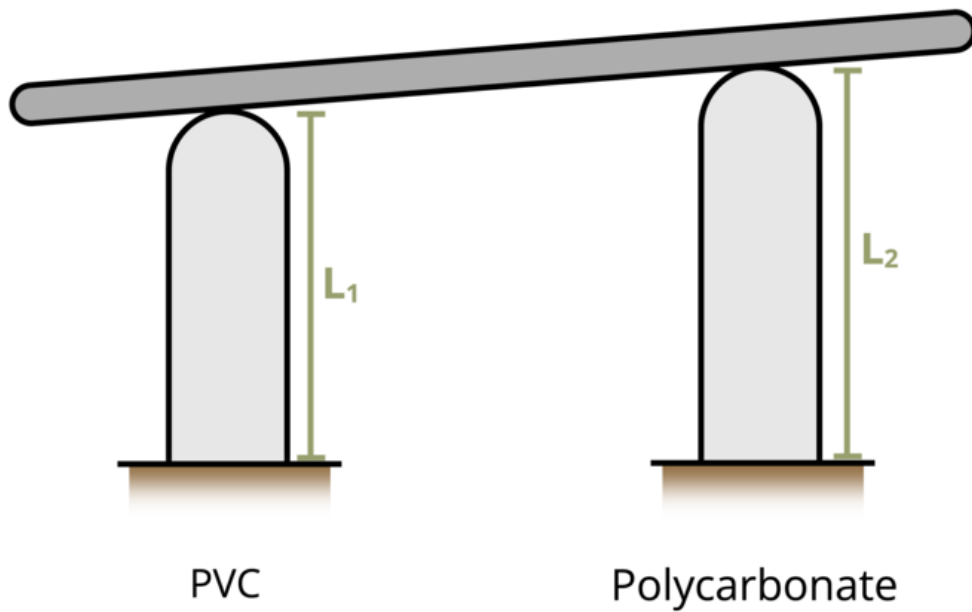
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.66 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import math
import io
import string
from datetime import datetime
from pathlib import Path
```

```

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="238"
L1=reactive.Value("__")
L2=reactive.Value("__")
aPVC=reactive.Value("__")
aPoly=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of \u00b0C", placeholder="Please enter your
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two posts support a slender bar as shown, where  $L_1$  =

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L1.set(random.randrange(30,50,1)/1000)
        L2.set(L1()+random.randrange(1,3,1)/1000)
        aPVC.set(random.randrange(40,60,1))
        aPoly.set(random.randrange(60,100,1)/10)

```

```

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    instr = (L2()-L1())/(aPVC()*10**-6*L1()-aPoly()*10**-6*L2())
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

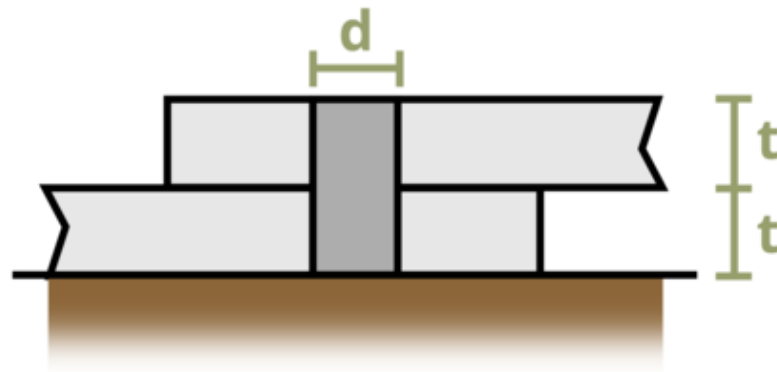
```

```
        # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

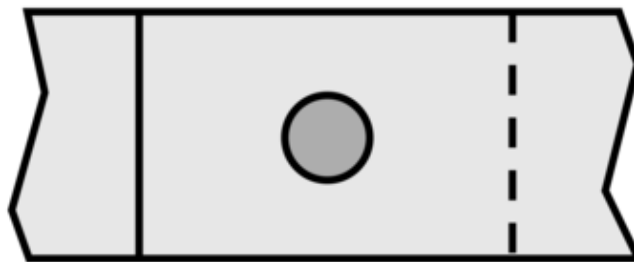
# App installation
app = App(app_ui, server)
```

Problem 5.67 - Thermal Deformation

Side view



Top view



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import math
import io
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="239"
t=reactive.Value("__")
d=reactive.Value("__")
delT=reactive.Value("__")
Epin=reactive.Value("__")
apin=reactive.Value("__")
vpin=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of in", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

```



```

@output
@render.ui
def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"Two rigid plates of thickness  $t = {t()}$  in. are held together w

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    t.set(random.randrange(30,80,1)/100)
    d.set(random.randrange(30,80,1)/100)
    delT.set(random.randrange(50,150,1))
    Epin.set(random.randrange(800,1200,10))
    apin.set(random.randrange(80,120,1)/10)
    vpin.set(random.randrange(35,45,1)/100)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    dzT = apin()*10**-6*delT()*2*t()
    dxT = apin()*10**-6*delT()*d()
    exM = dxT/d()
    ezM = vpin()*2*exM/(1-vpin())
    dzM = ezM*2*t()
    instr = dzT+dzM
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

```

```

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.68 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="240"
L=reactive.Value("__")
d=reactive.Value("__")
delta=reactive.Value("__")
```

```

delT = reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem**"),
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of mm", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A circular bar of length L = {L()} m and diameter d = {d()} mm")]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(30,80,1)/10)
        d.set(random.randrange(50,100,1))
        delta.set(random.randrange(5,50,1)/10)
        delT.set(random.randrange(150,350,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each submit
        dTherm = 13.3*10**-6*delT()*L()
        sigmax = (delta()/1000-dTherm)/(L()/3000)
        ey = -0.32*sigmax/3000+13.3*10**-6*delT()
        instr = ey*d()

```

```

if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across sul
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

```

```
# App installation
app = App(app_ui, server)
```

Problem 5.69 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="241"
L=reactive.Value("__")
d=reactive.Value("__")
delT = reactive.Value("__")
```

```

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate your problem statement**"),
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your answer"),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A circular bar of length L = {L()} in. and diameter d = {d()} in.")]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(10,30,1))
        d.set(random.randrange(3,20,1)/10)
        delT.set(random.randrange(150,350,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each submit
        dTherm = 7*10**-6*delT()*L()
        instr = (dTherm-0.01/1000)/(L()/29000)
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"

```



```

        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

    # Store the most recent encoded attempt in a reactive value so it persists across submissions
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else ""
    yield f"{final_encoded}\n\n"

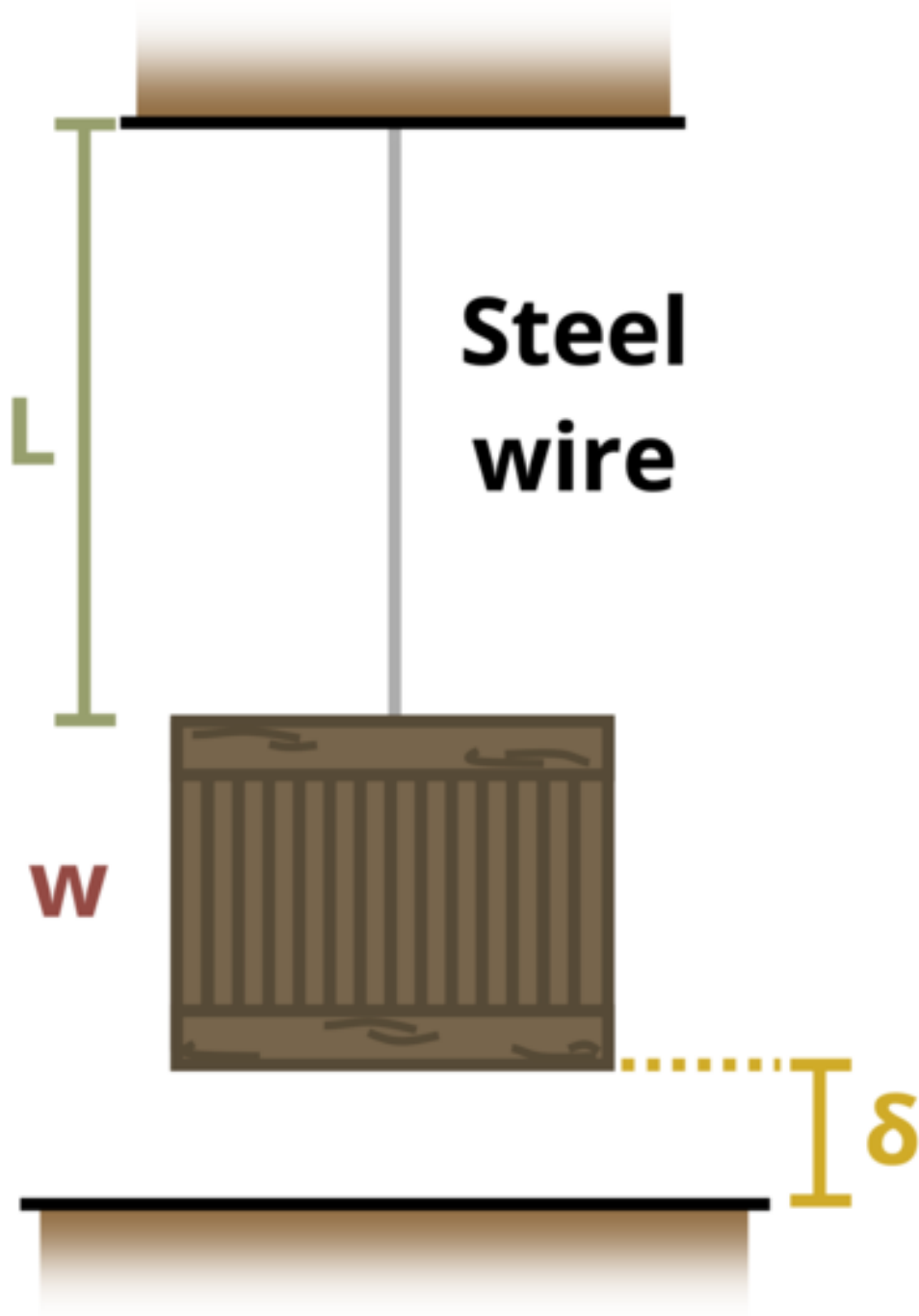
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written again
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 5.70 - Thermal Deformation



```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="242"
W=reactive.Value("__")
d=reactive.Value("__")
L=reactive.Value("__")
delta=reactive.Value("__")
T1=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of \u00b0C", placeholder="Please enter your
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output

```

```

@render.ui
def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A crate weighing  $W = \{W()\}$  kN is hung from a long steel cable of

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    W.set(random.randrange(50,100,1))
    d.set(random.randrange(50,100,1))
    L.set(random.randrange(50,100,1)/10)
    delta.set(random.randrange(5,10,1))
    T1.set(random.randrange(15,30,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each
    dLoad = W()*1000*L()/((d()/1000/2)**2*math.pi*190*10**9)*1000
    instr = (delta()-dLoad+T1()*6.5*10**-6*L()*1000)/(6.5*10**-6*L()*1000)
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")

```

```

        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

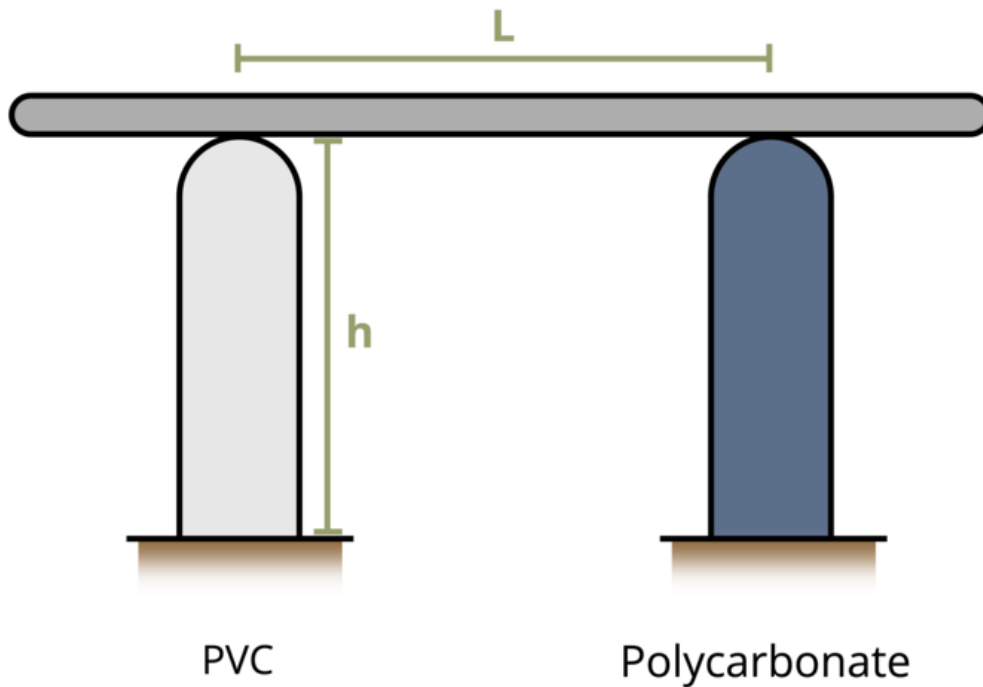
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.71 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
```

```

from pathlib import Path
from math import atan, degrees

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="243"
L=reactive.Value("__")
h=reactive.Value("__")
delT=reactive.Value("__")
apoly=reactive.Value("__")
apvc=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of °", placeholder="Please enter your answer
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A rigid bar is supported by two posts made from PVC and polycarl

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(50,100,1))

```



```

h.set(L()+random.randrange(15,40,1))
delT.set(random.randrange(50,100,1))
apoly.set(random.randrange(6,10,1)/10)
apvc.set(random.randrange(40,60,1))

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    delpvc = apvc()*10**-6*delT()*L()
    delpoly = apoly()*10**-6*delT()*L()
    height = delpvc-delpoly
    instr = degrees(atan(height/L()))
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")

```

```

async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

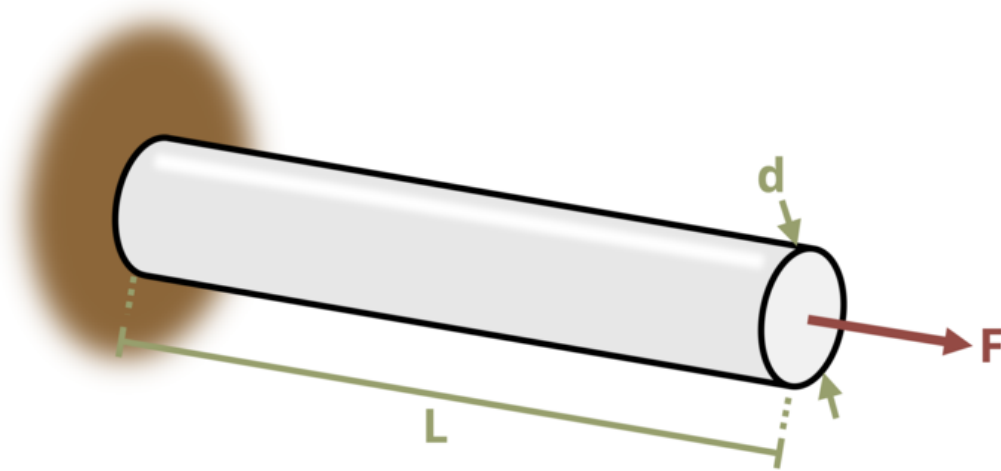
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```

Problem 5.72 - Thermal Deformation



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#!/ standalone: true
#!/ viewerHeight: 600
#!/ components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```

problem_ID="244"
L=reactive.Value("__")
d=reactive.Value("__")
delT=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
  #ui.markdown("**Please enter your ID number from your instructor and click to generate y
  #ui.input_text("ID","", placeholder="Enter ID Number Here"),
  #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
  ui.markdown("**Problem Statement**"),
  ui.output_ui("ui_problem_statement"),
  ui.input_text("answer","Your Answer in units of kip", placeholder="Please enter your ans
  ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
  #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
  # Initialize a counter for attempts
  attempt_counter = reactive.Value(0)

  @output
  @render.ui
  def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A brass bar of length L = {L()} in. and diameter d = {d()} in.

  #@reactive.Effect
  #@reactive.event(input.generate_problem)
  def randomize_vars():
    random.seed(101)
    L.set(random.randrange(10,30,1))
    d.set(random.randrange(10,50,1)/10)
    delT.set(random.randrange(50,100,1))

  @reactive.Effect
  @reactive.event(input.submit)
  def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    dTherm = d()*11*10**-6*delT()
    dP = 0.3/((d()/2)**2*math.pi)/15000/d()

```

```

instr = dTherm/dP
if math.isclose(float(input.answer()), instr, rel_tol=0.01):
    check = "*Correct*"
    correct_indicator = "JL"
else:
    check = "*Not Correct.*"
    correct_indicator = "JG"

# Generate random parts for the encoded attempt.
random_start = generate_random_letters(4)
random_middle = generate_random_letters(4)
random_end = generate_random_letters(4)
#encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

# Store the most recent encoded attempt in a reactive value so it persists across su
#session.encoded_attempt = reactive.Value(encoded_attempt)

# Append the attempt data to the attempts list without the encoded attempt
attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

# Show feedback to the user.
feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
m = ui.modal(
    feedback,
    title="Feedback",
    easy_close=True
)
ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"

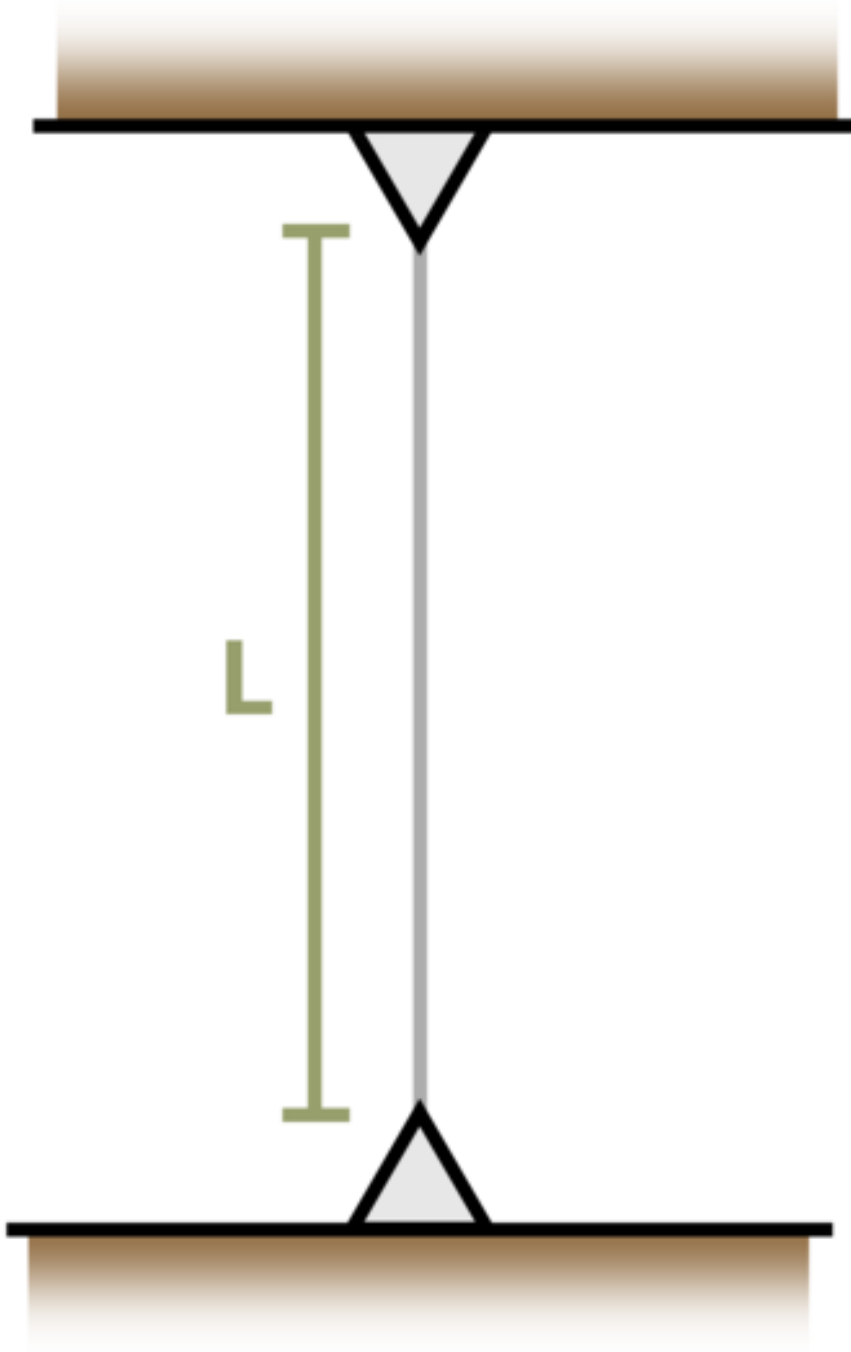
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not writ
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

```

```
# App installation
app = App(app_ui, server)
```


Problem 5.73 - Thermal Deformation




```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="253"
L=reactive.Value("__")
d=reactive.Value("__")
delT=reactive.Value("__")
E=reactive.Value("__")
alpha=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output

```

```

@render.ui
def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A cable of length L = {L()} m and diameter d = {d()} mm is conn

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    L.set(random.randrange(20,100,1)/10)
    d.set(random.randrange(10,30,1))
    delT.set(random.randrange(50,150,1))
    E.set(random.randrange(175,225,1))
    alpha.set(random.randrange(80,130,1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each s
    instr = alpha()*10**-6*delT()*E()*10**3
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

    # Store the most recent encoded attempt in a reactive value so it persists across sul
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(

```

```

        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

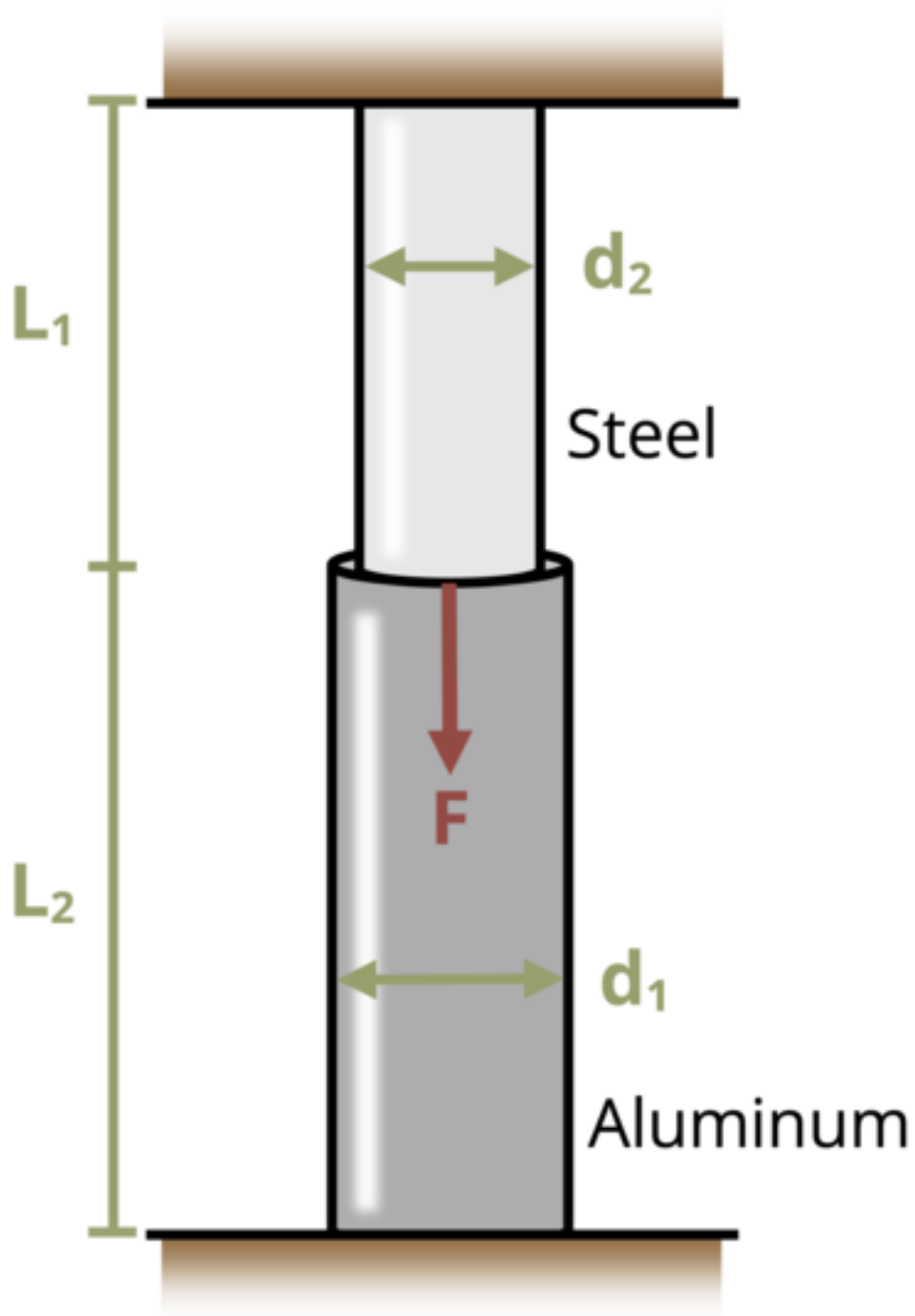
    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```


Problem 5.74 - Thermal Deformation



```

#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="260"
delT=reactive.Value("__")
F=reactive.Value("__")
L1=reactive.Value("__")
L2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui

```

```

def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"Two circular rods are placed between two fixed walls as shown.")]

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    delT.set(random.randrange(100,200,1))
    F.set(random.randrange(10,50,1))
    L1.set(random.randrange(5,20,1))
    L2.set(L1()*1.3)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1) # Increment the attempt counter on each submit
    dTs = L1()*6.5*10**-6*delT()
    dTal = L2()*13*10**-6*delT()
    Fs = (dTs+dTal-(F()*1000*L2()/(10*10**6*0.75**2*math.pi)))/(L1()/(29*10**6*0.5**2*math.pi))
    Fal = Fs+F()*1000
    instr = Fal/(0.75**2*math.pi)/1000
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{correct_indicator}"

    # Store the most recent encoded attempt in a reactive value so it persists across sessions
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n")

    # Show feedback to the user.

```

```

        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else None
    yield f"{final_encoded}\n\n"

    # Write the header for the remaining CSV data once
    yield "Timestamp,Attempt,Answer,Feedback\n"

    # Write the attempts data, ensure that the header from the attempts list is not written
    for attempt in attempts[1:]: # Skip the first element which is the header
        await asyncio.sleep(0.25) # This delay may not be necessary; adjust as needed
        yield attempt

# App installation
app = App(app_ui, server)

```